Complete

C# Interview Questions and Answers

500 Questions

Across 20 topic areas — from language basics and OOP to async, LINQ, ASP.NET Core and EF Core.

Difficulty levels: Beginner · Intermediate · Advanced

500 curated questions

November 2026

Contents

C# Basics & Syntax 3

Object-Oriented Programming 9

Value Types & Reference Types 15

Collections & Data Structures 21

Generics 27

Delegates, Events & Lambdas 33

LINQ 39

Exception Handling 45

Async & Await 51

Memory Management & Garbage Collection 57

Strings & Text 63

File I/O & Serialization 69

Threading & Concurrency 75

Reflection & Attributes 82

Modern C# Features 89

Design Patterns 96

Testing & Best Practices 104

.NET Platform & Tooling 112

ASP.NET Core & Web 120

Entity Framework & Data Access 128

C# Basics & Syntax

25 questions · 25 Beginner

1. What is C# and what kind of language is it?

Beginner | #language #overview | reference

C# is a modern, object-oriented, statically typed language created by Anders Hejlsberg at Microsoft and first released in 2002. It runs on the .NET runtime (CLR), compiles to Intermediate Language (IL), and is designed for productivity, type safety, and versioning. It is multi-paradigm, supporting object-oriented, functional, generic, and (in recent versions) pattern-based styles.

2. What is the difference between C# and .NET?

Beginner | #language #platform | reference

C# is a programming language; .NET is the platform and runtime that executes it. .NET provides the CLR (execution engine), the Base Class Library (BCL), and tooling. You can compile C# into IL that runs on .NET, but .NET also supports other languages such as F# and Visual Basic.

3. What is the entry point of a C# program?

Beginner | #entry-point #main | reference

The entry point is the Main method, which the runtime calls first. It can be declared as static void Main(), static int Main(), or with a string[] args parameter, and in modern C# you can use top-level statements to omit the explicit Main method.

```
public static void Main(string[] args)
{
    Console.WriteLine("Hello, C#");
}
```

4. What is the difference between Console.WriteLine and Console.Write?

Beginner | #console #io | reference

Console.WriteLine writes the value followed by a line terminator; Console.Write writes the value without a trailing newline. Use Write when you want subsequent output to continue on the same line.

```
Console.Write("A");
Console.Write("B"); // prints AB on one line
Console.WriteLine("C"); // then newline
```

5. What are C# keywords and identifiers?

Beginner | #syntax #keywords | reference

Keywords are reserved words with special meaning to the compiler, such as class, int, if, and return. Identifiers are the names you give to types, members, and variables; they cannot be keywords unless prefixed with @ (for example, @class).

6. What is the difference between var and an explicit type?

Beginner | #var #type-inference | reference

var tells the compiler to infer the type from the initializer at compile time, so the variable is still statically typed. There is no runtime penalty. Use explicit types when it improves readability, and note that var requires an initializer and cannot be used for fields without an initializer.

```
var count = 10;        // inferred as int
int total = 10;        // explicit
// var x;             // error: needs initializer
```

7. What is a namespace and why is it used?

Beginner | #namespaces #organization | reference

A namespace groups related types into a logical scope and prevents name collisions. You reference members with a fully qualified name or bring them into scope with a using directive. Since C# 10 you can use file-scoped namespaces to reduce indentation.

```
namespace MyApp.Data; // file-scoped

public class Customer { }
```

8. What are the C# comment styles?

Beginner | #comments #documentation | reference

C# supports single-line comments with //, multi-line comments with /* */, and XML documentation comments with ///. XML comments are compiled into documentation and are surfaced by IntelliSense.

```
// single line
/* multi
   line */
/// <summary>Adds two numbers.</summary>
int Add(int a, int b) => a + b;
```

9. What are the common numeric data types in C# and their sizes?

Beginner | #types #numbers | reference

Integer types include sbyte (8-bit), byte (8-bit), short (16-bit), ushort (16-bit), int (32-bit), uint (32-bit), long (64-bit), and ulong (64-bit). Floating types are float (32-bit), double (64-bit), and decimal (128-bit, base-10). char is 16-bit Unicode.

```
int i = 42;
long l = 9_000_000_000L;
double d = 3.14;
decimal m = 19.99m;
char c = 'A';
```

10. What is the difference between float, double, and decimal?

Beginner | #types #precision #decimal | reference

float and double are IEEE-754 binary floating-point types with approximate precision; double is the default for real literals. decimal is a base-10 type with 28-29 significant digits and is preferred for money because it avoids binary rounding artifacts. decimal is slower and has a smaller range than double.

```
double a = 0.1 + 0.2;      // 0.30000000000000004
decimal b = 0.1m + 0.2m;    // 0.3
```

11. What is the difference between implicit and explicit conversion?

Beginner | #conversion #casting | reference

An implicit conversion happens automatically when there is no data loss, such as int to long. An explicit conversion (a cast) is required when information could be lost, such as double to int, and you accept the risk of truncation or overflow.

```
int i = 5;
long l = i;        // implicit
double d = 5.9;
int t = (int)d;    // explicit, t == 5
```

12. What is boxing and unboxing?

Beginner | #boxing #value-types #performance | reference

Boxing converts a value type into an object (or interface) by allocating a heap object that wraps the value. Unboxing extracts the value from a boxed object with an explicit cast. Both have performance costs, so prefer generics to avoid them in hot paths.

```
int x = 42;
object boxed = x;      // boxing
int y = (int)boxed;    // unboxing
```

13. What are nullable value types?

Beginner | #nullable #value-types | reference

A nullable value type, written T?, wraps a value type so it can also hold null. It is shorthand for Nullable. You inspect the value with HasValue or Value, or use helpers such as GetValueOrDefault and the null-coalescing operators.

```
int? age = null;
if (age.HasValue)
    Console.WriteLine(age.Value);
int shown = age ?? 0;
```

14. What is the difference between const and readonly?

Beginner | #const #readonly #immutability | reference

const is a compile-time constant that must be initialized at declaration and is baked into callers at compile time. readonly is set at declaration or in a constructor, is evaluated at runtime, and can hold reference types whose contents may still change. Use static readonly for values that are fixed after initialization but not compile-time constants.

```
        public const double Pi = 3.14159;
        public static readonly DateTime Start = DateTime.UtcNow;
```

15. What are the C# operators for arithmetic and how does integer division work?

Beginner | #operators #arithmetic | reference

C# provides +, -, *, /, and % (remainder). When both operands are integers, / performs integer division and discards the remainder; cast one operand to a floating type to get a fractional result. Division by zero throws for integers and yields Infinity or NaN for floating types.

```
        int a = 7 / 2;          // 3
        double b = 7 / 2.0;     // 3.5
        int r = 7 % 2;          // 1
```

16. What are the C# logical and comparison operators?

Beginner | #operators #logic | reference

Comparison operators are ==, !=, <, >, <=, and >=. Logical operators are && (short-circuit AND), || (short-circuit OR), ! (NOT), & and | (non-short-circuit bitwise/logical). Use && and || when you want to avoid evaluating the right operand unnecessarily.

```
        if (x > 0 && y > 0) { }
        if (name != null && name.Length > 0) { }
```

17. What is the conditional (ternary) operator?

Beginner | #operators #ternary | reference

The ternary operator ?: evaluates a condition and returns one of two expressions. It is an expression, so it can be assigned or returned directly, unlike an if statement.

```
        string label = score >= 60 ? "Pass" : "Fail";
```

18. What is the null-coalescing operator and the null-coalescing assignment operator?

Beginner | #operators #null | reference

?? returns the left operand if it is not null, otherwise the right operand. ??= assigns the right operand to the left variable only if the left is null. Both work with nullable value types and reference types.

```
        string name = input ?? "Unknown";
        input ??= "default";
```

19. What is the null-conditional operator?

Beginner | #operators #null | reference

?. (and its indexer form ?[]) accesses a member only if the target is not null, returning null otherwise instead of throwing NullReferenceException. It short-circuits, so a chain like a?.b?.c stops at the first null.

```
int? len = text?.Length;
string first = list?[0];
```

20. What are the C# looping constructs?

Beginner | #loops #iteration | reference

C# offers for, foreach, while, and do-while. for is index-based, foreach iterates any type implementing IEnumerable/IEnumerable, while checks the condition before each iteration, and do-while checks after, so it always runs at least once.

```
for (int i = 0; i < 3; i++) Console.WriteLine(i);
foreach (var c in "abc") Console.WriteLine(c);
int n = 0;
while (n < 3) n++;
do { n--; } while (n > 0);
```

21. What do break, continue, goto, and return do in a loop?

Beginner | #loops #control-flow | reference

break exits the nearest loop or switch. continue skips to the next iteration. return exits the enclosing method. goto jumps to a labeled statement and is discouraged in normal control flow. You can also break out of nested loops with a goto to a label after the loops.

```
foreach (var x in items)
{
    if (x < 0) continue;
    if (x == target) break;
}
```

22. What is the switch statement and how does pattern matching extend it?

Beginner | #switch #pattern-matching | reference

A switch statement selects a block based on a value. Modern C# adds pattern matching: type patterns, constant patterns, relational patterns, property patterns, and when guards. Since C# 8, switch expressions produce a value and are often cleaner than statements.

```
string Describe(object o) => o switch
{
    int n when n < 0 => "negative",
    int => "non-negative int",
    string s => $"string of length {s.Length}",
    null => "null",
    _ => "other"
};
```

23. What is the difference between the == operator and the Equals method?

Beginner | #equality #operators | reference

For value types, == and Equals compare values by default. For reference types, == compares references unless the type overloads ==, while Equals can be overridden to compare values. Types such as string override both so they compare content. Prefer Equals for polymorphic value comparison and object.ReferenceEquals to test identity.

```
        string a = "hi", b = "hi";
        bool eq = a == b;                 // true (content)
        bool same = ReferenceEquals(a, b); // may be false
```

24. What is operator overloading in C#?

Beginner | #operators #overloading | reference

Operator overloading lets a type define how built-in operators behave for its instances using the operator keyword. You can overload most operators but not all, must declare them public static, and at least one operand must be the enclosing type. Overload == and != together, and override Equals and GetHashCode for consistency.

```
        public readonly struct Money
        {
            public decimal Amount { get; }
            public Money(decimal a) => Amount = a;
            public static Money operator +(Money a, Money b) => new(a.Amount + b.Amount);
        }
```

25. What is the difference between a statement and an expression?

Beginner | #syntax #fundamentals | reference

An expression produces a value (for example, a + b or a method call), while a statement performs an action (for example, an if or a declaration). Some constructs are expressions in modern C# that used to be statements, such as switch expressions, which makes them composable in larger expressions.

Object-Oriented Programming

25 questions · 15 Beginner · 10 Intermediate

26. What are the four pillars of object-oriented programming?

Beginner | #oop #fundamentals | reference

Encapsulation hides internal state behind a public API. Abstraction models essential behavior and hides detail. Inheritance lets a type reuse and extend another. Polymorphism lets a single interface represent multiple concrete types. C# supports all four through access modifiers, abstract types, class inheritance, and virtual/interface dispatch.

27. What is a class and what is an object?

Beginner | #class #object | reference

A class is a blueprint that defines fields, properties, methods, and events. An object (instance) is a concrete value created from the class with new, occupying memory and holding its own state. Classes are reference types.

```
public class Person
{
    public string Name { get; set; } = "";
}
var p = new Person { Name = "Ada" };
```

28. What is encapsulation and how is it implemented in C#?

Beginner | #encapsulation #properties | reference

Encapsulation protects an object's state by exposing controlled access through properties, methods, and access modifiers. Fields are typically private; public properties with get and set provide validation and allow the internal representation to change without breaking callers.

```
private int _age;
public int Age
{
    get => _age;
    set => _age = value >= 0 ? value : throw new ArgumentOutOfRangeException();
}
```

29. What is inheritance in C# and what are its limits?

Beginner | #inheritance #base-class | reference

Inheritance lets a derived class reuse and specialize a base class using the colon syntax. A class can inherit from only one base class, but it can implement many interfaces. Constructors are not inherited; the derived constructor calls a base constructor with base(...).

```
public class Animal { public virtual void Speak() { } }
public class Dog : Animal
{
    public override void Speak() => Console.WriteLine("Woof");
}
```

30. What is polymorphism and what are its two forms in C#?

Beginner | #polymorphism #virtual | reference

Polymorphism lets code call the same member on different types and get behavior appropriate to the runtime type. Compile-time (static) polymorphism uses method overloading and generics. Runtime (dynamic) polymorphism uses virtual/override methods and interface dispatch resolved at execution time.

31. What is the difference between virtual, override, and new?

Intermediate | #virtual #override #new | reference

virtual declares a member that derived classes may replace. override replaces a virtual or abstract member and participates in polymorphism. new hides an inherited member by name; calls through the base type still resolve to the base member, so new breaks polymorphic expectations and should be used deliberately.

```
class A { public virtual void M() => Console.Write("A"); }
class B : A { public override void M() => Console.Write("B"); }
class C : A { public new void M() => Console.Write("C"); }
A b = new B(); b.M(); // B
A c = new C(); c.M(); // A
```

32. What is the difference between abstract classes and interfaces?

Beginner | #abstract #interface | reference

An abstract class can hold state and implementation and is limited to single inheritance. An interface defines a contract and a type can implement many of them; since C# 8 interfaces can also have default implementations but no instance fields. Use an abstract class to share code and state, and interfaces to describe capabilities.

```
public interface IShape { double Area(); }
public abstract class Shape : IShape
{
    public abstract double Area();
    public string Name { get; set; } = "";
}
```

33. What is an abstract method and how does it differ from a virtual method?

Beginner | #abstract #virtual | reference

An abstract method has no body and must be overridden in every concrete derived class; it can only appear in an abstract class. A virtual method provides a default body that derived classes may override. Both are polymorphic, but abstract forces an implementation.

```
public abstract class Shape
{
    public abstract double Area();       // must override
    public virtual string Label() => "shape"; // may override
}
```

34. What are the access modifiers in C#?

Beginner | #access-modifiers #encapsulation | reference

public is accessible everywhere. private is limited to the containing type. protected is accessible to the type and derived types. internal is limited to the same assembly. protected internal is the union of protected and internal. private protected is the intersection of private and protected (same assembly and derived).

35. What is a constructor and what kinds exist in C#?

Beginner | #constructor #initialization | reference

A constructor initializes a new instance and has the same name as the type with no return type. C# supports instance constructors, static constructors (run once per type), private constructors (for factories or singletons), and copy-style constructors. If you define none, the compiler provides a parameterless one.

```
public class Point
{
    public int X { get; }
    public Point(int x) => X = x;
    static Point() { /* one-time setup */ }
}
```

36. What is a static constructor and when does it run?

Beginner | #static #constructor | reference

A static constructor runs at most once, before the type is first used (any static member access or instance creation). It cannot take parameters, cannot be called directly, and runs before any instance constructor. Use it to initialize static fields that need logic or to load configuration once.

37. What is the difference between a field and a property?

Beginner | #fields #properties | reference

A field is a data member stored directly in the object. A property is a member with get/set accessors that look like a field to callers but can contain logic. Properties support validation, computed values, different access levels per accessor, and are the recommended public API for exposing state.

```
private int _count;
public int Count => _count;                // read-only computed
public int Value { get; init; }            // init-only
```

38. What are auto-implemented properties and init-only setters?

Intermediate | #properties #immutability | reference

Auto-implemented properties let the compiler create the backing field, written as public string Name { get; set; }. An init accessor allows the property to be set only during object initialization, supporting immutable objects with object initializers.

```
public string Name { get; init; } = "";
var p = new Person { Name = "Ada" }; // allowed
// p.Name = "Bob";                   // error after init
```

39. What is the this keyword used for?

Beginner | #this #instance | reference

this refers to the current instance. It disambiguates a field from a parameter, passes the current object to other methods, supports extension methods as the first parameter modifier, and enables constructor chaining via this(...).

```
public class C
{
    private readonly int _x;
    public C() : this(0) { }
    public C(int x) => this._x = x;
}
```

40. What is the base keyword used for?

Beginner | #base #inheritance | reference

base accesses members of the immediate base class. It is used to call a base constructor from a derived constructor, to call a base method that was overridden, or to access hidden base members. base is a compile-time construct resolved to the base type.

```
public class Dog : Animal
{
    public Dog(string name) : base(name) { }
    public override void Speak()
    {
        base.Speak();
        Console.WriteLine("Woof");
    }
}
```

41. What is method overloading and how does it differ from overriding?

Intermediate | #overloading #overriding | reference

Overloading defines multiple methods with the same name but different parameter lists in the same type; the compiler picks the best match at compile time. Overriding replaces an inherited virtual method with the same signature and is resolved at runtime. Overloading is compile-time polymorphism; overriding is runtime polymorphism.

```
void Log(string s) { }
void Log(int i) { }        // overload
class Base { public virtual void Run() { } }
class Derived : Base { public override void Run() { } } // override
```

42. What is a sealed class or method?

Beginner | #sealed #inheritance | reference

A sealed class cannot be inherited from, and a sealed override method cannot be overridden further. Sealing can improve clarity and may help the runtime with devirtualization. A sealed override is declared as public sealed override.

```
public sealed class Config { }
public class Base { public virtual void M() { } }
public class Derived : Base { public sealed override void M() { } }
```

43. What is a partial class?

Intermediate | #partial #code-generation | reference

A partial class splits a type definition across multiple files that the compiler merges. All parts must use the partial keyword and share the same name and namespace. It is useful for generated code, such as source generators or designer files, and for separating large types logically.

```
// File1.cs
public partial class Order { public int Id { get; set; } }
// File2.cs
public partial class Order { public decimal Total { get; set; } }
```

44. What is a partial method?

Intermediate | #partial #methods | reference

A partial method splits a method declaration and its optional implementation across parts of a partial type. If no implementation is supplied the calls are removed by the compiler, so it has no runtime cost. C# 9 allows partial methods with accessibility and non-void return types when an implementation exists.

```
public partial class Order
{
    partial void OnValidated();
    public void Validate() { OnValidated(); }
}
```

45. What is the difference between composition and inheritance?

Intermediate | #composition #inheritance #design | reference

Inheritance expresses an is-a relationship and couples derived types to base internals. Composition expresses a has-a relationship by holding another object and delegating behavior. Composition is often more flexible and testable because it avoids fragile base classes and supports runtime swapping of collaborators. Prefer composition unless a true is-a relationship exists.

46. What is the object type and the System.Object methods?

Beginner | #object #equality | reference

object (alias System.Object) is the root of the type hierarchy. Every type derives from it. Its key members are ToString, Equals, GetHashCode, and GetType. You override ToString for readable output and override Equals and GetHashCode together when you define value equality.

```
public override string ToString() => $"Person({Name})";
public override int GetHashCode() => HashCode.Combine(Name);
```

47. How do you correctly implement value equality for a class?

Intermediate | #equality #hashcode | reference

Override Equals(object), implement IEquatable for the typed Equals, override GetHashCode so equal objects share a hash, and overload == and != to match. Follow the equality contract: reflexive, symmetric, transitive, and consistent. For immutable data types consider records, which generate these for you.

```
public sealed class Point : IEquatable<Point>
{
    public int X { get; init; }
    public int Y { get; init; }
    public bool Equals(Point? other) => other is not null && X == other.X && Y == other.Y;
    public override bool Equals(object? o) => Equals(o as Point);
    public override int GetHashCode() => HashCode.Combine(X, Y);
}
```

48. What is a destructor and what is finalization?

Intermediate | #finalizer #dispose #gc | reference

A destructor (~ClassName) runs when the garbage collector is about to reclaim the object, and is the C# syntax for overriding Finalize. Finalization is non-deterministic and costly. Prefer implementing IDisposable with using for deterministic cleanup and reserve finalizers for unmanaged resources, using the Dispose(bool) pattern with GC.SuppressFinalize.

```
public class Handle : IDisposable
{
    private IntPtr _p;
    public void Dispose() { _p = IntPtr.Zero; GC.SuppressFinalize(this); }
    ~Handle() { /* release unmanaged */ }
}
```

49. What is the difference between an is-a and a has-a relationship in class design?

Intermediate | #design #inheritance #composition | reference

Is-a means one type is a specialization of another and is modeled with inheritance (a Car is-a Vehicle). Has-a means a type contains another as a part and is modeled with fields or properties (a Car has-a Engine). Choosing correctly avoids deep, brittle hierarchies and keeps types loosely coupled.

50. What is the Liskov Substitution Principle in the context of C#?

Intermediate | #solid #lsp #design | reference

LSP states that objects of a derived type must be usable wherever the base type is expected without breaking correctness. Practically, an override must not strengthen preconditions, weaken postconditions, or throw new exceptions, and it should preserve the base contract. Violations usually signal that inheritance was the wrong choice and composition is better.

Value Types & Reference Types

25 questions · 6 Beginner · 15 Intermediate · 4 Advanced

51. What is the difference between a value type and a reference type?

Beginner | #value-types #reference-types | reference

Value types hold their data directly and are copied on assignment, so each variable has its own copy. Reference types hold a reference to an object on the heap, so assignment copies the reference and multiple variables can point to the same object. Structs, enums, and primitives are value types; classes, interfaces, delegates, arrays, and strings are reference types.

```
int a = 1; int b = a; b = 2;        // a still 1
var l1 = new List<int>(); var l2 = l1; l2.Add(9); // l1 has 9
```

52. Where are value types and reference types stored in memory?

Beginner | #memory #stack #heap | reference

Value types are usually stored inline: on the stack for local variables, or inside the containing object on the heap. Reference types store the object itself on the heap and a reference on the stack or in another object. A value type field inside a class lives on the heap with the object; a value type captured by a lambda may be hoisted to the heap.

53. What is a struct and when should you use one?

Beginner | #struct #value-types | reference

A struct is a value type that can have fields, properties, methods, and constructors but no parameterless instance field initializers before C# 10, no inheritance from other types, and no finalizer. Use a struct for small, immutable data (roughly 16 bytes or less) that represents a single value and is short-lived. Structs are copied on assignment, which can be costly for large types.

```
public readonly struct Point
{
    public int X { get; }
    public int Y { get; }
    public Point(int x, int y) => (X, Y) = (x, y);
}
```

54. What is a readonly struct and why does it matter?

Intermediate | #readonly #struct #performance | reference

A readonly struct marks all instance fields as readonly, guaranteeing immutability. It helps the compiler avoid defensive copies when the struct is used in readonly contexts and can enable better performance. Combined with readonly members, it prevents the hidden copy that the JIT otherwise makes for non-readonly structs.

```
public readonly struct Temperature
{
    public double Celsius { get; }
    public Temperature(double c) => Celsius = c;
    public double Fahrenheit => Celsius * 9 / 5 + 32;
}
```

55. What is a ref struct?

Intermediate | #ref-struct #span #performance | reference

A ref struct can only live on the stack and cannot be boxed, stored in a field of a class, used in arrays, or captured by async/lambda closures. It exists for high-performance scenarios like Span and ReadOnlySpan that must not escape the stack. C# 13 relaxes some rules, for example allowing ref struct locals in async methods when safe.

```
        Span<int> buffer = stackalloc int[16];
        ReadOnlySpan<char> slice = "hello".AsSpan(1, 3);
```

56. What is the default value of value types and reference types?

Beginner | #default-values #initialization | reference

Value type fields default to a zeroed value: 0 for numeric types, false for bool, '\0' for char, and a zeroed struct. Reference type fields default to null. Locals must be assigned before use unless they are fields, and C# 11 requires struct fields to be definitely assigned in constructors.

57. What is the difference between a struct and a class in terms of equality?

Intermediate | #equality #struct #record | reference

Structs use ValueType.Equals by default, which compares field values using reflection, and GetHashCode is also reflection-based, so both are slower than a custom implementation. Classes use reference equality by default. For performance, override Equals and GetHashCode in structs, or use readonly record struct which generates efficient value equality.

```
        public readonly record struct Money(decimal Amount, string Currency);
        var a = new Money(5, "USD");
        var b = new Money(5, "USD");
        Console.WriteLine(a == b); // true
```

58. What is the difference between Equals, ReferenceEquals, and == for objects?

Intermediate | #equality #referenceequals | reference

ReferenceEquals always compares identity and cannot be overridden. Equals is virtual and can be overridden for value equality. == is a static operator that is overloadable per type. For reference types without an overloaded ==, == behaves like ReferenceEquals. Use the one that matches the semantic you need and keep them consistent.

59. What is a nullable reference type?

Beginner | #nullable #reference-types #static-analysis | reference

Nullable reference types are an opt-in compile-time feature (enabled by enable) where string? means the reference may be null and string means it should not be. The compiler emits warnings to help avoid NullReferenceException. It does not add runtime checks; it is purely static analysis.

```
        #nullable enable
        string? maybe = null;
        string notNull = maybe ?? "default";
```

60. What do the null-forgiving operator and the null-conditional operator do with nullable reference types?

Intermediate | #nullable #operators | reference

The null-forgiving operator ! suppresses a nullable warning when you know a value is not null. The null-conditional operator ?. short-circuits member access on null. Combining them, x!.Length tells the compiler you assert x is not null. Use ! sparingly because it disables the safety net.

```
string? name = GetName();
int len = name!.Length;      // asserted not null
int? safe = name?.Length;    // null if name is null
```

61. What is the difference between deep copy and shallow copy?

Intermediate | #copy #clone #immutability | reference

A shallow copy duplicates the top-level object but shares referenced child objects, which MemberwiseClone and record with-expressions produce. A deep copy recursively duplicates all nested objects so the copies are fully independent. Choose deep copy when mutation of nested state must not be shared.

```
var a = new Order { Lines = new List<Line> { new() } };
var shallow = a with { };              // shares Lines list
var deep = a with { Lines = a.Lines.Select(l => l with { }).ToList() };
```

62. What is the difference between the stack and the heap?

Intermediate | #memory #stack #heap | reference

The stack is a LIFO region per thread used for method frames, locals, and value types; allocation and release are automatic and fast when a method returns. The heap stores objects managed by the garbage collector, is shared across threads, and is reclaimed by the GC. Stack overflow happens on deep recursion; heap pressure triggers GC.

63. What is a tuple and how does it differ from System.Tuple?

Intermediate | #tuple #deconstruction | reference

A value tuple, written (int Id, string Name), is a lightweight struct with named and positional elements and supports deconstruction and equality. System.Tuple is a reference type with properties Item1, Item2 and no built-in deconstruction or value equality. Prefer value tuples for transient groupings; define a class or record when the concept is meaningful and long-lived.

```
(int Id, string Name) row = (1, "Ada");
var (id, name) = row;
Console.WriteLine(row == (1, "Ada")); // true
```

64. What is an enum and what is its underlying type?

Beginner | #enum #flags | reference

An enum is a named set of integer constants, defaulting to int as the underlying type, though you can specify byte, sbyte, short, ushort, int, uint, long, or ulong. Enums are value types and improve readability and type safety. Use [Flags] with powers of two for bitmask combinations.

```
[Flags]
public enum Permissions { None = 0, Read = 1, Write = 2, Execute = 4 }
var p = Permissions.Read | Permissions.Write;
```

65. What are the pitfalls of using mutable structs?

Intermediate | #struct #immutability #pitfalls | reference

Because structs are copied by value, mutations can be lost. Mutating a struct returned from a property or stored in a readonly field changes a temporary copy, not the original. Mutable structs also cause defensive copies in readonly contexts and confuse equality. Prefer readonly structs and return new instances instead of mutating.

```
struct Counter { public int N; public void Inc() => N++; }
Counter Get() => new();
Get().Inc(); // mutates a temporary, N stays 0
```

66. What is a record type and what does it generate?

Intermediate | #record #immutability #equality | reference

A record is a reference type optimized for data. The compiler generates a constructor, init-only properties, value equality (Equals, GetHashCode, ==, !=), a readable ToString, and a with-expression for non-destructive mutation. record struct creates a value-type variant. Records support positional syntax for concise declarations.

```
public record Person(string First, string Last);
var a = new Person("Ada", "Lovelace");
var b = a with { Last = "Byron" };
Console.WriteLine(a == new Person("Ada", "Lovelace")); // true
```

67. What is a with-expression?

Intermediate | #record #with-expression | reference

A with-expression creates a copy of a record or struct with specified properties changed, using non-destructive mutation. It calls the copy constructor or clone and then sets the listed members. It works with records, record structs, anonymous types, and any type that has a suitable clone method.

```
var updated = original with { Status = Status.Closed, ClosedAt = DateTime.UtcNow };
```

68. What is the difference between a reference conversion and a boxing conversion?

Advanced | #conversion #boxing | reference

A reference conversion changes the static type without moving data because both types are reference types in the same hierarchy, for example string to object. A boxing conversion allocates a heap object and copies the value when a value type is treated as object or an interface. Reference conversions are cheap; boxing costs an allocation and a copy.

69. What is the purpose of the default literal and the default operator?

Intermediate | #default #generics | reference

default(T) produces the default value of a type and default infers it from the target type. It is useful in generics where you cannot hardcode a value, and to initialize a variable with a type-appropriate zero. For nullable types default is null.

```
T? first = default;
int zero = default;   // 0
string? none = default; // null
```

70. Why can structs implement interfaces but not inherit from other structs?

Advanced | #struct #interfaces #boxing | reference

Structs cannot inherit because they have no vtable-based object header and are copied by value; inheritance would require reference semantics and shared layout. A struct can implement an interface, but storing it as that interface boxes it, and calling interface members on the boxed copy. This is why interface dispatch on structs is a performance concern.

71. What is a Span and what problems does it solve?

Intermediate | #span #performance #memory | reference

Span is a ref struct that provides a type-safe view over contiguous memory, whether an array, stack allocation, or native buffer, without copying. It enables slicing and slicing-into-string scenarios with zero allocation. It cannot be stored on the heap, used across await, or captured, which keeps it safe.

```
ReadOnlySpan<char> span = "2026-11-13".AsSpan();
var year = int.Parse(span[..4]);
```

72. What is Memory and when do you use it instead of Span?

Intermediate | #memory #span #async | reference

Memory is a heap-friendly view over memory that, unlike Span, is a normal struct and can be stored in fields, used in async methods, and captured by closures. Convert it to Span with .Span when you need synchronous access. Use Memory when the buffer must outlive a single synchronous call.

```
async Task<int> CountAsync(Memory<byte> buffer)
{
    await ReadAsync(buffer);
    return buffer.Span.Length;
}
```

73. What is the difference between covariance, contravariance, and invariance?

Advanced | #variance #generics | reference

Covariance (out) lets a more derived type be used where a less derived one is expected, as with IEnumerable to IEnumerable. Contravariance (in) lets a less derived type be used where a more derived one is expected, as with IComparer to IComparer. Invariance allows only the exact type. These apply to generic interface and delegate type parameters.

```
IEnumerable<object> objs = new List<string>();   // covariance
IComparer<string> cmp = Comparer<object>.Default; // contravariance
```

74. What is the difference between implicit and explicit interface implementation?

Intermediate | #interfaces #implementation | reference

Implicit implementation makes the member part of the public surface and callable directly. Explicit implementation, written as void IFoo.Bar(), is only reachable through the interface and avoids name conflicts when two interfaces declare the same member. It is also used to hide members from the class's public API.

```
class C : IDisposable
{
    void IDisposable.Dispose() { }
}
// must call through interface: ((IDisposable)c).Dispose();
```

75. What is the difference between a type's static type and its runtime type?

Advanced | #types #polymorphism #dispatch | reference

The static type is what the compiler knows from the declaration and determines which members are accessible at compile time. The runtime type is the actual object type and determines virtual and interface dispatch at execution time. Casting or pattern matching converts a static type to a more specific one, with a check that can fail at runtime.

```
object o = "text";       // static type object
if (o is string s)       // runtime type string
    Console.WriteLine(s.Length);
```

Collections & Data Structures

25 questions · 5 Beginner · 15 Intermediate · 5 Advanced

76. What is the difference between an array and a List?

Beginner | #array #list | reference

An array has a fixed length set at creation and offers the fastest indexed access with minimal overhead. A List wraps an array and grows automatically by allocating a larger array and copying elements, offering Add, Remove, and other helpers at a small cost. Use an array when the size is fixed and a List when the collection changes.

```
int[] a = new int[3];
var list = new List<int> { 1, 2, 3 };
list.Add(4);
```

77. What is the difference between an array and an ArrayList?

Beginner | #arraylist #collections #boxing | reference

ArrayList is a non-generic collection that stores object references, so adding value types boxes them and retrieving them requires casts, which is slow and error-prone. Array and List are type-safe and avoid boxing. ArrayList is legacy; prefer generic collections in new code.

78. What is the difference between List, Dictionary, and HashSet?

Beginner | #list #dictionary #hashset | reference

List is an ordered sequence with index access and O(n) search. Dictionary maps keys to values with near O(1) lookup by key and requires unique keys. HashSet stores unique elements with near O(1) membership tests and set operations such as union and intersect. Choose by access pattern.

```
var list = new List<int> { 3, 1, 2 };
var map = new Dictionary<string, int> { ["a"] = 1 };
var set = new HashSet<int> { 1, 2, 3 };
```

79. How does Dictionary resolve collisions internally?

Intermediate | #dictionary #hashing #internals | reference

Dictionary uses an array of buckets and an array of entries. A key's hash code, combined with the comparer, selects a bucket index. Collisions are resolved by chaining entries through a next index, so lookup walks the short chain comparing hashes then calling Equals. When the load factor is exceeded, it resizes and rehashes, so a good GetHashCode matters for performance.

80. What is the difference between IEnumerable and ICollection and IList?

Beginner | #interfaces #collections | reference

IEnumerable supports forward iteration only. ICollection adds Count, Add, Remove, Contains, and Clear. IList adds indexed access and Insert/RemoveAt. The interfaces form an inheritance chain, so a List implements all three. Program to the least specific interface your caller needs.

```
void Print(IEnumerable<int> items)
{
    foreach (var i in items) Console.WriteLine(i);
}
```

81. What is the difference between IReadOnlyList and IList?

Intermediate | #readonly #collections #api-design | reference

IReadOnlyList exposes Count and index access without mutation methods, signaling to callers that the collection should not be changed. IList includes mutation methods. Passing an IReadOnlyList communicates intent and prevents accidental modification, though it does not prevent mutation if the caller casts back to the concrete type.

```
        IReadOnlyList<string> Names => _names;
        private readonly List<string> _names = new();
```

82. What is a LinkedList and when is it better than List?

Intermediate | #linkedlist #data-structures | reference

LinkedList is a doubly linked list where each node references the previous and next node. Insertion and removal at a known node are O(1) and there is no resizing, but there is no index access and traversal is cache-unfriendly. Use it when you do frequent insertions and removals in the middle and rarely index.

```
        var ll = new LinkedList<int>();
        var node = ll.AddLast(1);
        ll.AddAfter(node, 2);
```

83. What is a Queue and a Stack?

Intermediate | #queue #stack | reference

Queue is first-in-first-out: Enqueue adds to the back and Dequeue removes from the front. Stack is last-in-first-out: Push adds to the top and Pop removes from the top. Both are generic, efficient, and used for scheduling and backtracking respectively.

```
        var q = new Queue<int>(); q.Enqueue(1); var first = q.Dequeue();
        var s = new Stack<int>(); s.Push(1); var top = s.Pop();
```

84. What is a PriorityQueue?

Advanced | #priorityqueue #heap #algorithms | reference

PriorityQueue is a generic heap that dequeues the element with the smallest priority first. Enqueue takes an element and a priority; Dequeue returns the element with the lowest priority, breaking ties by insertion order for equal priorities. It is implemented as a binary heap with O(log n) enqueue and dequeue.

```
        var pq = new PriorityQueue<string, int>();
        pq.Enqueue("low", 10);
        pq.Enqueue("high", 1);
        Console.WriteLine(pq.Dequeue()); // high
```

85. What is a SortedDictionary versus a SortedList?

Intermediate | #sorted #dictionary #data-structures | reference

Both keep keys sorted and support O(log n) lookup. SortedDictionary is a red-black tree with O(log n) insertion and removal and lower memory for frequent updates. SortedList is a sorted array with O(log n) lookup via binary search but O(n) insertion and removal because it shifts elements. Prefer SortedDictionary when you mutate often and SortedList when you read often.

86. How do you iterate a dictionary and access its keys and values?

Beginner | #dictionary #iteration | reference

A Dictionary enumerates KeyValuePair, and you can access Key and Value on each pair. You can also iterate Keys or Values directly. Use TryGetValue to avoid a double lookup when you need to test and read.

```
foreach (var (key, value) in map)
    Console.WriteLine($"{key}={value}");
if (map.TryGetValue("a", out var v)) Console.WriteLine(v);
```

87. What is the difference between a shallow copy of a collection and a deep copy?

Intermediate | #copy #collections | reference

A shallow copy creates a new collection with the same element references, so mutating an element affects both. A deep copy clones each element, so the collections are independent. Array.Clone and List constructors make shallow copies. Use Select with a clone or a record with-expression for deep copies.

```
var shallow = new List<Order>(orders);      // shares Order objects
var deep = orders.Select(o => o with { }).ToList();
```

88. What is a jagged array and how does it differ from a multidimensional array?

Intermediate | #arrays #jagged #multidimensional | reference

A jagged array is an array of arrays, int[][], where each inner array can have a different length. A multidimensional array, int[,], is a single rectangular block with fixed dimensions. Jagged arrays are more flexible and often faster because each row is a separate array; rectangular arrays are more convenient for matrices.

```
int[][] jagged = { new[] { 1 }, new[] { 2, 3 } };
int[,] rect = new int[2, 3];
```

89. What are the common collection interfaces in System.Collections.Generic?

Intermediate | #interfaces #collections | reference

IEnumerable for iteration, ICollection for mutable collections, IList for indexed lists, IDictionary for key-value maps, ISet for set operations, and IReadOnlyCollection and IReadOnlyList for read-only views. Choosing the right interface keeps APIs flexible and testable.

90. What is an ImmutableArray and when should you use it?

Advanced | #immutable #collections | reference

ImmutableArray is a lightweight struct wrapper around an array that is fixed after creation. It is a value type, so it has value-copy semantics for the reference, and it requires the System.Collections.Immutable package. It is useful for read-mostly data and thread-safe sharing because it cannot change after construction.

```
ImmutableArray<int> a = ImmutableArray.Create(1, 2, 3);
var b = a.Add(4); // returns a new array, a is unchanged
```

91. What is the purpose of the Collection and ReadOnlyCollection wrapper classes?

Intermediate | #collections #readonly #api-design | reference

Collection is a customizable base class for a collection that delegates storage to an inner IList and exposes overridable hooks like InsertItem. ReadOnlyCollection wraps an existing list and throws on mutation, giving callers a safe view without copying. Expose ReadOnlyCollection or IReadOnlyList to protect internal state.

```
        private readonly List<string> _items = new();
        public ReadOnlyCollection<string> Items => _items.AsReadOnly();
```

92. What is ConcurrentDictionary and why use it?

Intermediate | #concurrency #dictionary #threading | reference

ConcurrentDictionary is a thread-safe dictionary designed for concurrent reads and writes without external locking. It uses fine-grained locking per bucket and lock-free reads. Methods such as GetOrAdd, AddOrUpdate, and TryUpdate run atomically, but the value factory in GetOrAdd may run more than once under contention, so it must be side-effect free.

```
        var cache = new ConcurrentDictionary<int, string>();
        var v = cache.GetOrAdd(1, k => $"value-{k}");
```

93. What is a BlockingCollection and a Channel?

Intermediate | #concurrency #producer-consumer #channels | reference

BlockingCollection is a thread-safe, optionally bounded collection for producer-consumer patterns using blocking Add and Take. Channel from System.Threading.Channels is a modern asynchronous alternative with bounded or unbounded capacity and async read/write, ideal for pipelines and async producers.

```
        var channel = Channel.CreateBounded<int>(100);
        await channel.Writer.WriteAsync(1);
        var item = await channel.Reader.ReadAsync();
```

94. What is the difference between a generic and a non-generic collection?

Intermediate | #generics #collections | reference

Generic collections such as List and Dictionary are type-safe, avoid boxing, and provide compile-time checks and better performance. Non-generic collections such as ArrayList and Hashtable store object and require casts and boxing. Use generic collections in almost all new code.

95. How does HashSet determine uniqueness?

Advanced | #hashset #equality #hashing | reference

HashSet uses a hash table keyed by the element's hash code and comparer. It calls GetHashCode to find the bucket and Equals to confirm equality. Elements are unique per the comparer. You can supply a custom IEqualityComparer for case-insensitive or domain-specific equality.

```
        var set = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
        set.Add("Apple");
        Console.WriteLine(set.Contains("apple")); // true
```

96. What is the difference between Add and AddRange on List?

Intermediate | #list #performance | reference

Add appends a single element and may trigger a resize if capacity is exceeded. AddRange appends a sequence in one operation and can resize once for the whole batch, which is more efficient than many Add calls. Preallocate capacity with the List constructor when you know the size.

```
var list = new List<int>(capacity: 1000);
list.Add(1);
list.AddRange(Enumerable.Range(2, 999));
```

97. What is an iterator and how do you implement one with yield?

Intermediate | #iterator #yield #lazy | reference

An iterator is a method or property that returns IEnumerable or IEnumerator and uses yield return to produce values lazily. The compiler builds a state machine so elements are generated on demand, which supports infinite or streaming sequences with low memory. yield break ends the sequence early.

```
IEnumerable<int> Fibonacci()
{
    int a = 0, b = 1;
    while (true)
    {
        yield return a;
        (a, b) = (b, a + b);
    }
}
```

98. What are the memory and performance characteristics of List growth?

Advanced | #list #performance #memory | reference

List starts with a small or zero capacity and doubles its internal array when full, allocating a new array and copying elements. Doubling gives amortized O(1) append. Growth creates garbage from discarded arrays, so pre-sizing with capacity avoids repeated allocations in hot paths. Count is the number of elements; Capacity is the allocated size.

99. How do you remove duplicates from a collection?

Intermediate | #linq #distinct #hashset | reference

Use Distinct from LINQ to produce a sequence with unique elements, or add elements to a HashSet and enumerate it. Distinct preserves the first occurrence order, while HashSet does not guarantee order. For complex types supply an IEqualityComparer.

```
var unique = items.Distinct().ToList();
var set = new HashSet<int>(items);
```

100. What is the difference between IEnumerable and IQueryable?

Advanced | #linq #iqueryable #performance | reference

IEnumerable executes in memory in the process and uses delegates for LINQ operators. IQueryable carries an expression tree that a provider such as Entity Framework can translate into a remote query, so filtering happens at the data source. Using IEnumerable over a queryable pulls all rows into memory, which can be a serious performance bug.

```
        IQueryable<Order> q = db.Orders.Where(o => o.Total > 100); // SQL WHERE
        IEnumerable<Order> e = q.AsEnumerable();               // filters in memory
```

Generics

25 questions · 2 Beginner · 14 Intermediate · 9 Advanced

101. What are generics and why are they useful?

Beginner | #generics #type-safety | reference

Generics let you define types and methods with a placeholder type parameter, such as List. They give compile-time type safety, eliminate casts and boxing, and let one implementation serve many types. The runtime creates specialized code per value type and shares code for reference types, so there is little to no performance cost.

```
        public T First<T>(IList<T> items) => items[0];
```

102. What is a generic constraint and what kinds exist?

Beginner | #generics #constraints | reference

Constraints restrict which types can be used as a type argument and unlock members on the type parameter. Common constraints are where T : class (reference type), struct (value type), new() (parameterless constructor), a base class, an interface, unmanaged, notnull, and a type parameter. Multiple constraints are combined with commas.

```
        public T Create<T>() where T : new() => new T();
        public void Save<T>(T item) where T : IEntity, IValidatable { }
```

103. What is the difference between a generic method and a generic class?

Intermediate | #generics #methods #classes | reference

A generic method declares its own type parameters that are scoped to that method, so a non-generic class can host one. A generic class declares type parameters that apply to all its members and are fixed when the type is constructed. Use a generic method for a single operation and a generic class when the type parameter is part of the type's identity.

```
        public class Repository<T> { public void Add(T item) { } }      // class
        public static T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) > 0 ? a : b; // method
```

104. What is the difference between the class and struct constraints?

Intermediate | #generics #constraints | reference

where T : class restricts T to reference types, so T can be null and you can use reference comparison. where T : struct restricts T to non-nullable value types and enables default(T) to be a zeroed value. The struct constraint also excludes nullable value types unless you also allow them.

105. What is the new() constraint and what does it require?

Intermediate | #generics #constraints #new | reference

where T : new() requires T to have a public parameterless constructor, so you can instantiate it with new T(). It must be the last constraint listed. It is often used in factories and generic builders to create instances of an unknown type.

```
public static List<T> CreateMany<T>(int n) where T : new()
{
    var list = new List<T>(n);
    for (int i = 0; i < n; i++) list.Add(new T());
    return list;
}
```

106. What is a generic interface and how does it differ from a generic class?

Intermediate | #generics #interfaces | reference

A generic interface declares type parameters as part of its contract, such as IRepository. Implementers supply the type argument. It lets you abstract over families of types while keeping type safety. Interfaces support variance annotations (in/out) on their type parameters, which classes do not.

```
public interface IRepository<T> where T : class
{
    T? Get(int id);
    void Add(T entity);
}
```

107. What are generic type parameter variance annotations in and out?

Advanced | #generics #variance | reference

out T (covariance) means T is only used as an output, so a more derived type can substitute. in T (contravariance) means T is only used as input, so a less derived type can substitute. They apply to interfaces and delegates and enable assignment compatibility such as IEnumerable to IEnumerable.

```
public interface IProducer<out T> { T Produce(); }
public interface IConsumer<in T> { void Consume(T item); }
```

108. How does the runtime handle generics and what is reification?

Advanced | #generics #runtime #jit | reference

C# generics are reified: the runtime knows the type arguments at execution time. For reference type arguments, one shared native code version is used. For value type arguments, the JIT generates specialized code so there is no boxing. Reification enables reflection over generic arguments and typeof(List).

109. What is the default value of a generic type parameter and how do you get it?

Intermediate | #generics #default | reference

The default value is null for reference types, a zeroed value for value types, and null for nullable value types. Use default(T) or the default literal to obtain it without knowing the type. You cannot assign a non-default value generically without constraints.

```
public T? GetOrDefault<T>(IList<T> items, int index)
{
    return index >= 0 && index < items.Count ? items[index] : default;
}
```

110. Why can't you use arithmetic operators on an unconstrained generic type?

Intermediate | #generics #generic-math #operators | reference

Operators are not part of the type parameter contract unless constrained, because the compiler cannot know if the type supports + or <. Historically you used dynamic, expression trees, or a delegate like Func. Since C# 11 static abstract members in interfaces allow generic math with INumber and similar interfaces.

```
public static T Sum<T>(IEnumerable<T> values) where T : INumber<T>
{
    T total = T.Zero;
    foreach (var v in values) total += v;
    return total;
}
```

111. What is the unmanaged constraint?

Advanced | #generics #unmanaged #interop | reference

where T : unmanaged requires T to be a value type whose fields are all unmanaged, meaning no reference types anywhere in its layout. It enables pointer operations and interop and is stricter than the struct constraint. It is used for low-level buffer and interop code.

```
unsafe void Fill<T>(T* ptr, int count) where T : unmanaged { }
```

112. What is the notnull constraint?

Intermediate | #generics #nullable #constraints | reference

where T : notnull disallows nullable type arguments. For reference types it means the type argument is a non-nullable reference type; for value types it excludes Nullable. It helps the compiler produce correct nullable warnings in generic code.

113. What is the difference between a generic type and an open or closed constructed type?

Intermediate | #generics #reflection #types | reference

An open constructed type still has unassigned type parameters, such as List or Dictionary<,>. A closed constructed type has all type arguments supplied, such as List. Reflection distinguishes them with ContainsGenericParameters and IsGenericTypeDefinition.

114. How do you overload methods using generic type parameters?

Intermediate | #generics #overloading | reference

Two methods cannot differ only by the names of their type parameters, but they can differ by arity, for example Foo() and Foo(). Overload resolution prefers the more specific non-generic or constrained method when multiple candidates apply.

```
void Log<T>(T value) { }
void Log<T1, T2>(T1 a, T2 b) { }
```

115. What are static abstract interface members and generic math?

Advanced | #generics #generic-math #static-abstract | reference

C# 11 lets interfaces declare static abstract members, including operators and static factory members like T.Zero and T.One. This enables generic algorithms that use arithmetic on a type parameter, provided the constraint names an interface such as INumber. It is the basis of the System.Numerics generic math feature.

```
static T Average<T>(IEnumerable<T> xs) where T : INumber<T>
{
    T sum = T.Zero; int n = 0;
    foreach (var x in xs) { sum += x; n++; }
    return sum / T.CreateChecked(n);
}
```

116. What is the difference between List and List with a constraint?

Intermediate | #generics #type-safety | reference

List can hold any reference type and requires casting to retrieve elements, losing type safety. A generic List constrained to a base class preserves the element type and avoids casts, giving compile-time safety and better performance. Prefer the constrained generic to the object-based collection.

117. What is the difference between IEnumerable and IReadOnlyList?

Advanced | #generics #variance #collections | reference

IEnumerable is covariant and supports only forward iteration, so IEnumerable is assignable to IEnumerable. IReadOnlyList is invariant and adds Count and indexing but not covariance. Choose covariance when you only iterate and need substitutability, and IReadOnlyList when you need indexing.

118. How do you constrain a type parameter to derive from another type parameter?

Intermediate | #generics #constraints | reference

You can write where TDerived : TBase, constraining one type parameter to be the other or derive from it. This is common in fluent builders and generic base repositories where the derived entity must be a subtype of a base entity.

```
public abstract class Repository<TEntity, TKey> where TEntity : Entity<TKey>
{
    public abstract TEntity? Find(TKey id);
}
```

119. What is a generic delegate and how is it declared?

Intermediate | #generics #delegates | reference

A generic delegate declares type parameters on the delegate itself, such as delegate TResult Mapper(T input). The built-in Action<...> and Func<...> are generic delegates. Custom generic delegates are rare today because Func and Action cover most cases, but they remain useful for named, self-documenting signatures.

```
        public delegate T Factory<out T>();
        Factory<Customer> f = () => new Customer();
```

120. Why is typeof(List<>) different from typeof(List)?

Advanced | #generics #reflection #types | reference

typeof(List<>) returns the open generic type definition with unassigned parameters, while typeof(List) returns a closed constructed type. You create a closed type at runtime with MakeGenericType. Generic type definitions are used in reflection to inspect and build constructed types.

```
        var open = typeof(List<>);
        var closed = open.MakeGenericType(typeof(int)); // List<int>
```

121. What is a generic type alias or using directive?

Intermediate | #generics #using #syntax | reference

A using alias directive creates a shorthand for a type, including closed generic types, to shorten long names or disambiguate. It does not create a new type, only a name. C# 12 added the ability to alias any type, including tuples and arrays, and to create aliases with the global modifier.

```
        using Orders = System.Collections.Generic.Dictionary<int, MyApp.Order>;
```

122. What performance issues can generics introduce?

Advanced | #generics #performance #jit | reference

Value type instantiations cause the JIT to generate code per type, which increases code size and JIT time. Constraining to an interface can cause interface dispatch, and using a struct through an interface boxes it. Large structs passed to generic methods are copied. These costs are usually small, but in hot loops prefer concrete types or careful constraints.

123. What is the difference between a generic method and a method taking object?

Intermediate | #generics #boxing #type-safety | reference

A method taking object accepts anything but requires boxing for value types and casts for retrieval, and provides no compile-time type safety. A generic method keeps the exact type, avoids boxing, and lets the compiler enforce correctness. Use generics unless you truly need to handle heterogeneous types.

124. How do you apply multiple constraints and what is their ordering rule?

Advanced | #generics #constraints | reference

You list constraints after the where clause separated by commas: a base class first, then interfaces, then the new() constraint last. You can also combine class, struct, notnull, and type parameter constraints. Violating the ordering is a compile error.

```
public class Cache<TKey, TValue>
    where TKey : notnull
    where TValue : class, ICloneable, new() { }
```

125. What is the difference between covariance in arrays and covariance in generics?

Advanced | #variance #arrays #generics | reference

Arrays are covariant since C# 1, so a string[] is assignable to object[], but this is unsound: writing an int into the object[] throws ArrayTypeMismatchException at runtime. Generic interfaces and delegates are covariant only when declared with out and are checked statically. Prefer covariant generic interfaces over array covariance.

```
object[] arr = new string[1];
// arr[0] = 5; // throws ArrayTypeMismatchException at runtime
```

Delegates, Events & Lambdas

25 questions · 4 Beginner · 15 Intermediate · 6 Advanced

126. What is a delegate?

Beginner | #delegate #callbacks | reference

A delegate is a type-safe reference to a method with a specific signature. It is like a function pointer but type-safe and object-oriented. You declare a delegate type, create an instance pointing at a method, and invoke it. Delegates enable callbacks, events, and passing methods as values.

```
public delegate int MathOp(int a, int b);
MathOp add = (a, b) => a + b;
Console.WriteLine(add(2, 3)); // 5
```

127. What are Action, Func, and Predicate?

Beginner | #action #func #delegates | reference

Action represents a method that returns void. Func represents a method that returns TResult. Predicate represents a method returning bool and is equivalent to Func. They are built-in generic delegates that remove the need to declare custom delegate types.

```
Action<string> log = s => Console.WriteLine(s);
Func<int, int, int> add = (a, b) => a + b;
Predicate<int> isEven = n => n % 2 == 0;
```

128. What is a multicast delegate?

Beginner | #delegate #multicast | reference

A multicast delegate holds a list of methods through the + and - operators. Invoking it calls all methods in order. If the delegate returns a value, only the last method's result is returned. If a method throws, later methods are not called, so guard each handler.

```
Action a = () => Console.Write("A");
a += () => Console.Write("B");
a(); // AB
```

129. What is an event in C#?

Beginner | #event #observer | reference

An event is a member that lets a class notify subscribers when something happens. It is based on a delegate and can only be raised from within the declaring class, which protects the invocation list. Subscribers use += and -= to attach and detach handlers.

```
public class Button
{
    public event EventHandler? Clicked;
    public void Click() => Clicked?.Invoke(this, EventArgs.Empty);
}
```

130. What is the difference between a delegate and an event?

Intermediate | #delegate #event #encapsulation | reference

A delegate field can be invoked and reassigned by anyone who can access it. An event wraps a delegate so outside code can only add or remove handlers, not invoke or clear it. This encapsulation prevents external code from firing the event or overwriting the subscriber list.

131. What is a lambda expression and what forms does it take?

Intermediate | #lambda #anonymous | reference

A lambda expression is an anonymous function written as parameters => expression or parameters => { statements }. Expression lambdas are concise; statement lambdas allow multiple statements. Lambdas can be converted to delegates or expression trees depending on the target type.

```
Func<int, int> square = x => x * x;
Action<string> greet = name => { Console.WriteLine($"Hi {name}"); };
```

132. What is an anonymous method?

Intermediate | #anonymous #delegate | reference

An anonymous method uses the delegate keyword to define an inline method without a name. It predates lambdas and can omit the parameter list when unused. Lambdas are preferred today, but anonymous methods remain valid and can ignore parameters with delegate { }.

```
Func<int, int, int> add = delegate (int a, int b) { return a + b; };
```

133. What is a closure and how do captured variables behave?

Intermediate | #closure #capture #lambda | reference

A closure is a lambda or anonymous method that captures variables from its enclosing scope. Captured variables are hoisted into a compiler-generated object so they outlive the enclosing method frame. Capturing a loop variable before C# 5 caused classic bugs; foreach iteration variables are now per-iteration, but for-loop variables are still shared.

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
{
    int copy = i;
    actions.Add(() => Console.Write(copy)); // 012
}
```

134. How do you unsubscribe from an event and why does it matter?

Intermediate | #event #memory-leak | reference

Use -= with the same handler reference to detach. Failing to unsubscribe keeps the publisher alive as long as it holds the delegate, and keeps the subscriber alive as long as the publisher lives, which leaks memory for long-lived publishers. Store the handler in a field when you need to detach it later.

```
_timer.Tick += OnTick;
// later
_timer.Tick -= OnTick;
```

135. What is the EventHandler and EventHandler pattern?

Intermediate | #event #eventargs #conventions | reference

The .NET convention is a method with signature (object? sender, TEventArgs e) returning void. EventHandler uses EventArgs; EventHandler uses a custom EventArgs subclass carrying data. The sender is the source of the event and e carries event data. Custom EventArgs should be immutable and derive from EventArgs.

```
public sealed class PriceChangedEventArgs : EventArgs
{
    public decimal OldPrice { get; init; }
    public decimal NewPrice { get; init; }
}
public event EventHandler<PriceChangedEventArgs>? PriceChanged;
```

136. How do you raise an event safely in a multithreaded context?

Intermediate | #event #threading #race | reference

Copy the delegate to a local variable and test for null before invoking, or use the null-conditional operator with Invoke. The local copy avoids a race where a subscriber detaches between the null check and the invocation. Since C# 6, Event?.Invoke(sender, args) is the idiomatic safe form.

```
var handler = PriceChanged;
if (handler is not null) handler(this, args);
// or simply
PriceChanged?.Invoke(this, args);
```

137. What is an expression tree?

Advanced | #expression-trees #linq #reflection | reference

An expression tree is a data structure that represents code as objects, built from System.Linq.Expressions.Expression. The compiler converts lambdas assigned to Expression> into trees instead of delegates. Providers such as Entity Framework translate the tree into SQL. Trees are immutable and can be visited, analyzed, and compiled to delegates.

```
Expression<Func<Order, bool>> predicate = o => o.Total > 100;
var body = predicate.Body; // BinaryExpression
```

138. What is the difference between a lambda and an expression tree?

Advanced | #lambda #expression-trees | reference

A lambda compiled to a delegate becomes executable code. A lambda compiled to Expression<...> becomes data describing the code, which can be inspected and translated. The same syntax serves both; the target type decides. Expression trees cannot contain statements such as loops in the basic form.

```
Func<int,int> f = x => x + 1;             // delegate
Expression<Func<int,int>> e = x => x + 1; // tree
```

139. What is the difference between Func and a custom delegate?

Intermediate | #func #delegates #api-design | reference

Func is a built-in delegate that covers most cases and reduces boilerplate. A custom delegate gives a meaningful name that documents intent, supports XML documentation, and can have its own variance annotations. Use Func for generic plumbing and a named delegate when the signature carries domain meaning.

140. How does delegate equality and removal work with multiple handlers?

Intermediate | #delegate #equality | reference

Delegates compare equal when they have the same target and method. The -= operator removes the last matching entry in the invocation list. Removing a handler that was never added is a no-op. This is why you must keep the exact same delegate instance or method reference to detach.

141. What is the performance cost of delegates and lambdas?

Advanced | #delegate #performance #allocations | reference

A delegate call is an indirect call, which may not inline and can cost more than a direct call. Lambdas that capture variables allocate a closure object, adding GC pressure. Caching a delegate in a static field avoids repeated allocation. In hot paths prefer direct calls or pass delegates that the JIT can devirtualize.

142. What is a static lambda and when is it useful?

Intermediate | #lambda #static #performance | reference

Since C# 9 you can write static lambdas with the static modifier, which forbids capturing any state from the enclosing scope. This prevents accidental closures and their allocations. Use static lambdas when the lambda does not need outer variables, especially in performance-sensitive code.

```
        items.Select(static x => x * 2);
```

143. What is the difference between subscribing with a method group and a lambda?

Intermediate | #delegate #event #method-group | reference

A method group conversion creates a delegate targeting the method, and it can be detached with -= because it compares equal to another conversion of the same method. A lambda creates a new delegate each time, so you cannot reliably detach an equivalent lambda. Prefer method groups when you must unsubscribe.

```
        timer.Tick += OnTick;      // can be removed
        // timer.Tick += (s, e) => { }; // hard to remove
```

144. What is the difference between an event accessor and a field-like event?

Advanced | #event #accessors | reference

A field-like event declares the event with just a delegate type and the compiler generates add and remove accessors and a backing field. You can write custom add and remove accessors to control subscription, for example to store handlers in a dictionary or to enforce thread safety. Custom accessors are also used in interfaces and explicit implementations.

```
private EventHandler? _handlers;
public event EventHandler Custom
{
    add => _handlers += value;
    remove => _handlers -= value;
}
```

145. What is the difference between a callback and a return value?

Intermediate | #callback #delegate #design | reference

A return value hands control back to the caller once. A callback passes a delegate that the callee invokes later, possibly multiple times or after an asynchronous operation. Callbacks enable inversion of control and streaming but can complicate error handling and lifetime management.

146. How do you build a simple observer pattern with events?

Intermediate | #observer #events #patterns | reference

Expose an event on the subject, raise it when state changes, and let observers subscribe with handlers. The subject does not know the concrete observer types, which decouples them. Ensure you unsubscribe to avoid leaks, and consider weak events or a message bus for long-lived subjects.

```
stock.PriceChanged += (s, e) => Console.WriteLine(e.NewPrice);
stock.UpdatePrice(120m);
```

147. What are weak events and why are they needed?

Advanced | #events #weak-references #memory-leak | reference

A normal event holds a strong reference to each handler's target, so a long-lived publisher keeps subscribers alive. Weak events hold weak references to subscribers so they can be collected when no longer used elsewhere. They prevent memory leaks in scenarios such as UI models subscribed to long-lived services.

148. What is a delegate's Target and Method property?

Intermediate | #delegate #reflection | reference

Delegate.Target is the instance the method is invoked on, or null for a static method. Delegate.Method is the MethodInfo of the method. Together they define delegate equality. These properties are useful for diagnostics and for matching handlers when detaching.

149. How do you pass a method with parameters as a delegate using partial application?

Intermediate | #delegate #closure #functional | reference

C# has no built-in partial application, but you can wrap a method in a lambda that supplies some arguments and returns a delegate for the rest. This is a closure over the fixed arguments. It is a common way to adapt a method to a delegate signature.

```
        Func<int, int> AddFive = x => Add(5, x);
        int Add(int a, int b) => a + b;
```

150. What is the difference between Invoke and dynamic invocation of a delegate?

Advanced | #delegate #invoke #performance | reference

Invoke calls the delegate directly and is type-checked and fast. Dynamic invocation through Delegate.DynamicInvoke uses object[] arguments and reflection, which is slower and loses compile-time safety, throwing at runtime on mismatch. Prefer the typed Invoke or casting to a known delegate type.

```
        Func<int,int> f = x => x + 1;
        f.Invoke(1);          // typed, fast
        f.DynamicInvoke(1);   // slow, object-based
```

LINQ

25 questions · 5 Beginner · 14 Intermediate · 6 Advanced

151. What is LINQ?

Beginner | #linq #overview | reference

LINQ (Language Integrated Query) is a set of features and operators that let you query collections, databases, XML, and other sources with a uniform syntax. It provides a fluent API over IEnumerable and IQueryable and also query syntax. LINQ operators are implemented as extension methods in System.Linq.

152. What is the difference between query syntax and method syntax in LINQ?

Beginner | #linq #syntax | reference

Query syntax uses keywords such as from, where, and select and resembles SQL. Method syntax chains extension methods such as Where and Select. The compiler translates query syntax into method calls, so they are equivalent. Query syntax can be clearer for joins; method syntax is more compact and supports all operators.

```
        var q1 = from n in numbers where n > 2 select n;
        var q2 = numbers.Where(n => n > 2);
```

153. What is deferred execution in LINQ?

Beginner | #linq #deferred-execution | reference

Most LINQ operators build a query object that runs only when enumerated, not when defined. This lets you compose queries cheaply and reflect the latest data. Operators such as ToList, ToArray, Count, First, and Sum force execution. Calling a query twice re-executes it, which can be surprising with mutable sources.

```
        var q = numbers.Where(n => n > 2); // not executed yet
        var list = q.ToList();             // executes now
```

154. What is the difference between Where and Select?

Beginner | #linq #where #select | reference

Where filters elements by a predicate, keeping those that satisfy it. Select projects each element into a new form, such as a property or an anonymous type. They are often chained, with Where reducing the set before Select shapes it.

```
        var names = people
            .Where(p => p.Age >= 18)
            .Select(p => p.Name);
```

155. What does SelectMany do?

Beginner | #linq #selectmany #flatten | reference

SelectMany projects each element to a sequence and flattens the results into one sequence. It is the LINQ equivalent of nested loops and is used to flatten one-to-many relationships, such as orders to their lines, or to combine sequences with a correlation.

```
        var allLines = orders.SelectMany(o => o.Lines);
```

156. What is the difference between First, FirstOrDefault, Single, and SingleOrDefault?

Intermediate | #linq #first #single | reference

First returns the first element and throws if the sequence is empty. FirstOrDefault returns default if empty. Single requires exactly one element and throws if empty or if there is more than one. SingleOrDefault returns default if empty but still throws if there is more than one. Use the Single variants to assert uniqueness.

```
        var a = items.First();
        var b = items.FirstOrDefault();
        var c = items.Single(x => x.Id == id);
```

157. What is the difference between Any and Count?

Intermediate | #linq #any #count #performance | reference

Any() returns true if the sequence has at least one element and stops after the first, which is O(1) in the best case. Count() enumerates the whole sequence to count elements. Use Any for existence checks and reserve Count for when the number is actually needed.

```
        if (orders.Any()) { }
        int total = orders.Count();
```

158. What is the difference between GroupBy and ToLookup?

Intermediate | #linq #groupby #lookup | reference

GroupBy defers execution and produces groups lazily, and each group can be enumerated once in a streaming manner. ToLookup executes immediately and returns an ILookup that supports indexed access to groups by key. Use ToLookup when you need repeated keyed lookups.

```
        var byDept = employees.ToLookup(e => e.Department);
        foreach (var e in byDept["Sales"]) Console.WriteLine(e.Name);
```

159. What is the difference between inner join and group join?

Intermediate | #linq #join #outer-join | reference

An inner join returns only elements that have matches in both sequences. A group join pairs each left element with a sequence of matching right elements, including empty matches, and is the basis of left outer joins when combined with DefaultIfEmpty.

```
        var outer = from c in customers
                    join o in orders on c.Id equals o.CustomerId into custOrders
                    from o in custOrders.DefaultIfEmpty()
                    select new { c.Name, Order = o };
```

160. What is the difference between OrderBy and ThenBy?

Intermediate | #linq #sorting | reference

OrderBy establishes the primary sort and returns an IOrderedEnumerable. ThenBy adds a secondary sort on that ordered sequence. Using OrderBy twice replaces the primary sort, so always use ThenBy for tie-breakers. OrderByDescending and ThenByDescending provide descending order.

```
var sorted = people
    .OrderBy(p => p.LastName)
    .ThenBy(p => p.FirstName);
```

161. What is the difference between ToList and ToArray?

Intermediate | #linq #materialization | reference

ToList materializes into a List that can grow and be modified. ToArray materializes into a fixed-size array. Both force execution. Arrays have slightly less overhead and a smaller memory footprint; lists are easier to modify afterward. Choose by what you do next.

162. What is the difference between Aggregate and Sum?

Intermediate | #linq #aggregate #sum | reference

Sum is a specialized reduction for numeric sequences. Aggregate is the general fold that applies an accumulator function across a sequence, optionally with a seed and a result selector. Use Sum for totals and Aggregate for custom accumulation such as building a string or a composite value.

```
var total = numbers.Sum();
var csv = names.Aggregate((a, b) => a + "," + b);
```

163. What is the difference between IEnumerable and IQueryable in LINQ?

Intermediate | #linq #iqueryable #ef-core | reference

IEnumerable operators run in memory over the objects. IQueryable operators build an expression tree that a provider translates, for example into SQL, so filtering and projection happen at the source. Mixing the two by calling AsEnumerable too early pulls all data into memory.

164. What is the difference between Distinct, Union, Intersect, and Except?

Advanced | #linq #set-operations | reference

Distinct removes duplicates from one sequence. Union combines two sequences removing duplicates. Intersect returns elements present in both. Except returns elements in the first not in the second. All use the default comparer or a supplied IEqualityComparer and preserve first-sequence order for the left-side operators.

```
var common = a.Intersect(b);
var onlyA = a.Except(b);
var all = a.Union(b);
```

165. What is the difference between Zip and SelectMany?

Advanced | #linq #zip #selectmany | reference

Zip pairs elements from two sequences by position, stopping at the shorter one, and applies a result selector. SelectMany flattens a sequence of sequences or correlates each element with many others. Use Zip for element-wise combination and SelectMany for one-to-many expansion.

```
        var sums = a.Zip(b, (x, y) => x + y);
```

166. What is the difference between Take, Skip, and their While variants?

Intermediate | #linq #paging #take #skip | reference

Take(n) returns the first n elements; Skip(n) bypasses the first n. TakeWhile returns elements while a predicate is true and stops at the first false; SkipWhile skips while true and returns the rest. They are used for paging and for stream processing with a stopping condition.

```
        var page = items.Skip(20).Take(10);
        var leading = items.TakeWhile(x => x != null);
```

167. How does LINQ's deferred execution interact with mutable collections?

Advanced | #linq #deferred-execution #pitfalls | reference

Because the query is a recipe, not a snapshot, it reflects the source at enumeration time. If the source changes between definition and execution, the results change, which can cause surprises or InvalidOperationException if the collection is modified during enumeration. Materialize with ToList when you need a stable snapshot.

```
        var q = list.Where(x => x > 0);
        list.Add(5); // affects q when enumerated
        var snapshot = q.ToList();
```

168. What does DefaultIfEmpty do?

Intermediate | #linq #outer-join | reference

DefaultIfEmpty returns the sequence itself if it has elements, or a single-element sequence containing default(T) if it is empty. It is the key to left outer joins in LINQ and to guarding downstream operators against empty sequences.

```
        var result = from c in customers
                     join o in orders on c.Id equals o.CustomerId into g
                     from o in g.DefaultIfEmpty()
                     select new { c.Name, OrderId = o?.Id };
```

169. What is the difference between Count() and the Count property?

Intermediate | #linq #performance #count | reference

Count() is an extension method that enumerates or uses ICollection.Count when available. The Count property on ICollection or List is a direct field read and is O(1). For arrays and lists use the property to avoid the method call overhead, though the LINQ method optimizes for ICollection.

```
        int a = list.Count;      // property
        int b = list.Count();    // LINQ extension
```

170. What is the difference between AsEnumerable, AsQueryable, and AsParallel?

Advanced | #linq #plinq #iqueryable | reference

AsEnumerable switches a query to in-memory evaluation. AsQueryable lifts an in-memory sequence to IQueryable for provider-style composition. AsParallel enables PLINQ to run operators on multiple threads. Each changes where and how the query executes, with major implications for performance and semantics.

```
var inMemory = db.Orders.AsEnumerable().Where(ComplexRule);
var parallel = numbers.AsParallel().Select(Heavy).ToArray();
```

171. What does the let keyword do in query syntax?

Intermediate | #linq #query-syntax #let | reference

The let keyword introduces a range variable that stores an intermediate result so it can be reused in later clauses without recomputation. It is translated into a transparent identifier in the method syntax, improving readability and avoiding repeated calculations.

```
var q = from p in people
        let full = p.First + " " + p.Last
        where full.StartsWith("A")
        select full;
```

172. What is the difference between Contains and Any for a membership test?

Advanced | #linq #contains #any | reference

Contains checks whether a specific value is present using equality and can use ICollection optimizations. Any(predicate) checks whether any element satisfies a condition. For a single known value use Contains; for a condition use Any. Both short-circuit on the first match.

```
bool hasId = ids.Contains(42);
bool hasAdult = people.Any(p => p.Age >= 18);
```

173. What are the dangers of using LINQ inside a hot loop?

Intermediate | #linq #performance #allocations | reference

Each LINQ query allocates iterators and delegates, and deferred execution can cause repeated work if the query is enumerated in a loop. Materialize once outside the loop, use arrays and for loops in the hottest paths, and avoid closure allocations. Profile before optimizing, but know that LINQ has real allocation cost.

174. How do you project into an anonymous type and when should you use a named type?

Intermediate | #linq #anonymous-types #projection | reference

An anonymous type created with new { ... } is convenient for temporary projections within a method. It cannot be returned from a public method or used across assemblies easily. Use a named class, record, or tuple when the projection must cross a boundary or be referenced by name.

```
var summary = orders.Select(o => new { o.Id, o.Total });
public record OrderSummary(int Id, decimal Total);
```

175. How does LINQ to Entities differ from LINQ to Objects?

Advanced | #linq #ef-core #expression-trees | reference

LINQ to Objects runs delegates in memory over .NET objects. LINQ to Entities translates expression trees to SQL and runs them in the database, so only supported operations work and some methods throw or silently client-evaluate. Keep queries translatable, project only needed columns, and inspect generated SQL.

Exception Handling

25 questions · 4 Beginner · 16 Intermediate · 5 Advanced

176. What is an exception in C#?

Beginner | #exceptions #errors | reference

An exception is an object that represents an error or unexpected condition during execution. It derives from System.Exception and propagates up the call stack until caught by a try/catch or it reaches the top and terminates the process. Exceptions carry a message, a stack trace, and optionally an inner exception.

177. What are try, catch, and finally?

Beginner | #try-catch #finally | reference

try wraps code that may throw. catch handles specific exception types. finally always runs whether or not an exception was thrown, which is ideal for releasing resources. A try must have at least one catch or a finally. Order catch blocks from most specific to most general.

```
try
{
    DoWork();
}
catch (IOException ex)
{
    Log(ex);
}
finally
{
    Cleanup();
}
```

178. What is the difference between throw and throw ex?

Beginner | #exceptions #stack-trace #rethrow | reference

throw; rethrows the current exception and preserves the original stack trace. throw ex; throws the caught exception again but resets the stack trace to the current location, losing the original origin. Always use a bare throw to rethrow.

```
catch (Exception ex)
{
    Log(ex);
    throw; // preserves stack trace
}
```

179. What are the common built-in exception types?

Beginner | #exceptions #types | reference

Common types include ArgumentException, ArgumentNullException, ArgumentOutOfRangeException, InvalidOperationException, NullReferenceException, IndexOutOfRangeException, KeyNotFoundException, FormatException, OverflowException, IOException, and NotImplementedException. Choose the most specific type that matches the condition.

180. When should you throw an exception versus return a result?

Intermediate | #exceptions #design #error-handling | reference

Throw for exceptional conditions that the caller cannot reasonably be expected to handle in normal flow, such as invalid arguments or broken invariants. Use return values, nullable types, or result objects for expected outcomes such as not found or validation failure. Exceptions are costly, so do not use them for ordinary control flow.

181. What is a custom exception and how do you create one?

Intermediate | #exceptions #custom | reference

A custom exception derives from Exception or a more specific base such as ArgumentException and follows the standard pattern: three constructors, one parameterless, one with a message, and one with a message and inner exception, plus serialization support if needed. Give it a descriptive name ending in Exception.

```
public class InsufficientFundsException : Exception
{
    public InsufficientFundsException() { }
    public InsufficientFundsException(string message) : base(message) { }
    public InsufficientFundsException(string message, Exception inner) : base(message, inner) { }
}
```

182. What is an inner exception and why is it useful?

Intermediate | #exceptions #inner-exception #diagnostics | reference

An inner exception captures the original cause when you wrap and rethrow a lower-level error as a more meaningful one. It preserves the chain for diagnostics. Pass it to the constructor and include it when logging so the root cause is visible.

```
try { ParseConfig(path); }
catch (IOException ex)
{
    throw new ConfigurationException($"Cannot read {path}", ex);
}
```

183. What is the difference between catch (Exception) and catch?

Intermediate | #exceptions #catch | reference

catch (Exception) catches all CLS-compliant exceptions, which is effectively all managed exceptions. A bare catch catches everything including non-CLS exceptions in some legacy scenarios. Both should be used sparingly because they hide programming errors. Prefer catching specific types and rethrowing or logging.

184. How does the using statement relate to exceptions?

Intermediate | #exceptions #using #dispose | reference

The using statement calls Dispose in a finally block, so resources are released even if an exception is thrown. This is the deterministic cleanup mechanism for IDisposable. If the body throws, Dispose still runs and any exception it throws may replace the original, so Dispose should not throw.

```
using var stream = File.OpenRead(path);
// stream.Dispose() runs on scope exit, even on exception
```

185. What is exception filtering with the when clause?

Intermediate | #exceptions #when #filtering | reference

A catch block can include when (condition) to run only if the condition is true. Unlike catching and rethrowing, filtering does not unwind the stack, so the original stack trace is preserved and other handlers can still see the exception. It is ideal for logging and selective handling.

```
catch (Exception ex) when (ex is not OperationCanceledException)
{
    Log(ex);
    throw;
}
```

186. What are the performance implications of exceptions?

Intermediate | #exceptions #performance | reference

Throwing is expensive: the runtime captures a stack trace and unwinds frames, which can be orders of magnitude slower than a normal return. In hot paths, use TryParse and TryGetValue instead of exceptions, and reserve exceptions for truly exceptional cases. Catching without throwing is relatively cheap.

187. What is the ExceptionDispatchInfo and how does it preserve a stack trace?

Advanced | #exceptions #stack-trace #advanced | reference

ExceptionDispatchInfo.Capture captures an exception and its stack trace, and Throw rethrows it without resetting the trace. It is used when you must catch an exception, store it, and rethrow it later, for example in async coordination or when completing a task with an exception.

```
var edi = ExceptionDispatchInfo.Capture(ex);
// later
edi.Throw();
```

188. What is an AggregateException?

Intermediate | #exceptions #aggregate #tasks | reference

AggregateException holds one or more exceptions, typically from parallel operations or Task.Wait on multiple tasks. Its InnerExceptions collection contains the individual failures. Awaiting a task unwraps the first exception by default, so use Task.WhenAll or inspect AggregateException when you need all errors.

```
try { Task.WaitAll(tasks); }
catch (AggregateException ae)
{
    foreach (var e in ae.InnerExceptions) Log(e);
}
```

189. What is the difference between SystemException and ApplicationException?

Intermediate | #exceptions #types | reference

SystemException is the base for exceptions thrown by the runtime, such as NullReferenceException and IndexOutOfRangeException. ApplicationException was intended as a base for user exceptions but Microsoft no longer recommends deriving from it. Derive custom exceptions directly from Exception or a specific framework exception.

190. What is exception neutrality and why does it matter?

Advanced | #exceptions #best-practices | reference

Exception neutrality means a method should not catch and swallow exceptions it cannot handle, nor should it alter the exception contract of the methods it calls. Swallowing exceptions hides failures and makes debugging hard. Catch only what you can handle, add context, and rethrow with a bare throw otherwise.

191. What happens to exceptions in a finally block?

Intermediate | #exceptions #finally | reference

If code in a finally block throws, that exception replaces any in-flight exception from the try, hiding the original. This is why finally should only perform cleanup that is unlikely to throw. If both occur, the finally exception propagates and the original is lost unless captured.

192. What is the NullReferenceException and how do you prevent it?

Intermediate | #exceptions #null #defensive-programming | reference

NullReferenceException occurs when you dereference a null reference. Prevent it with nullable reference types, guard clauses, the null-conditional and null-coalescing operators, and ArgumentNullException.ThrowIfNull for parameters. Fix the root cause rather than catching it.

```
public void Send(string body)
{
    ArgumentNullException.ThrowIfNull(body);
    _client.Post(body);
}
```

193. How do you handle exceptions in async methods?

Advanced | #exceptions #async #tasks | reference

An exception thrown in an async method is captured in the returned Task and rethrown when awaited, preserving the original stack. Exceptions thrown before the first await may surface synchronously. Do not let exceptions escape into fire-and-forget tasks; observe them or store them. AggregateException appears with Task.WaitAll and Parallel.

```
async Task RunAsync()
{
    try { await WorkAsync(); }
    catch (HttpRequestException ex) { Log(ex); }
}
```

194. What is the StackTrace and how do you read it?

Intermediate | #exceptions #diagnostics #debugging | reference

The stack trace lists the call frames from the throw site outward, with file and line when debug symbols are available. Read from top to bottom to find where the error originated and the path that led there. Inner exceptions have their own traces, so follow the chain to the root cause.

195. What is the difference between a fault and a cancellation in async code?

Intermediate | #exceptions #async #cancellation | reference

A fault is an error represented by a faulted Task that throws when awaited. Cancellation is cooperative and signaled by OperationCanceledException carrying a CancellationToken. Treat cancellation as normal flow, not an error, and do not log it as a failure.

```
try { await WorkAsync(ct); }
catch (OperationCanceledException) when (ct.IsCancellationRequested) { /* expected */ }
```

196. How do you log exceptions effectively?

Intermediate | #exceptions #logging | reference

Log the full exception including type, message, stack trace, and inner exceptions, plus contextual data such as request id or user id. Log once at the boundary that handles the error, not at every layer, to avoid noise. Use structured logging so the fields are queryable.

197. What is the difference between a recoverable and a non-recoverable exception?

Advanced | #exceptions #design #resilience | reference

A recoverable exception is one the caller can reasonably handle, such as a transient network error, by retrying or falling back. A non-recoverable exception indicates a broken invariant or programming error, such as NullReferenceException or OutOfMemoryException, where continuing is unsafe. Design APIs to distinguish the two, for example with a transient flag or a result type.

198. What does Exception.ThrowIfNull do?

Intermediate | #exceptions #guards #modern-csharp | reference

ArgumentNullException.ThrowIfNull is a static helper that throws an ArgumentNullException if the argument is null, otherwise does nothing. It replaces manual null checks and produces a correctly named parameter in the exception. Similar helpers exist such as ArgumentException.ThrowIfNullOrEmpty.

```
public void Add(string name)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(name);
    _names.Add(name);
}
```

199. What is a guard clause?

Intermediate | #guard-clauses #defensive-programming | reference

A guard clause validates arguments or preconditions at the start of a method and returns or throws early. It keeps the main logic unindented and makes assumptions explicit. Use guard clauses for null checks, ranges, and state validation, throwing the most specific exception type.

```
public decimal Divide(decimal a, decimal b)
{
    if (b == 0) throw new DivideByZeroException();
    return a / b;
}
```

200. How do you handle exceptions at an application boundary such as an API?

Advanced | #exceptions #aspnet-core #middleware | reference

Use a global handler or middleware that catches unhandled exceptions, maps known exception types to appropriate status codes, logs the details, and returns a safe response without leaking internals. In ASP.NET Core this is done with the exception handler middleware or IExceptionHandler. Never return raw stack traces to clients.

```
        app.UseExceptionHandler(errorApp => errorApp.Run(async context =>
        {
            context.Response.StatusCode = StatusCodes.Status500InternalServerError;
            await context.Response.WriteAsJsonAsync(new { error = "Unexpected error" });
        }));
```

Async & Await

25 questions · 3 Beginner · 17 Intermediate · 5 Advanced

201. What is asynchronous programming and why is it useful?

Beginner | #async #overview | reference

Asynchronous programming lets a thread start an operation that takes time, such as I/O or a network call, and do other work instead of blocking. This improves scalability and responsiveness because threads are not held waiting. In .NET it is built on Task and the async/await keywords.

202. What does the async keyword do?

Beginner | #async #compiler | reference

async marks a method so the compiler can transform it into a state machine that supports await. It allows await inside the method and changes the return type to Task, Task, or ValueTask, or to void only for event handlers. async alone does not create a thread; it enables suspension and resumption.

```
public async Task<int> GetCountAsync()
{
    await Task.Delay(100);
    return 42;
}
```

203. What does the await keyword do?

Beginner | #await #async | reference

await suspends the current method until the awaited Task completes, without blocking the thread. When the task finishes, execution resumes after the await. If the task faulted, await rethrows its exception. If the method already completed, await continues synchronously without a context switch.

```
var data = await httpClient.GetStringAsync(url);
```

204. What is a Task and what states can it be in?

Intermediate | #task #async #states | reference

A Task represents an asynchronous operation. It can be WaitingForActivation, WaitingToRun, Running, WaitingForChildrenToComplete, RanToCompletion, Canceled, or Faulted. Check Status, IsCompleted, IsCompletedSuccessfully, IsFaulted, or IsCanceled. Task carries a result of type T.

205. What is the difference between Task and Thread?

Intermediate | #task #thread #concurrency | reference

A Thread is an OS-level thread you create and manage. A Task is a higher-level abstraction scheduled on the thread pool, representing a unit of work that may complete asynchronously without occupying a thread the whole time. Tasks compose with await and continuations, while threads require manual coordination.

206. What is the difference between Task and ValueTask?

Intermediate | #valuetask #task #performance | reference

Task is a class allocated on the heap for each operation. ValueTask is a struct that can wrap a result or a Task, avoiding allocation when the operation completes synchronously or is cached. Use ValueTask in high-throughput APIs where most calls complete synchronously, and never await a ValueTask more than once.

```
public ValueTask<int> ReadAsync(byte[] buffer)
{
    if (buffer.Length == 0) return ValueTask.FromResult(0);
    return new ValueTask<int>(SlowReadAsync(buffer));
}
```

207. Why is async void generally discouraged?

Intermediate | #async-void #pitfalls | reference

An async void method cannot be awaited and its exceptions cannot be observed by the caller, so they are raised on the synchronization context and can crash the process. Use async Task everywhere except event handlers, where the signature forces void. In event handlers, wrap the body in try/catch.

```
private async void Button_Click(object sender, EventArgs e)
{
    try { await SaveAsync(); }
    catch (Exception ex) { ShowError(ex); }
}
```

208. What is the SynchronizationContext and why does it matter?

Intermediate | #synchronizationcontext #configureawait #async | reference

SynchronizationContext represents the environment where continuations should run, such as a UI thread or the ASP.NET request context. By default await captures it and resumes on it. In UI apps this keeps code on the UI thread; in libraries use ConfigureAwait(false) to avoid capturing it and reduce overhead.

```
var data = await client.GetStringAsync(url).ConfigureAwait(false);
```

209. What does ConfigureAwait(false) do?

Intermediate | #configureawait #async #deadlock | reference

ConfigureAwait(false) tells await not to resume on the captured synchronization context, so the continuation runs on a thread pool thread. In library code this avoids deadlocks and improves performance. In UI event handlers keep the default true when you need to touch UI elements after the await.

210. What causes an async deadlock and how do you avoid it?

Advanced | #deadlock #async #pitfalls | reference

Blocking on a task with .Result or .Wait() while a synchronization context exists can deadlock: the continuation needs the captured context, but the blocked thread holds it. Avoid blocking on async code; make the call chain async all the way, and use ConfigureAwait(false) in libraries. In ASP.NET Core there is no synchronization context, which hides the bug until it appears in a UI or legacy app.

```
// Deadlock risk in UI apps
var data = GetDataAsync().Result; // do not do this
// Prefer
var data = await GetDataAsync();
```

211. What is the difference between Task.Run and async I/O?

Intermediate | #task-run #async #performance | reference

Task.Run offloads CPU-bound work to a thread pool thread. For I/O you should await the asynchronous API directly, such as File.ReadAllTextAsync, so no thread is consumed while waiting. Wrapping I/O in Task.Run wastes a thread and gives no benefit.

```
await File.WriteAllTextAsync(path, content);       // true async I/O
await Task.Run(() => ComputeHash(data));            // CPU-bound offload
```

212. How do you run multiple tasks concurrently and await them all?

Intermediate | #task-whenall #concurrency #async | reference

Start the tasks first, then await Task.WhenAll with the collection. WhenAll completes when every task completes and rethrows the first exception, though all exceptions are available in the tasks themselves. Awaiting tasks sequentially by starting each after the previous defeats concurrency.

```
var tasks = urls.Select(u => client.GetStringAsync(u));
var results = await Task.WhenAll(tasks);
```

213. What is the difference between Task.WhenAll and Task.WhenAny?

Intermediate | #task-whenany #timeout #async | reference

WhenAll waits for all tasks and completes when every one finishes. WhenAny completes when the first task finishes, returning that task, which is useful for timeouts or racing operations. With WhenAny you must still observe the other tasks to avoid unobserved exceptions.

```
var completed = await Task.WhenAny(task, Task.Delay(1000));
if (completed != task) throw new TimeoutException();
```

214. How does cancellation work with CancellationToken?

Intermediate | #cancellation #async #token | reference

A CancellationTokenSource produces a CancellationToken that is passed to async methods. Calling Cancel requests cancellation; well-behaved methods observe the token and throw OperationCanceledException. You can also cancel after a timeout with CancelAfter or a linked token source. Cancellation is cooperative, so methods must check the token.

```
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
await DownloadAsync(url, cts.Token);
```

215. What is a state machine in the context of async methods?

Advanced | #async #state-machine #compiler | reference

The compiler rewrites an async method into a state machine struct or class that tracks where execution paused. Each await saves the state and registers a continuation. When the awaited task completes, the state machine resumes at the saved point. This is why async methods have some overhead and why ref locals cannot cross an await.

216. What is IAsyncEnumerable and when do you use it?

Intermediate | #iasyncenumerable #async-streams #async | reference

IAsyncEnumerable represents an asynchronous stream you consume with await foreach. Each element may be produced asynchronously, which suits paging, streaming from a database, or reading large files. It supports cancellation and can be produced with async iterators using yield return inside an async method.

```
        await foreach (var row in GetRowsAsync(ct))
            Console.WriteLine(row);

        async IAsyncEnumerable<int> GetRowsAsync([EnumeratorCancellation] CancellationToken ct)
        {
            for (int i = 0; i < 10; i++)
            {
                await Task.Delay(10, ct);
                yield return i;
            }
        }
```

217. What is the difference between parallel and asynchronous execution?

Intermediate | #parallel #async #concurrency | reference

Parallelism uses multiple threads or cores to do CPU-bound work faster. Asynchrony frees the current thread while waiting for I/O, improving scalability without extra threads. They are complementary: you might use async I/O plus parallel CPU work with Parallel.For or PLINQ.

218. What does Task.Delay do and how does it differ from Thread.Sleep?

Intermediate | #task-delay #async #threading | reference

Task.Delay returns a task that completes after a delay without blocking a thread, so the thread is free to do other work. Thread.Sleep blocks the current thread for the duration. In async code use await Task.Delay; in synchronous tests or console code Thread.Sleep may be acceptable.

```
        await Task.Delay(500);   // non-blocking
        Thread.Sleep(500);        // blocks the thread
```

219. What is the difference between await and ContinueWith?

Advanced | #await #continuation #async | reference

await is a compiler-supported construct that resumes the method, handles exceptions naturally, and captures the synchronization context. ContinueWith attaches a continuation with an explicit TaskContinuationOptions and receives the completed task, so it does not rethrow automatically. Prefer await for readability and correctness.

```
await DoWorkAsync();
// versus
DoWorkAsync().ContinueWith(t => Handle(t.Result));
```

220. What is a fire-and-forget task and what are its risks?

Intermediate | #fire-and-forget #async #pitfalls | reference

A fire-and-forget task is started without being awaited, so its result and exceptions are unobserved. If it faults, the exception may be lost or surface later as an UnobservedTaskException. Use a safe wrapper that logs exceptions, or track the task. In hosted services use a background queue instead.

```
static async void FireAndForget(Func<Task> work)
{
    try { await work(); }
    catch (Exception ex) { Log(ex); }
}
```

221. What is the difference between IAsyncDisposable and IDisposable?

Intermediate | #dispose #async #resources | reference

IDisposable.Dispose releases resources synchronously. IAsyncDisposable.DisposeAsync returns a ValueTask for cleanup that may involve asynchronous I/O, such as flushing a stream. Use await using for async disposal. A type can implement both.

```
await using var stream = new FileStream(path, FileMode.Open);
// DisposeAsync is awaited on scope exit
```

222. How do you implement a timeout for an async operation?

Advanced | #timeout #async #cancellation | reference

Use a linked CancellationTokenSource with CancelAfter, or await Task.WhenAny against Task.Delay. The cancellation approach is preferred because it stops the underlying work, while WhenAny only stops waiting. .NET 8 also provides a CancellationTokenSource constructor with a timeout.

```
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(3));
await LongRunningAsync(cts.Token);
```

223. What is a SemaphoreSlim and how does it help with async concurrency?

Intermediate | #semaphore #throttling #async | reference

SemaphoreSlim limits how many operations run concurrently and supports async waiting with WaitAsync. Use it to throttle parallel calls such as limiting outbound HTTP requests. Dispose it when done, and always release in a finally or using pattern.

```
private static readonly SemaphoreSlim Gate = new(5);
await Gate.WaitAsync();
try { await CallApiAsync(); } finally { Gate.Release(); }
```

224. What is the async method builder and how does a custom awaitable work?

Advanced | #async #awaitable #advanced | reference

Async methods are compiled using an async method builder, by default AsyncTaskMethodBuilder, which coordinates the state machine. A type is awaitable if it has a GetAwaiter method returning an awaiter with IsCompleted, OnCompleted, and GetResult. This enables custom awaitables such as ValueTask and UniTask-style types.

225. Why should you avoid async in a constructor?

Intermediate | #async #constructor #factory | reference

Constructors cannot be async because the caller must receive a fully initialized object, and an async constructor would return before initialization completes. Use a static async factory method that returns Task, or an asynchronous initialization method called explicitly. This keeps the object valid when handed to callers.

```
public sealed class Client
{
    private Client() { }
    public static async Task<Client> CreateAsync()
    {
        var c = new Client();
        await c.InitializeAsync();
        return c;
    }
}
```

Memory Management & Garbage Collection

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

226. How does the managed heap differ from the stack in .NET?

Beginner | #stack #heap #fundamentals | reference

The stack is a per-thread LIFO region used for value-type locals, method parameters, and return addresses; it is fast and freed automatically when a frame returns. The heap is a process-wide region where reference-type objects are allocated and managed by the garbage collector. Stack memory is limited and short-lived, while heap memory is larger, shared, and reclaimed non-deterministically.

227. What is the managed heap?

Beginner | #heap #gc #clr | reference

The managed heap is the region the CLR uses to allocate reference types, with memory handed out by a bump pointer from the allocation pointer. It is divided into the small object heap and the large object heap, and is compacted by the garbage collector. Because it is managed, you rarely free memory explicitly; the GC does it for you.

228. What are the garbage collection generations 0, 1, and 2?

Beginner | #gc #generations | reference

The GC uses three generations to exploit the fact that most objects die young. New objects go to generation 0, survivors are promoted to generation 1, and long-lived objects end up in generation 2. Collecting a lower generation is cheap and frequent, while a generation 2 (full) collection is expensive and rare.

229. What is the difference between finalization and IDisposable?

Beginner | #finalization #idisposable | reference

A finalizer (destructor) is run by the GC finalizer thread on a non-deterministic schedule and is only a safety net for unmanaged resources. IDisposable.Dispose gives deterministic cleanup that runs when the caller invokes it, typically via using. Best practice is to implement Dispose for prompt release and keep a finalizer only when you directly hold unmanaged resources.

230. What is the Dispose pattern?

Beginner | #dispose-pattern #idisposable | reference

The Dispose pattern is the recommended way to release both managed and unmanaged resources. A public Dispose() calls a protected virtual Dispose(bool disposing) that disposes managed objects when disposing is true and frees unmanaged resources in both cases. If a finalizer exists it should call Dispose(false), and Dispose should call GC.SuppressFinalize(this).

```
public class ResourceHolder : IDisposable
{
    private bool _disposed;
    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;
        if (disposing) { /* dispose managed */ }
        /* free unmanaged */
        _disposed = true;
    }
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
}
```

231. What is the difference between IDisposable and IAsyncDisposable?

Beginner | #idisposable #iasyncdisposable | reference

IDisposable exposes a synchronous Dispose() for releasing resources. IAsyncDisposable exposes ValueTask DisposeAsync() for resources that must be released asynchronously, such as streams flushed over the network. Use await using with IAsyncDisposable and prefer it when disposal performs I/O.

232. How do using and await using work?

Beginner | #using #disposal | reference

using ensures Dispose is called when control leaves the block, even if an exception is thrown, and it is compiled to a try/finally. await using does the same for IAsyncDisposable by awaiting DisposeAsync. C# 8 introduced using declarations so a single using var disposes at the end of the enclosing scope.

```
using var stream = File.OpenRead("data.bin");
await using var conn = new SqlConnection(cs);
```

233. How do structs and classes differ in memory layout?

Beginner | #struct #class #value-types | reference

A class instance lives on the heap and a variable holds a reference to it, so copying a variable copies the reference. A struct is a value type stored inline wherever the variable lives, so assigning it copies the whole value. Structs avoid heap allocation and GC pressure but should stay small and ideally immutable to avoid costly copies.

234. What is boxing and why does it allocate?

Beginner | #boxing #allocations | reference

Boxing wraps a value type in a heap-allocated object so it can be treated as a reference type, such as when storing an int in a non-generic collection or calling an interface method on a struct through a reference. Unboxing extracts the value back. Boxing allocates on the heap and adds GC pressure, so generics and spans are preferred to avoid it in hot paths.

```
int x = 42;
object boxed = x;      // heap allocation
int y = (int)boxed;    // unbox
```

235. Why should you avoid calling GC.Collect?

Beginner | #gc-collect #performance | reference

GC.Collect forces a collection that the GC would otherwise schedule itself, and a full (generation 2) collection can pause the application and disrupt the GC's tuning heuristics. It is rarely needed because the collector adapts to allocation patterns, and calling it frequently usually hurts throughput. Reserve explicit collection for specific scenarios like memory-leak diagnostics or after freeing large amounts of unmanaged-backed memory.

236. What are the GC modes and what is background GC?

Intermediate | #gc-modes #background-gc | reference

The .NET GC runs in workstation mode (optimized for client responsiveness) or server mode (one heap and thread per core for throughput), selectable via project settings or runtime config. Background (concurrent) GC collects generation 2 on a dedicated thread while the app keeps running, reducing pause times. Server GC can also use background collection to keep throughput high with shorter pauses.

237. What does GC.SuppressFinalize do and why call it in Dispose?

Intermediate | #gc #finalization | reference

GC.SuppressFinalize tells the runtime not to call the object's finalizer because cleanup already happened via Dispose. This removes the object from the finalization queue, so it is collected in one pass instead of surviving to be finalized. It is standard to call it at the end of Dispose when the type has a finalizer.

```
public void Dispose()
{
    Dispose(true);
    GC.SuppressFinalize(this);
}
```

238. What are common causes of memory leaks in .NET?

Intermediate | #memory-leaks #events | reference

Because the GC reclaims only unreachable objects, leaks occur when objects stay reachable unintentionally. Frequent causes include event handlers that are never unsubscribed, static fields or caches that hold references forever, long-lived collections that grow without bounds, and closures capturing more than intended. Tools like dotnet-counters, dotnet-dump, and Visual Studio diagnostic tools help find the retaining roots.

239. What is array and object pooling and when should you use it?

Intermediate | #pooling #arraypool | reference

ArrayPool and MemoryPool lend reusable buffers to avoid repeated large allocations, and Microsoft.Extensions.ObjectPool reuses whole object instances. You rent a buffer, use it, and return it in a finally block, optionally clearing it to avoid leaking sensitive data. Pooling reduces GC pressure and LOH churn in allocation-heavy, high-throughput code.

```
var pool = ArrayPool<byte>.Shared;
var buffer = pool.Rent(4096);
try { /* use buffer */ }
finally { pool.Return(buffer, clearArray: true); }
```

240. How do Span and Memory help reduce allocations?

Intermediate | #span #memory #allocations | reference

Span is a stack-only, ref struct that provides a type-safe view over contiguous memory (arrays, stack memory, or unmanaged memory) without copying. Memory is a heap-friendly equivalent usable in async methods and fields because it is not a ref struct. Slicing with AsSpan or Slice creates views rather than new arrays, cutting allocations in parsing and buffer-processing code.

```
ReadOnlySpan<char> span = "hello world".AsSpan();
var word = span.Slice(0, 5); // no allocation
```

241. How does string interning affect memory?

Intermediate | #string-interning #memory | reference

The CLR keeps a pool of literal strings so identical literals share one instance, saving memory and enabling fast reference-equality checks. You can add runtime strings with String.Intern, but that pins them for the life of the process. Interning many unique runtime strings can therefore increase memory rather than reduce it, so use it carefully.

```
var a = "hello";
var b = "hello";
bool same = ReferenceEquals(a, b); // true (interned)
```

242. What is the generational hypothesis?

Intermediate | #gc #generations #theory | reference

The generational hypothesis is the empirical observation that most objects die young and that older objects tend to stay alive. The GC exploits this by checking generation 0 frequently, where collection is cheap, and checking older generations rarely. This makes collections fast on average and avoids repeatedly scanning long-lived objects.

243. What are GC roots?

Intermediate | #gc-roots #gc | reference

GC roots are the references the collector treats as definitely alive, such as local variables and parameters on thread stacks, static fields, CPU registers, and objects kept alive by GCHandles. The GC builds a reachable graph from these roots and reclaims everything unreachable. Understanding roots is key to diagnosing why an object is not collected.

244. What is SafeHandle and why is it preferred over IntPtr?

Intermediate | #safehandle #interop | reference

SafeHandle wraps an operating-system handle and guarantees it is released exactly once even if the finalizer runs during an exception or a partially completed operation. Because it is a critical finalizer object, the CLR ensures it is not aborted mid-cleanup. Prefer SafeHandle over raw IntPtr fields to make native-resource management reliable and to avoid handle leaks.

245. What is memory pressure and how do you manage it?

Intermediate | #memory-pressure #gc | reference

Memory pressure means the process is consuming enough memory that allocations or the GC become constrained. You can inform the runtime about large unmanaged allocations with GC.AddMemoryPressure and remove it with GC.RemoveMemoryPressure so the GC accounts for memory it cannot see. Monitor GC heap size, allocation rate, and generation 2 frequency to detect pressure early.

246. What is a weak reference and when would you use WeakReference?

Advanced | #weakreference #caching | reference

A weak reference points at an object without preventing its collection, so the GC can reclaim it when memory is needed. WeakReference and its generic form let you check TryGetTarget to see whether the object is still alive before using it. They suit caches of expensive objects that can be rebuilt, where you want to keep them only while memory allows.

```
var weak = new WeakReference<byte[]>(new byte[1_000_000]);
if (weak.TryGetTarget(out var data)) { /* still alive */ }
```

247. What is GCHandle and what kinds of handles exist?

Advanced | #gchandle #interop #pinning | reference

GCHandle lets you get an unmanaged-style handle to a managed object so native or unsafe code can hold a reference to it. Options include Normal, Pinned (which also fixes the address so it will not move), Weak, WeakTrackResurrection, and AddrOfPinnedObject to obtain a raw pointer. Handles must be freed with Free; a leaked pinned handle prevents compaction and can fragment the heap.

```
var arr = new int[100];
var h = GCHandle.Alloc(arr, GCHandleType.Pinned);
IntPtr ptr = h.AddrOfPinnedObject();
h.Free();
```

248. What is stackalloc and what are its constraints?

Advanced | #stackalloc #span #unsafe | reference

stackalloc allocates a block of memory on the stack, avoiding heap allocation and GC, which is useful for small temporary buffers in hot paths. In modern C# it can target Span or ReadOnlySpan for type safety, and the memory is freed automatically when the method returns. The buffer size must be small enough to avoid a stack overflow, and the memory must not outlive the method, so it cannot be returned.

```
Span<byte> buffer = stackalloc byte[256];
// use buffer; released when method returns
```

249. What is the large object heap and why can it fragment?

Advanced | #loh #fragmentation #gc | reference

Objects of 85,000 bytes or more are allocated on the large object heap (LOH), which the GC collects only during generation 2 collections. Because the LOH is not compacted by default, freed gaps between live objects can cause fragmentation and failed large allocations. You can reduce pressure by pooling large buffers, and compaction can be requested via GCSettings.LargeObjectHeapCompactionMode, but it is costly.

250. Why is finalization non-deterministic and what problems does that cause?

Advanced | #finalization #gc #determinism | reference

Finalizers run on a dedicated finalizer thread whenever the GC decides, with no ordering or timing guarantees, and the object survives an extra collection cycle before it can be reclaimed. This makes finalization unsuitable for time-sensitive cleanup such as releasing locks or closing files, and a blocked finalizer thread can stall all finalization. Implement IDisposable for deterministic cleanup and keep finalizers as a fallback for unmanaged resources only.

Strings & Text

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

251. Why are strings immutable in .NET?

Beginner | #immutability #string | reference

A System.String cannot change after it is created; operations like Replace or ToUpper return a new string. Immutability makes strings safe to share across threads, allows them to be used as dictionary keys with cached hash codes, and enables literal interning. The cost is that repeated modification creates many temporary strings and extra allocations.

252. When should you use StringBuilder instead of string concatenation?

Beginner | #stringbuilder #performance | reference

Use StringBuilder when you build a string through many appends or in a loop, because it mutates an internal buffer instead of allocating a new string each time. For a fixed, small number of concatenations the compiler folds them into a single String.Concat call, so a StringBuilder adds overhead without benefit. The rule of thumb is to reach for StringBuilder for loops or an unknown number of pieces.

```
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++)
    sb.Append(i).Append(',');
string result = sb.ToString();
```

253. What is string interning?

Beginner | #interning #string | reference

Interning stores one copy of each distinct string literal in a process-wide pool so identical literals share the same instance. This lets the runtime compare interned strings by reference for speed and saves memory for repeated literals. String.Intern adds a runtime string to the pool, but interned strings live for the life of the process.

```
string a = "hello";
string b = "hello";
bool same = ReferenceEquals(a, b); // true
```

254. What is the difference between == and Equals for strings?

Beginner | #equality #string | reference

For strings the == operator and the instance Equals method both perform value comparison, checking that the characters are equal, not that the references match. Equals has overloads that accept a StringComparison for culture or case control, while == always uses ordinal comparison. Use the explicit StringComparison overloads when culture or case sensitivity matters.

255. What is the difference between verbatim and regular string literals?

Beginner | #literals #verbatim | reference

A regular string literal interprets escape sequences such as \n and \t, while a verbatim literal prefixed with @ treats backslashes and line breaks literally. Verbatim strings are convenient for file paths and regular expressions where escapes would be noisy. To include a double quote in a verbatim string you double it.

```
        string path = @"C:\temp\logs";
        string quote = @"He said ""hi""";
```

256. What are common string escape sequences in C#?

Beginner | #escape-sequences #literals | reference

C# uses backslash escapes such as \n for newline, \t for tab, \\ for a backslash, \" for a double quote, \0 for null, and \uXXXX or \UXXXXXXXX for Unicode code points. The $ prefix enables interpolation and @ makes a literal string. Unicode escapes let you embed any character without typing it directly.

257. What is the difference between char and Rune?

Beginner | #char #rune #unicode | reference

A char is a single UTF-16 code unit, so it can only represent characters in the Basic Multilingual Plane and cannot hold an emoji or other supplementary character on its own. System.Text.Rune represents a full Unicode scalar value, handling surrogate pairs correctly. Use Rune when you must process text containing characters outside the BMP.

258. How do Unicode, UTF-8, and UTF-16 relate to .NET strings?

Beginner | #unicode #utf8 #utf16 | reference

Unicode assigns a code point to every character, while UTF-8 and UTF-16 are encodings that represent those code points as bytes or 16-bit units. .NET strings are sequences of UTF-16 code units, and Encoding.UTF8 converts them to and from bytes. Supplementary characters need two UTF-16 code units, a surrogate pair, which is why string length may exceed the visible character count.

```
        byte[] bytes = System.Text.Encoding.UTF8.GetBytes("héllo");
        string text = System.Text.Encoding.UTF8.GetString(bytes);
```

259. What is string interpolation and how does it compare to string.Format?

Beginner | #interpolation #formatting | reference

String interpolation with the $ prefix embeds expressions directly in a literal, which the compiler typically rewrites to String.Format or to a more efficient handler. It is more readable and less error-prone than positional placeholders in string.Format because arguments sit next to their text. Both respect the current or a specified culture when formatting values.

```
        string name = "Ada";
        int age = 36;
        string s = $"{name} is {age}";
```

260. How do int.Parse and int.TryParse differ?

Beginner | #parsing #tryparse | reference

int.Parse throws a FormatException or OverflowException when the input is invalid, so it should only be used when you are certain the text is valid. int.TryParse returns a bool indicating success and outputs the value through an out parameter, avoiding exceptions for expected bad input. Prefer TryParse for user input and other untrusted text.

```
        if (int.TryParse(input, out int value))
            Console.WriteLine(value * 2);
```

261. What is the difference between ordinal and culture-aware string comparison?

Intermediate | #comparison #culture #ordinal | reference

Ordinal comparison compares the numeric UTF-16 code units and is fast and predictable, making it right for identifiers, file paths, and protocol strings. Culture-aware comparison applies linguistic rules, including case folding and collation, so it is appropriate for sorting or displaying human text. Choosing the wrong one causes subtle bugs, such as sorting surprises or security checks that pass when they should not.

262. How do you do case-insensitive string comparison correctly?

Intermediate | #comparison #case-insensitive | reference

Pass StringComparison.OrdinalIgnoreCase for culture-independent, case-insensitive checks, or StringComparison.CurrentCultureIgnoreCase when linguistic rules matter. Avoid ToLower or ToUpper for comparison because they allocate and can behave incorrectly for certain cultures, such as the Turkish dotless i. Methods like Equals, Contains, and StartsWith all accept a StringComparison argument.

```
        bool match = fileName.Equals("readme.txt", StringComparison.OrdinalIgnoreCase);
```

263. What is the difference between Substring and slicing with AsSpan?

Intermediate | #span #substring #performance | reference

Substring allocates a new string holding the requested characters, which adds GC pressure when done repeatedly. AsSpan().Slice creates a ReadOnlySpan view over the original string without copying, but the span is a ref struct and cannot be stored on the heap or used across await. Use spans for transient parsing and Substring when you need an actual string.

```
        ReadOnlySpan<char> span = "2024-01-15".AsSpan();
        var year = span.Slice(0, 4); // no allocation
```

264. What are interpolated string handlers in C# 10 and later?

Intermediate | #interpolation #handlers #csharp-10 | reference

Interpolated string handlers let the compiler pass the interpolation parts to a custom type instead of always building a string. A handler is a struct with an AppendLiteral method, AppendFormatted overloads, and a constructor that takes the interpolation hole count and a format string. This enables zero-allocation logging and custom builders, and string interpolation can target a Span without allocating.

```
[InterpolatedStringHandler]
public readonly struct Builder { /* AppendLiteral, AppendFormatted */ }
```

265. What are raw string literals in C# 11?

Intermediate | #raw-string #csharp-11 #literals | reference

Raw string literals are delimited by three or more double quotes and need no escaping for quotes, backslashes, or newlines inside. The closing delimiter's indentation is stripped from each line, making embedded JSON or XML readable. Interpolation with $ still works, and the number of dollar signs controls how many braces start an interpolation.

```
string json = """
{
    "name": "Ada",
    "role": "engineer"
}
""";
```

266. How do you format a value for a specific culture?

Intermediate | #culture #formatting | reference

Pass a CultureInfo to ToString, for example value.ToString(CultureInfo.InvariantCulture), or supply it as the IFormatProvider to string.Format and interpolation. Invariant culture produces stable, machine-readable output for storage and protocols, while a specific culture matches user expectations for display. The current culture is used by default, which can cause inconsistent results across machines.

```
decimal price = 1234.5m;
string invariant = price.ToString(CultureInfo.InvariantCulture);
```

267. What is the difference between ToUpper and ToUpperInvariant?

Intermediate | #casing #culture | reference

ToUpper applies the casing rules of the current culture, which can change behavior or produce unexpected results for some cultures, most famously Turkish where lowercase i maps to a dotted uppercase I. ToUpperInvariant uses culture-independent rules and is stable across environments. Use the invariant form for comparisons, identifiers, and protocol values.

268. How does StringSplitOptions affect string splitting?

Intermediate | #split #stringsplitoptions | reference

String.Split accepts StringSplitOptions such as RemoveEmptyEntries to drop empty results and TrimEntries to trim whitespace from each entry. Combining them avoids post-processing loops and empty tokens. Without RemoveEmptyEntries, consecutive separators or leading separators produce empty strings in the result.

```
        var parts = csv.Split(',', StringSplitOptions.TrimEntries | StringSplitOptions.RemoveEmptyEntries);
```

269. What do string normalization forms NFC and NFD mean?

Intermediate | #normalization #unicode | reference

Unicode normalization converts text to a canonical form so that characters that look the same compare as equal. NFC composes base characters with combining marks into a single code point, while NFD decomposes them. Strings that are visually identical can have different code units, so normalize before comparing or storing if you handle international text.

270. What is the difference between string.Format, interpolation, and concatenation?

Intermediate | #formatting #concatenation | reference

Concatenation with + is simplest but creates a new string per operation and is inefficient in loops. string.Format uses indexed placeholders and is convenient for templates and localization where the order may change. Interpolation with $ is the most readable for inline values and the compiler can optimize it, sometimes to a custom handler or single concat.

```
        string a = "x" + y;
        string b = string.Format("{0}: {1}", key, value);
        string c = $"{key}: {value}";
```

271. What are the key RegexOptions and when is SourceGeneratedRegex useful?

Advanced | #regex #source-generation #performance | reference

RegexOptions such as IgnoreCase, Multiline, Singleline, and Compiled change matching behavior and performance; Compiled builds IL for a pattern and speeds up heavy use at the cost of JIT time and memory. In .NET 7 and later, [GeneratedRegex] source-generates a specialized matcher at compile time, avoiding runtime parsing and startup cost. Prefer source-generated regex for static patterns in hot or startup-sensitive code.

```
        [GeneratedRegex("\\d+", RegexOptions.Compiled)]
        private static partial Regex NumberRegex();
```

272. How do you detect and handle surrogate pairs correctly?

Advanced | #surrogate-pairs #unicode #rune | reference

A supplementary character is encoded as a high surrogate followed by a low surrogate in UTF-16, so iterating by char can split a character in half. Use char.IsSurrogatePair, StringInfo, or System.Text.Rune to treat such characters as single units, and Rune.EnumerateRunes for enumeration. Correct handling prevents corruption when reversing, truncating, or counting text with emoji or rare scripts.

```
        foreach (var rune in "a■b".EnumerateRunes())
            Console.WriteLine(rune.Value);
```

273. What is StringBuilder pooling and when does it help?

Advanced | #stringbuilder #pooling #performance | reference

Allocating a StringBuilder per operation creates garbage, so high-throughput code can reuse instances from an ObjectPool or use StringBuilderCache-style patterns. The pooled builder is cleared and returned after use, reducing allocation and GC work. Pooling helps most in short-lived, frequent formatting such as logging, and is unnecessary when builders are rare or long-lived.

274. How do you compare strings safely for security-sensitive checks?

Advanced | #security #comparison #ordinal | reference

Use StringComparison.Ordinal for security decisions such as validating tokens, paths, or identifiers, because culture-aware rules can treat different strings as equal or equal strings as different. Avoid ToLower/ToUpper-based checks and normalize input first when Unicode equivalence is a concern. For secrets, use fixed-time comparison helpers to avoid timing side channels.

275. What are the trade-offs of using ReadOnlySpan for text processing?

Advanced | #span #performance #parsing | reference

ReadOnlySpan avoids allocations and lets methods like int.Parse, DateTime.Parse, and StartsWith operate on slices directly, which is valuable in parsers and hot loops. Because it is a ref struct it cannot be boxed, stored in a field of a class, captured in a lambda, or used across await. Keep span-based parsing synchronous, then materialize a string only when you must persist the result.

```
        ReadOnlySpan<char> line = "key=value".AsSpan();
        int eq = line.IndexOf('=');
        var key = line.Slice(0, eq);
        var val = line.Slice(eq + 1);
```

File I/O & Serialization

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

276. What is the difference between File and FileInfo?

Beginner | #file #fileinfo | reference

File is a static class whose methods take a path and perform a one-off operation, such as File.Exists or File.ReadAllText. FileInfo is an instance class that represents a specific file, caches metadata like length and attributes, and exposes instance methods such as Open and Delete. Use File for single calls and FileInfo when you perform several operations on the same file.

277. What is the Stream class hierarchy in .NET?

Beginner | #stream #io | reference

Stream is the abstract base for byte-oriented I/O, with derived types like FileStream for files, MemoryStream for in-memory bytes, NetworkStream for sockets, and GZipStream for compression. StreamReader and StreamWriter wrap a stream to read and write text with a chosen encoding, and BinaryReader/BinaryWriter handle primitive binary values. All streams support synchronous and, usually, asynchronous read and write methods.

278. Why should streams be disposed with using or await using?

Beginner | #stream #using #disposal | reference

Streams hold operating-system resources such as file handles and socket descriptors that are released only when the stream is disposed. using guarantees Dispose runs even if an exception is thrown, and await using does the same for IAsyncDisposable by awaiting DisposeAsync. Relying on finalization instead leaks handles until the GC runs, which can exhaust the limit under load.

```
using var stream = File.OpenRead("data.bin");
// stream disposed at end of scope
```

279. How do you read and write a text file asynchronously?

Beginner | #async #file #io | reference

File.ReadAllTextAsync and File.WriteAllTextAsync read or write an entire file in one async call and are convenient for small files. For large files use a StreamReader with ReadLineAsync or ReadToEndAsync, or copy bytes with CopyToAsync, so you do not load everything into memory. Always await the returned task and pass a CancellationToken when the operation should be cancellable.

```
string text = await File.ReadAllTextAsync("input.txt");
await File.WriteAllTextAsync("output.txt", text);
```

280. Why is Path.Combine preferred over string concatenation for paths?

Beginner | #path #io | reference

Path.Combine joins path segments with the correct directory separator for the current platform, so code works on both Windows and Linux. Manual concatenation can produce doubled or missing separators and fails when a later segment is rooted, in which case Path.Combine returns the rooted segment. Use Path.GetFileName, GetExtension, and GetDirectoryName to extract path parts safely.

```
        string full = Path.Combine("logs", "app", "today.txt");
        string name = Path.GetFileName(full);
```

281. How do you create and enumerate directories?

Beginner | #directory #io | reference

Directory.CreateDirectory creates a directory and all missing parents, and Directory.Exists checks for one. Directory.EnumerateFiles and EnumerateDirectories stream results lazily and are preferred over GetFiles and GetDirectories, which materialize the whole array. The Enumerate overloads accept search patterns and SearchOption.AllDirectories for recursion.

```
        Directory.CreateDirectory("logs");
        foreach (var file in Directory.EnumerateFiles("logs", "*.txt"))
            Console.WriteLine(file);
```

282. What does FileShare control when opening a file?

Beginner | #fileshare #locking | reference

FileShare specifies what other operations are allowed on the same file while your handle is open, with values None, Read, Write, ReadWrite, and Delete. FileShare.None locks the file exclusively, while ReadWrite lets others open it concurrently. Choosing the wrong sharing mode causes IOException because another process already holds a conflicting handle.

```
        using var fs = new FileStream("data.bin", FileMode.Open, FileAccess.Read, FileShare.Read);
```

283. What is the difference between FileMode.Create and FileMode.Open?

Beginner | #filemode #io | reference

FileMode controls how the file is opened: Open requires it to exist, Create creates or overwrites it, OpenOrCreate opens it or creates a new one, Append opens it for appending or creates it, and CreateNew fails if it already exists. Truncate opens an existing file and sets its length to zero. Pick the mode that matches whether existing data must be preserved.

284. What is System.Text.Json and how do you serialize an object?

Beginner | #system-text-json #serialization | reference

System.Text.Json is the built-in, high-performance JSON library in modern .NET. JsonSerializer.Serialize converts an object to a JSON string or UTF-8 bytes, and JsonSerializer.Deserialize parses JSON back into a type. By default it matches property names case-sensitively and uses PascalCase names, which you can change through JsonSerializerOptions.

```
        var json = JsonSerializer.Serialize(person);
        var person2 = JsonSerializer.Deserialize<Person>(json);
```

285. How do you handle nulls in System.Text.Json?

Beginner | #json #nulls #options | reference

By default null properties are written to JSON and missing JSON members leave CLR properties at their default. JsonIgnoreCondition on a property, or DefaultIgnoreCondition on options, controls whether null values are written or ignored on serialization or deserialization. Use JsonIgnoreCondition.WhenWritingNull to omit null properties and keep payloads small.

```
var options = new JsonSerializerOptions
{
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};
```

286. How do JsonPropertyName and JsonIgnore affect serialization?

Intermediate | #json #attributes | reference

JsonPropertyName sets the JSON member name for a property, so C# PascalCase names can map to camelCase or other wire formats. JsonIgnore excludes a property from both serialization and deserialization, while JsonInclude can opt a non-public member in. These attributes give precise control without changing the CLR type.

```
public class User
{
    [JsonPropertyName("user_name")]
    public string Name { get; set; }
    [JsonIgnore]
    public string Secret { get; set; }
}
```

287. How does System.Text.Json support polymorphism?

Intermediate | #json #polymorphism | reference

Polymorphic serialization requires opting in with JsonDerivedType attributes on the base type, which add a type discriminator to the JSON. On deserialization the discriminator selects the derived type, and you can customize the discriminator name and value. Without this opt-in, serializing a derived instance through a base reference only writes base properties.

```
[JsonDerivedType(typeof(Dog), "dog")]
[JsonDerivedType(typeof(Cat), "cat")]
public abstract class Animal { }
```

288. What are common deserialization pitfalls in System.Text.Json?

Intermediate | #json #deserialization #pitfalls | reference

Property name matching is case-sensitive by default unless PropertyNameCaseInsensitive is set, and read-only properties are not populated unless they are collections or set by a constructor. Private setters and init-only members need constructor matching or JsonInclude. Trailing commas and comments are rejected unless JsonCommentHandling or AllowTrailingCommas are configured.

289. How do records work with System.Text.Json?

Intermediate | #records #json | reference

System.Text.Json supports records through their parameterized constructors, matching JSON properties to constructor parameters by name, so positional records deserialize correctly when names align. Immutable init-only properties are set during construction, and JsonPropertyName can adjust matching. This makes records a good fit for immutable data transfer objects.

```
public record Person(string Name, int Age);
var p = JsonSerializer.Deserialize<Person>(json);
```

290. How do you write a custom JsonConverter?

Intermediate | #json #converter #custom | reference

Derive from JsonConverter and override Read and Write, then register the converter on JsonSerializerOptions.Converters or annotate the type or property with JsonConverterAttribute. Custom converters are useful for types whose default shape is undesirable, such as a strongly typed ID or a custom date format. Use Utf8JsonReader and Utf8JsonWriter inside Read and Write for best performance.

```
public class MoneyConverter : JsonConverter<Money>
{
    public override Money Read(ref Utf8JsonReader r, Type t, JsonSerializerOptions o) => /* ... */;
    public override void Write(Utf8JsonWriter w, Money v, JsonSerializerOptions o) => /* ... */;
}
```

291. How do you serialize enums as strings with System.Text.Json?

Intermediate | #json #enums | reference

Add JsonStringEnumConverter to JsonSerializerOptions.Converters, or annotate the enum with [JsonConverter(typeof(JsonStringEnumConverter))], to write enum values as their names instead of numbers. String enums are more readable and resilient to reordering of members. The converter also accepts names during deserialization and can be case-insensitive via its constructor options.

```
var options = new JsonSerializerOptions
{
    Converters = { new JsonStringEnumConverter() }
};
```

292. How does source generation improve System.Text.Json performance?

Intermediate | #json #source-generation #performance | reference

The System.Text.Json source generator emits serialization metadata at compile time, avoiding reflection and enabling trimming and Native AOT. You declare a partial JsonSerializerContext with [JsonSerializable] attributes, then serialize using that context. This reduces startup cost and works in ahead-of-time compiled apps where reflection-based serialization is unavailable.

```
[JsonSerializable(typeof(Person))]
internal partial class AppJsonContext : JsonSerializerContext { }
var json = JsonSerializer.Serialize(person, AppJsonContext.Default.Person);
```

293. What are XmlSerializer and DataContractSerializer and how do they differ?

Intermediate | #xml #serialization | reference

XmlSerializer maps public members and XML attributes to control element and attribute names, and it supports only a subset of types but produces interoperable XML. DataContractSerializer uses opt-in [DataContract] and [DataMember] attributes, can serialize private members, and supports more complex graphs and versioning. Choose based on whether you need loose XML interop or richer contract control.

294. Why is BinaryFormatter deprecated?

Intermediate | #binary-serialization #security #deprecation | reference

BinaryFormatter deserialization can execute arbitrary code from untrusted payloads, making it a serious security risk, and its format is not version-tolerant. Microsoft marked it obsolete and disabled it by default in .NET 8 and later, with the API eventually removed. Migrate to safer, explicit formats such as System.Text.Json, protobuf, or MessagePack.

295. How do you stream JSON with Utf8JsonReader and Utf8JsonWriter?

Intermediate | #json #streaming #utf8jsonreader | reference

Utf8JsonReader is a forward-only, allocation-light reader over UTF-8 bytes that you advance token by token, and Utf8JsonWriter writes JSON incrementally to a buffer or stream. They let you process very large documents without building an object graph in memory. Both are low-level and require you to track state yourself, unlike JsonSerializer or JsonDocument.

```
var reader = new Utf8JsonReader(utf8Bytes);
while (reader.Read())
{
    // inspect reader.TokenType and value
}
```

296. How does Newtonsoft.Json compare to System.Text.Json?

Advanced | #json #newtonsoft #comparison | reference

System.Text.Json is built in, faster, uses UTF-8 natively, and supports trimming and AOT through source generation, but it has fewer features than Newtonsoft.Json. Newtonsoft.Json offers richer flexibility, such as dynamic handling with JObject, more converter options, and lenient parsing by default, at the cost of performance and an extra dependency. Choose System.Text.Json for new, performance-sensitive code and Newtonsoft when you need its specific features.

297. How do you stream JSON with IAsyncEnumerable?

Advanced | #json #iasyncenumerable #streaming | reference

System.Text.Json can serialize and deserialize IAsyncEnumerable with the streaming methods JsonSerializer.DeserializeAsyncEnumerable and the SerializeAsync overloads, emitting items incrementally. This keeps memory low when consuming a large array of objects from a network stream or API. Combine it with await foreach and a CancellationToken for responsive, cancellable processing.

```
await foreach (var item in JsonSerializer.DeserializeAsyncEnumerable<Item>(stream, options))
    Process(item);
```

298. What are memory-mapped files and when are they useful?

Advanced | #memory-mapped-files #io #performance | reference

MemoryMappedFile maps a file or shared memory region into the process address space so it can be accessed as a byte buffer, letting the OS page data in and out on demand. This is efficient for very large files, random access, and inter-process communication via named shared memory. You access the data through MemoryMappedViewAccessor or a safe accessor and must dispose both the view and the file.

```
using var mmf = MemoryMappedFile.CreateFromFile("big.dat");
using var accessor = mmf.CreateViewAccessor();
byte b = accessor.ReadByte(0);
```

299. How do you compress and decompress streams with GZipStream?

Advanced | #compression #gzipstream #streams | reference

GZipStream wraps another stream and compresses or decompresses data as it is read or written, so you can pipe a FileStream through it and then through a StreamReader for text. Set CompressionLevel to trade speed for size, and dispose the GZipStream after the inner stream so buffered data is flushed. BrotliStream is an alternative with better ratios for web content.

```
using var file = File.Create("data.gz");
using var gz = new GZipStream(file, CompressionLevel.Optimal);
using var writer = new StreamWriter(gz);
writer.Write(text);
```

300. What is buffering in file I/O and how do you manage temp files safely?

Advanced | #buffering #temp-files #io | reference

FileStream and StreamReader buffer data so reads and writes happen in larger blocks, reducing syscalls; the buffer size can be tuned and FileOptions.SequentialScan hints access patterns. For temporary files, Path.GetTempFileName creates a unique file but is limited in number and leaves files behind, so prefer Path.GetRandomFileName combined with Path.Combine into the temp directory, and always delete the file in a finally block. Dispose or Flush streams so buffered data reaches disk before deletion or reuse.

```
string tmp = Path.Combine(Path.GetTempPath(), Path.GetRandomFileName());
try { await File.WriteAllTextAsync(tmp, data); }
finally { File.Delete(tmp); }
```

Threading & Concurrency

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

301. What is the difference between a thread, a Task, and the thread pool?

Beginner | #thread #task #thread-pool | reference reference

A Thread is an operating-system thread that you create and manage explicitly, including its lifetime and background status. A Task is a higher-level abstraction that represents an asynchronous or parallel unit of work and is normally scheduled on the thread pool, so it does not necessarily own a dedicated thread. The thread pool is a managed set of reusable worker threads that amortizes the cost of thread creation and is the default scheduler for Task.Run and ThreadPool.QueueUserWorkItem.

302. How do you create and start a thread with the Thread class?

Beginner | #thread #threadstart #join | reference

You construct a Thread with a ThreadStart or ParameterizedThreadStart delegate, optionally set IsBackground, and call Start. A background thread is terminated automatically when the process exits, whereas a foreground thread keeps the process alive. Join blocks the calling thread until the target thread finishes, and you should avoid Abort because it is unsupported on .NET Core and later.

```
var t = new Thread(() => Console.WriteLine("working"));
t.IsBackground = true;
t.Start();
t.Join();
```

303. What is the .NET thread pool and why use it?

Beginner | #thread-pool #queueuserworkitem | reference

The thread pool maintains a pool of worker threads that are reused to execute queued work items, avoiding the overhead of creating and destroying a thread per operation. You submit work with ThreadPool.QueueUserWorkItem, Task.Run, or by starting a Task. The pool grows on demand and you can tune it with ThreadPool.SetMinThreads and SetMaxThreads, but long-running blocking work should use a dedicated thread instead of occupying pool threads.

304. What is a race condition?

Beginner | #race-condition #synchronization | reference

A race condition occurs when two or more threads access shared mutable state without synchronization and the result depends on the timing of their interleaving. A classic example is incrementing a counter with count++ executed on multiple threads, because the read-modify-write is not atomic and updates can be lost. Fixes include locking, Interlocked operations, or removing the shared mutable state entirely.

```
int count = 0;
Parallel.For(0, 100000, _ => count++);
// count is usually less than 100000
```

305. What does the lock statement do?

Beginner | #lock #monitor #synchronization | reference

The lock statement acquires an exclusive monitor on a reference-type object for the duration of a block, ensuring that only one thread executes the block at a time. It is compiler sugar for Monitor.Enter in a try block with Monitor.Exit in a finally block. You must lock on a dedicated private object rather than on this, a Type, or a string, because external code could also lock those and cause unexpected contention or deadlocks.

```
        private readonly object _gate = new();

        void Increment()
        {
            lock (_gate)
            {
                _count++;
            }
        }
```

306. What is a deadlock?

Beginner | #deadlock #locks | reference

A deadlock is a situation where two or more threads are permanently blocked, each waiting for a resource that another thread holds. The classic case is two threads that each acquire two locks in opposite order, so each holds one lock and waits for the other. Deadlocks can be avoided with a consistent lock-acquisition order, timeouts such as Monitor.TryEnter, or by minimizing the number of locks held simultaneously.

307. Are .NET collections thread-safe?

Beginner | #collections #thread-safety #concurrent | reference

Most general-purpose collections such as List, Dictionary, and Queue are not thread-safe for concurrent writes, and even concurrent reads during a write are unsafe. For shared mutable state you should use the System.Collections.Concurrent types like ConcurrentDictionary and ConcurrentQueue, or guard access with a lock. Alternatively, share only immutable data, which is inherently safe to read from any number of threads.

308. What is context switching overhead in multithreading?

Beginner | #context-switching #performance | reference

A context switch is the operating-system operation of saving one thread's register and stack state and restoring another's so the CPU can run a different thread. It has a real cost in CPU cycles and cache disruption, so creating far more threads than there are cores can make a program slower rather than faster. Using the thread pool and limiting parallelism to roughly the number of cores usually minimizes context switching.

309. What is starvation in multithreading?

Beginner | #starvation #scheduling | reference

Starvation is when a thread is perpetually denied the CPU time or a lock it needs because other threads keep getting scheduled or keep reacquiring the lock first. It often results from unfair locking or from high-priority threads monopolizing resources, and can be worsened by priority inversion. Fair scheduling, avoiding unbounded critical sections, and using fair synchronization primitives reduce the risk.

310. How does CancellationToken support cancellation in threads and tasks?

Beginner | #cancellation #cancellationtoken | reference

CancellationToken is a cooperative cancellation mechanism: a producer calls Cancel on the linked CancellationTokenSource, and consumers poll IsCancellationRequested or call ThrowIfCancellationRequested. Long-running loops should check the token periodically, and blocking APIs accept it to abort waiting. You can combine sources with CreateLinkedTokenSource so that any one source cancels the operation.

```
using var cts = new CancellationTokenSource();
foreach (var item in items)
{
    cts.Token.ThrowIfCancellationRequested();
    Process(item);
}
```

311. What is the relationship between the lock statement and the Monitor class?

Intermediate | #monitor #lock #pulse | reference

lock is syntactic sugar for Monitor.Enter with a matching Monitor.Exit in a finally block, so the Monitor class exposes the underlying behavior. Monitor additionally provides TryEnter for non-blocking or timed acquisition and Wait/Pulse/PulseAll for signaling between threads that share the monitor. Wait releases the lock while waiting and reacquires it before returning, which enables efficient producer-consumer coordination.

```
lock (_gate)
{
    while (!_ready)
        Monitor.Wait(_gate);
}
```

312. What are Interlocked operations and when should you use them?

Intermediate | #interlocked #atomic #lock-free | reference

The Interlocked class provides atomic operations such as Increment, Decrement, Add, Exchange, and CompareExchange that are performed as a single indivisible step without a lock. They are faster than locking for simple updates to a shared integer or reference and also act as full memory barriers. CompareExchange is the building block for many lock-free algorithms because it updates a value only if it still equals an expected value.

```
Interlocked.Increment(ref _counter);
Interlocked.CompareExchange(ref _value, newValue, expected);
```

313. What does the volatile keyword do and what does it not guarantee?

Intermediate | #volatile #memory-model | reference

volatile tells the compiler and runtime that a field may be modified by other threads, so reads and writes must not be cached in a register or reordered across the access. It prevents certain optimization and ordering problems for a single field but does not make compound operations such as increment atomic. For atomicity you need Interlocked or a lock, and volatile applies only to specific field types like int, bool, and reference types.

```
        private volatile bool _stop;
        // reading _stop in a loop sees updates from other threads
```

314. What is the difference between Mutex, Semaphore, and SemaphoreSlim?

Intermediate | #mutex #semaphore #semaphoreslim | reference reference

A Mutex is a mutually exclusive lock that also works across processes, is owned by a thread, and can be reentrant, but it is slower and has thread affinity. A Semaphore limits the number of threads that can enter a region to a configured count and can also be named for cross-process use. SemaphoreSlim is a lighter in-process version that supports async waiting with WaitAsync and is preferred for purely intra-process, asynchronous synchronization.

315. When would you use ReaderWriterLockSlim?

Intermediate | #readerwriterlockslim #readers-writer | reference

ReaderWriterLockSlim allows multiple concurrent readers or a single exclusive writer, which suits data that is read far more often than it is modified. Readers call EnterReadLock and writers call EnterWriteLock, and there is an upgradeable read mode for the rare case of promoting a read to a write. It is much faster than the older ReaderWriterLock, though for very short critical sections a plain lock is often still preferable.

```
        _rw.EnterReadLock();
        try { Read(); }
        finally { _rw.ExitReadLock(); }
```

316. How do ConcurrentDictionary and ConcurrentQueue differ from their non-concurrent counterparts?

Intermediate | #concurrentdictionary #concurrentqueue #thread-safety | reference reference

ConcurrentDictionary and ConcurrentQueue are thread-safe collections designed for concurrent producers and consumers without external locking. ConcurrentDictionary provides atomic compound operations such as GetOrAdd, AddOrUpdate, and TryRemove, while ConcurrentQueue is a lock-free FIFO queue offering Enqueue, TryDequeue, and TryPeek. They trade some single-threaded throughput and memory for safe concurrent access, and individual members are thread-safe but sequences of calls are not.

```
        var cache = new ConcurrentDictionary<int, string>();
        var value = cache.GetOrAdd(42, k => Compute(k));
```

317. What are Parallel.For and Parallel.ForEach used for?

Intermediate | #parallel #data-parallelism | reference

Parallel.For and Parallel.ForEach implement data parallelism by partitioning a range or collection and running the iterations across multiple threads from the thread pool. You can tune behavior with ParallelOptions, including MaxDegreeOfParallelism and a CancellationToken, and break early with ParallelLoopState. If any iteration throws, the call throws an AggregateException containing the individual failures. For task-based parallelism where each unit is an independent Task, await them together with Task.WhenAll or Task.WhenAny instead of using the Parallel class.

```
Parallel.For(0, 1000, i =>
{
    results[i] = Compute(i);
});
```

318. What is PLINQ and when is it worthwhile?

Intermediate | #plinq #parallel #linq | reference

PLINQ runs a LINQ query in parallel by adding AsParallel, partitioning the source and processing elements on multiple threads. Operators like AsOrdered, AsSequential, and WithDegreeOfParallelism control ordering and concurrency. PLINQ can speed up CPU-bound queries over large in-memory sources, but it adds overhead and may be slower for small or I/O-bound sequences, so it should be measured rather than assumed to help.

```
var sum = data.AsParallel()
              .Where(x => x.IsActive)
              .Sum(x => x.Amount);
```

319. What are ThreadLocal and [ThreadStatic] and how do they differ?

Intermediate | #threadlocal #threadstatic #thread-affinity | reference reference

Both provide storage that is unique to each thread, so no synchronization is required for that data. [ThreadStatic] applies to a static field but has no automatic initializer, so the value is default on every thread and any initializer runs only on the first thread. ThreadLocal is a generic instance type with a value factory, supports tracking of values, and implements IDisposable so its per-thread slots can be released.

```
private static readonly ThreadLocal<int> _counter =
    new ThreadLocal<int>(() => 0);
```

320. What is the difference between async/await and threading?

Intermediate | #async #threading #thread-pool | reference reference

async/await is about not blocking a thread while an operation is in progress, not about starting new threads; an awaited I/O operation typically frees the thread to do other work and resumes on a continuation. Threading and the thread pool are about running work in parallel on multiple threads. Use async for I/O-bound work and Task.Run or Parallel for CPU-bound work, and remember that the thread pool is shared, so blocking pool threads harms scalability.

321. What is the .NET memory model and what are memory barriers?

Advanced | #memory-model #memory-barrier #volatile | reference reference

The memory model defines the ordering guarantees the CLR and hardware provide for reads and writes observed across threads, including that plain reads and writes may be reordered by the compiler or CPU. A memory barrier is a fence that prevents such reordering and flushes pending writes so other threads see them in order. volatile, Interlocked operations, lock, and Thread.MemoryBarrier all introduce barriers, and lock provides both mutual exclusion and full fences.

322. How do atomicity and lock-free programming relate?

Advanced | #atomic #lock-free #compareexchange | reference

Atomicity means an operation completes as an indivisible unit, so no other thread can observe an intermediate state. Lock-free programming builds on atomic primitives such as Interlocked.CompareExchange to update shared data without blocking threads, guaranteeing that at least one thread always makes progress. Lock-free code avoids deadlock and priority inversion but is hard to get right and must handle problems like the ABA problem and memory reclamation.

```
int original;
do
{
    original = _value;
} while (Interlocked.CompareExchange(
    ref _value, original + 1, original) != original);
```

323. How can deadlocks be avoided or detected?

Advanced | #deadlock #lock-ordering #diagnostics | reference

The most reliable avoidance technique is a global lock ordering: every thread acquires multiple locks in the same predefined order, which makes circular waits impossible. You can also use timeouts with Monitor.TryEnter or Mutex.WaitOne to abandon a wait instead of blocking forever, keep critical sections short, and avoid calling unknown code while holding a lock. Detection at runtime is difficult in .NET because there is no built-in deadlock detector; teams typically rely on dump analysis and tooling such as the Visual Studio parallel stacks window.

324. How do you implement a producer-consumer pattern with BlockingCollection?

Advanced | #blockingcollection #producer-consumer | reference

BlockingCollection wraps a concurrent collection, by default a ConcurrentQueue, and adds blocking and bounding semantics for producer-consumer scenarios. Producers call Add and then CompleteAdding, while consumers enumerate GetConsumingEnumerable or use TryTake with a timeout; a bounded capacity applies backpressure to producers. Take blocks when empty and Add blocks when full, and the collection should be disposed when finished.

```
using var queue = new BlockingCollection<int>(boundedCapacity: 100);
var producer = Task.Run(() =>
{
    foreach (var i in Enumerable.Range(0, 1000)) queue.Add(i);
    queue.CompleteAdding();
});
foreach (var item in queue.GetConsumingEnumerable())
    Consume(item);
producer.Wait();
```

325. When should you prefer System.Threading.Channels over BlockingCollection?

Advanced | #channels #producer-consumer #async | reference reference

Channels provide an asynchronous producer-consumer pipeline whose operations return ValueTask, so consumers and producers can await instead of blocking a thread. You create a bounded or unbounded channel with Channel.CreateBounded or Channel.CreateUnbounded, write with WriteAsync and read with ReadAsync or ReadAllAsync, and complete the writer to signal the end. Because blocking collection calls block a thread, Channels are the better choice in async code, while BlockingCollection remains fine for purely synchronous scenarios.

```
        var channel = Channel.CreateBounded<int>(100);
        await channel.Writer.WriteAsync(42);
        var item = await channel.Reader.ReadAsync();
```

Reflection & Attributes

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

326. What is reflection in C#?

Beginner | #reflection #overview | reference

Reflection is the ability of a program to inspect and manipulate its own metadata and types at runtime. Using the System.Reflection namespace you can discover types, enumerate members, read attributes, create instances, and invoke methods or access fields and properties dynamically. It is powerful for frameworks, serializers, and dependency injection, but it is slower than direct calls and can break under trimming or AOT compilation.

327. What is the difference between typeof, GetType, and nameof?

Beginner | #typeof #gettype #nameof | reference reference

typeof(T) is a compile-time operator that returns the Type object for a type name known at compile time. GetType() is an instance method that returns the runtime type of a value, which may be a derived type. nameof returns the simple string name of a variable, type, or member and is resolved at compile time, so it is useful for argument exceptions and property-change notifications.

```
        Type t = typeof(string);
        Type r = "hello".GetType();
        string n = nameof(DateTime);
```

328. What does the Type class represent?

Beginner | #type #reflection | reference

The System.Type class is the entry point to reflection and represents a type declaration, including its name, namespace, base type, implemented interfaces, and members. You obtain a Type via typeof, GetType, or Type.GetType with an assembly-qualified name. Type also provides methods to create instances, query assignability, and access generic definitions.

329. What are attributes in C#?

Beginner | #attributes #metadata | reference

Attributes are declarative metadata that you attach to assemblies, types, members, parameters, and return values using square-bracket syntax. They derive from System.Attribute and are stored in metadata, where they can be read at runtime through reflection or consumed by the compiler. Common built-in attributes include Obsolete, Serializable, Conditional, DebuggerDisplay, CallerMemberName, which fills a parameter with the calling member's name, and JsonPropertyName, which maps a property to its JSON name.

330. What does the Obsolete attribute do?

Beginner | #obsolete #attributes | reference

Obsolete marks a type or member as deprecated so the compiler emits a warning, or an error when IsError is set to true, at each use. You can supply a message and the flag to guide callers toward a replacement. It is a compile-time aid only and does not prevent use through reflection or dynamic code.

```
[Obsolete("Use NewMethod instead", error: true)]
public void OldMethod() { }
```

331. What is Activator.CreateInstance used for?

Beginner | #activator #reflection #instantiation | reference

Activator.CreateInstance creates an instance of a type at runtime, either from a Type object or from an assembly-qualified name, optionally passing constructor arguments. It is convenient for plugin or factory scenarios where the concrete type is not known at compile time. Because it uses reflection it is slower than a direct constructor call, and it throws MissingMethodException if no matching constructor exists.

```
var instance = Activator.CreateInstance(typeof(Foo), arg1, arg2);
```

332. What is the Conditional attribute?

Beginner | #conditional #attributes #debugging | reference

Conditional is a method attribute that tells the compiler to include calls to the method only when a specified preprocessor symbol is defined. It is most often used with DEBUG so that diagnostic or logging helpers compile away entirely in release builds. Methods marked Conditional must return void and cannot be used with out parameters or as overrides.

```
[Conditional("DEBUG")]
void Log(string message) => Console.WriteLine(message);
```

333. What does the Serializable attribute do in .NET?

Beginner | #serializable #attributes #serialization | reference

Serializable indicates that a type can be serialized by classic binary or SOAP formatters, which was the default remoting and binary serialization mechanism. It has no effect on modern serializers such as System.Text.Json or XML serialization, which use public properties instead. Because BinaryFormatter is obsolete and unsafe, the attribute is now largely a legacy marker and should not be relied on for new code.

334. What is the nameof operator and why is it preferred over string literals?

Beginner | #nameof #refactoring | reference

nameof is a compile-time operator that returns the unqualified string name of a symbol such as a variable, type, member, or parameter. Because the compiler resolves it, the value stays correct after refactoring and produces a build error if the symbol is renamed or removed. It is commonly used in ArgumentNullException, property-change notifications, and logging to avoid magic strings.

```
throw new ArgumentNullException(nameof(value));
```

335. What is the dynamic keyword?

Beginner | #dynamic #dlr | reference

dynamic declares a variable whose operations are resolved at runtime by the Dynamic Language Runtime instead of by the compiler. It allows calls that the compiler cannot verify, such as invoking members of a COM object or a dynamic language, and failures surface as RuntimeBinderException at execution time. It is convenient but bypasses compile-time checking and adds runtime dispatch overhead. The ExpandoObject type is a dictionary-like object whose members can be added or removed at runtime and is commonly used together with dynamic.

```
dynamic d = GetObject();
d.DoSomething(42); // resolved at runtime
```

336. How do you get and invoke members with reflection?

Intermediate | #getmethod #bindingflags #invoke | reference reference

You use GetMethod, GetProperty, GetField, and similar methods on Type, passing a name and optional BindingFlags to control visibility and static versus instance lookup. For overloaded methods you can pass parameter types to disambiguate. You then invoke with MethodInfo.Invoke for methods, PropertyInfo.SetValue or GetValue for properties, and wrap calls in try/catch because reflection surfaces the target's exceptions wrapped in TargetInvocationException.

```
var method = typeof(Calculator).GetMethod("Add",
    BindingFlags.Public | BindingFlags.Instance);
var result = method.Invoke(calc, new object[] { 2, 3 });
```

337. What are BindingFlags and why are they needed?

Intermediate | #bindingflags #reflection | reference

BindingFlags is a flags enumeration that selects which members a reflection lookup returns and how matching is performed. By default GetMethod and GetProperty consider only public members, so you must add flags like NonPublic to reach private or internal members and Instance or Static to choose the kind. Flags such as IgnoreCase, DeclaredOnly, and FlattenHierarchy further refine the search.

```
var priv = typeof(Foo).GetField("_secret",
    BindingFlags.NonPublic | BindingFlags.Instance);
```

338. How do you create and apply a custom attribute?

Intermediate | #custom-attributes #attributes | reference

Define a class that derives from System.Attribute, conventionally with a name ending in Attribute, and provide constructors or settable properties for the data it carries. Then apply it with square brackets wherever the AttributeUsage allows. At runtime you retrieve instances with methods such as GetCustomAttribute or GetCustomAttributes, which construct attribute objects from the stored metadata.

```
[AttributeUsage(AttributeTargets.Class)]
public sealed class PluginAttribute : Attribute
{
    public string Name { get; }
    public PluginAttribute(string name) => Name = name;
}
```

339. What is AttributeUsage and what does it control?

Intermediate | #attributeusage #attributes | reference

AttributeUsage is itself an attribute applied to an attribute class to declare where the attribute may be used, whether it can be applied more than once, and whether derived types inherit it. The AttributeTargets flags restrict application to classes, methods, properties, parameters, and so on, AllowMultiple enables repetition, and Inherited controls propagation to derived types. Without it, an attribute defaults to all targets, single use, and inherited.

340. How do you read attributes at runtime?

Intermediate | #attributes #getcustomattribute #reflection | reference

You call methods such as GetCustomAttribute, GetCustomAttributes, or the Attribute.IsDefined helper on a Type or MemberInfo obtained through reflection. These methods materialize attribute instances by running their constructors and setting named properties from metadata, so the constructor logic executes each time you query. For performance-sensitive code, cache the results or use IsDefined when you only need to know whether the attribute is present.

```
var attr = typeof(Foo).GetCustomAttribute<PluginAttribute>();
if (attr is not null) Console.WriteLine(attr.Name);
```

341. What are the performance costs of reflection and how do you mitigate them?

Intermediate | #reflection #performance #caching | reference reference

Reflection is far slower than direct calls because member lookup, argument marshalling, and invocation all go through runtime machinery, and boxing occurs when value types are passed as object arguments. Mitigations include caching Type and MemberInfo objects, caching compiled delegates built with Delegate.CreateDelegate, using Expression trees compiled to delegates, and relying on source generators for AOT-friendly code. Avoiding reflection in hot loops and preferring generic or interface-based designs usually matters most.

342. What is the difference between expression trees and reflection?

Intermediate | #expression-trees #reflection #performance | reference

Expression trees represent code as data in the System.Linq.Expressions namespace and can be compiled into delegates at runtime with Compile, so after the one-time compilation cost they run at near-native speed. Reflection invokes members interpretively and is slower but works uniformly on any member without building an expression. Libraries often use reflection once to build an expression tree and then cache the compiled delegate for subsequent calls.

```
Expression<Func<Foo, int>> e = f => f.Value;
var getter = e.Compile();
int v = getter(foo);
```

343. How do you load an assembly and what is AssemblyLoadContext?

Intermediate | #assembly #assemblyloadcontext #plugins | reference reference

Assembly.Load loads by an assembly name, Assembly.LoadFrom loads from a path and can resolve dependencies, and Assembly.LoadFile loads without resolving from the load context. AssemblyLoadContext, introduced in .NET Core, provides an isolated loading context with explicit control over resolving dependencies and allows unloading of assemblies via its Unload method. This isolation is important for plugins that must load, version, and later release their own dependencies. An assembly also exposes its modules through GetModules, since a multi-module assembly groups its types into one or more modules.

```
        var alc = new AssemblyLoadContext("plugin", isCollectible: true);
        var asm = alc.LoadFromAssemblyPath(path);
```

344. What is an assembly-qualified name?

Intermediate | #assembly-qualified-name #type.gettype | reference

An assembly-qualified name uniquely identifies a type across assemblies by combining the type's full name with the display name of the assembly, including its version, culture, and public key token. It is the string form used by Type.GetType and by configuration when a type must be located without a compile-time reference. Using a partial name works only for types in mscorlib or the calling assembly, so fully qualifying is safer for external types.

```
        var t = Type.GetType(
          "System.Text.StringBuilder, System.Runtime");
```

345. How do you use reflection with generic types, such as MakeGenericType?

Intermediate | #generics #makegenerictype #reflection | reference

You obtain the open generic definition with typeof(List<>) and then call MakeGenericType with the type arguments to produce a closed constructed type, after which you can create instances or invoke members. To inspect generics, IsGenericType, IsGenericTypeDefinition, GetGenericArguments, and GetGenericTypeDefinition help navigate between open and closed forms. Constraint violations throw ArgumentException at the point of construction.

```
        var open = typeof(Dictionary<,>);
        var closed = open.MakeGenericType(typeof(string), typeof(int));
        var dict = Activator.CreateInstance(closed);
```

346. How do you check assignability and inheritance with IsAssignableFrom, is, and as?

Advanced | #isassignablefrom #is #as | reference reference

The is operator performs a runtime type test that can also pattern-match and declare a variable, while as performs a safe cast that yields null on failure for reference or nullable types. Type.IsAssignableFrom answers whether an instance of one type could be assigned to a variable of another, which is useful when you only have Type objects from reflection. For interfaces and base classes, use IsAssignableFrom or the is pattern rather than comparing types for equality.

```
        if (typeof(IEnumerable).IsAssignableFrom(t)) { }
        if (obj is string s) { }
        var str = obj as string;
```

347. What is Reflection.Emit used for?

Advanced | #reflection-emit #il #dynamic-assembly | reference

System.Reflection.Emit builds assemblies, types, and methods at runtime by emitting IL instructions into a dynamic assembly. It underpins advanced scenarios such as custom serializers, proxies, and mocking frameworks that generate strongly typed code for speed. Emit is complex and largely unavailable or restricted under Native AOT and trimming, so modern alternatives like source generators and expression trees are usually preferred.

348. How do source generators compare with reflection?

Advanced | #source-generators #reflection #aot | reference reference

Source generators run during compilation, inspect the user's code through the Roslyn API, and emit additional C# source that is compiled into the assembly. Because the generated code is ordinary compiled code, it works under trimming and Native AOT and avoids runtime reflection overhead. Reflection remains necessary for genuinely dynamic scenarios, but for repetitive, discoverable work such as JSON serialization contexts, logging, or mapping, generators are the modern recommendation.

349. Why does reflection break under trimming and Native AOT, and how do you address it?

Advanced | #trimming #aot #reflection | reference reference

The trimmer removes code that appears unused so it can shrink the published app, but it cannot always see members accessed only through reflection, so those members may be trimmed away and fail at runtime. Native AOT compiles ahead of time and has no JIT, so dynamically generated code from Reflection.Emit is unavailable. You mitigate this with DynamicallyAccessedMembers annotations, the RequiresUnreferencedCode and RequiresDynamicCode attributes, trimming roots or descriptors, and by preferring source generators that produce static code.

350. What are the trade-offs between dynamic and reflection?

Advanced | #dynamic #reflection #dlr | reference reference

dynamic resolves member access at runtime through the DLR, giving concise syntax that reads like static code and interoperates with COM and dynamic languages, but it loses compile-time checking, IntelliSense, and performance. Reflection exposes metadata explicitly and is more verbose but allows inspection, caching, and invocation of members that are not known at compile time, including private ones. Choose dynamic for ergonomic late binding and reflection when you need to discover or manipulate types generically; both require runtime validation and neither is friendly to trimming.

Modern C# Features

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

351. What is a record in C#?

Beginner | #records #equality #immutability | reference

A record is a reference type that the compiler gives value-based equality, a readable ToString, and support for non-destructive mutation via with-expressions. You can declare one with positional parameters, which also generates public init-only properties and a primary constructor. Records are ideal for immutable data and DTOs, while a record struct, added in C# 10, provides the same value semantics as a value type.

```
public record Person(string FirstName, string LastName);
var a = new Person("Ada", "Lovelace");
var b = a with { LastName = "Byron" };
```

352. What are init-only setters?

Beginner | #init #immutability | reference

An init accessor allows a property to be assigned only during object initialization, such as in an object initializer or constructor, after which it becomes read-only. This lets you build immutable objects without writing a constructor for every property combination. init replaces the older pattern of get-only properties set through constructors and works with records and with-expressions.

```
public class Point
{
    public int X { get; init; }
    public int Y { get; init; }
}
```

353. How do you use a with-expression to perform non-destructive mutation?

Beginner | #with-expression #records | reference

A with-expression performs non-destructive mutation by copying an existing record or struct and overriding selected properties, leaving the original unchanged. It uses the compiler-generated copy constructor and object initializer syntax. This is the idiomatic way to produce modified copies of immutable data.

```
var p1 = new Person("Ada", "Lovelace");
var p2 = p1 with { LastName = "Byron" };
```

354. What are top-level statements?

Beginner | #top-level-statements #entry-point | reference

Top-level statements let you write executable code directly at the start of a file without an explicit Main method or Program class. The compiler synthesizes the entry point, and any using directives or type declarations can still follow. It reduces boilerplate for small programs, console apps, and samples while remaining fully compatible with the rest of the language.

```
Console.WriteLine("Hello, world!");
int x = 42;
Console.WriteLine(x);
```

355. What are file-scoped namespaces?

Beginner | #namespaces #file-scoped | reference

A file-scoped namespace declares a namespace for the entire file with a single statement ending in a semicolon, removing the need to indent everything inside a block. It reduces nesting and is the default style in modern templates. Only one file-scoped namespace may appear per file.

```
namespace MyApp.Services;

public class Foo { }
```

356. What are global usings and implicit usings?

Beginner | #global-usings #implicit-usings | reference

A global using directive, written as global using, makes a namespace available to every file in the project so you do not repeat the same using in each file. Implicit usings are a project setting, enabled by default in modern templates, that adds a curated set of common namespaces such as System and System.Linq automatically. Both reduce boilerplate while keeping the imports visible and controllable.

```
global using System.Text.Json;
// ImplicitUsings enabled via <ImplicitUsings>enable</ImplicitUsings>
```

357. What is target-typed new?

Beginner | #target-typed-new #syntax | reference

Target-typed new lets you write new() without repeating the type name when the target type is known from the context, such as a variable declaration, assignment, field, or argument. It reduces verbosity without changing behavior. It cannot be used when the target type is ambiguous, such as with var. C# 9 also added target-typed conditional expressions, where a conditional such as flag ? 1 : 0 infers its type from the assignment target, and C# 12 added using alias directives that can alias any type, not just a named type.

```
List<int> numbers = new();
Dictionary<string, int> map = new();
```

358. What is pattern matching in C#?

Beginner | #pattern-matching #is #switch | reference

Pattern matching tests a value against a shape and can bind parts of it to new variables. It appears in is expressions, switch statements, and switch expressions, and supports type patterns, constant patterns, property patterns, positional patterns, relational patterns, and logical combinators. It makes type checks and conditional logic more expressive and less error-prone than chained casts.

```
if (obj is string { Length: > 5 } s)
    Console.WriteLine(s);
```

359. What is a switch expression?

Beginner | #switch-expression #pattern-matching | reference

A switch expression returns a value from the first matching arm, using the expression form of switch with => arms and no fall-through. Arms can combine patterns, and the compiler warns if the arms do not cover all cases, which is useful with exhaustive matching. It is more concise and safer than a switch statement when you simply map an input to an output.

```
string Describe(int n) => n switch
{
    0 => "zero",
    < 0 => "negative",
    _ => "positive"
};
```

360. What are ranges and indices in C#?

Beginner | #ranges #indices #slicing | reference

Index, written with the ^ operator, counts from the end, so ^1 is the last element. Range, written with .., selects a sub-sequence and can be open-ended, such as 1..^1 or ..3. Both work on arrays, strings, and spans through compiler support and the Index and Range types, making slicing concise.

```
int[] a = { 1, 2, 3, 4, 5 };
var last = a[^1];
var middle = a[1..^1];
```

361. How do property patterns and relational patterns work?

Intermediate | #property-patterns #relational-patterns #logical-patterns | reference

Property patterns match on the values of a value's properties, such as Person { Age: >= 18 }, and can nest to arbitrary depth. Relational patterns use the operators <, >, <=, and >= to compare an input to a constant, and logical patterns combine them with and, or, and not. Together they express rich conditions directly in switch arms or is expressions without intermediate variables.

```
string Category(Person p) => p switch
{
    { Age: < 13 } => "child",
    { Age: >= 13 and < 20 } => "teen",
    _ => "adult"
};
```

362. What are list patterns and when were they introduced?

Intermediate | #list-patterns #csharp-11 | reference

List patterns, added in C# 11, match sequences such as arrays and spans by position and length. A discard matches one element, a slice pattern with .. matches zero or more elements, and the pattern requires an exact length unless a slice is present. They are useful for matching on the start or end of a sequence without manual index checks.

```
int[] data = { 1, 2, 3 };
if (data is [1, .., 3]) { }
```

363. What are raw string literals?

Intermediate | #raw-string-literals #csharp-11 | reference

Raw string literals, added in C# 11, use three or more double quotes to define a string where backslashes and quotes are taken literally and no escaping is needed. The closing delimiter's indentation is stripped from every line, which keeps multi-line JSON, SQL, or regex readable and correctly indented. More opening quotes allow embedded sequences of quotes in the content.

```
var json = """
    {
        "name": "Ada"
    }
    """;
```

364. What are required members?

Intermediate | #required #csharp-11 | reference

The required modifier, introduced in C# 11, marks a property or field that callers must set during object creation. The compiler enforces this by requiring an object initializer or a constructor annotated with SetsRequiredMembers, otherwise it produces an error. Required members let you enforce initialization of essential state without forcing every value through a constructor.

```
public class Config
{
    public required string Name { get; init; }
}
var c = new Config { Name = "app" };
```

365. What are primary constructors for classes and structs?

Intermediate | #primary-constructors #csharp-12 | reference

Introduced in C# 12, primary constructors let a class or struct declare constructor parameters in its declaration, such as class Person(string name). The parameters are in scope throughout the type's body and can initialize fields or properties, but unlike records they do not automatically become properties. They reduce boilerplate for capturing values used by the type without exposing them publicly.

```
public class Logger(string category)
{
    public void Log(string msg) =>
        Console.WriteLine($"[{category}] {msg}");
}
```

366. What are collection expressions?

Intermediate | #collection-expressions #csharp-12 | reference

Collection expressions, added in C# 12, use square brackets to initialize collections, arrays, and spans uniformly, for example int[] a = [1, 2, 3]. The spread element .. inserts the contents of another collection, and the compiler chooses an efficient construction path for the target type. They unify syntax across arrays, List, spans, and types with a collection builder.

```
int[] a = [1, 2, 3];
int[] b = [0, .. a, 4];
List<string> names = ["a", "b"];
```

367. What is the null-forgiving operator and how does it relate to nullable reference types?

Intermediate | #nullable-reference-types #null-forgiving | reference reference

Nullable reference types, enabled with enable, make the compiler track null-state and emit warnings for possible null dereferences and unnecessary null checks. The null-forgiving operator ! suppresses a warning when you know a value is not null but the compiler cannot prove it. These warnings are compile-time only and do not add runtime checks or change behavior.

```
string? maybe = GetValue();
int len = maybe!.Length; // suppress warning
```

368. What are tuples and deconstruction in modern C#?

Intermediate | #tuples #deconstruction | reference reference

Tuples group several values into a lightweight value type, and named tuple elements like (int Id, string Name) make the code self-documenting. Deconstruction assigns the elements to separate variables in one statement, and types can support it by providing a Deconstruct method. Tuples are convenient for returning multiple values without defining a type, though they should not replace a named type when the data has broader meaning.

```
(int Id, string Name) Get() => (1, "Ada");
var (id, name) = Get();
```

369. What are local functions and how do they compare to lambdas?

Intermediate | #local-functions #lambdas | reference

A local function is a named function declared inside another method, capturing variables from the enclosing scope like a lambda but without allocating a delegate unless it is converted to one. Local functions support recursion, iterator and async modifiers, and generic parameters, and they are ideal for private helpers that belong to one method. Lambdas are anonymous expressions better suited to passing as arguments. Lambdas themselves gained natural types, so you can write var f = (int x) => x * 2, and default parameter values, and both lambdas and local functions pair well with nameof to avoid magic strings.

```
int Factorial(int n)
{
    int Fact(int x) => x <= 1 ? 1 : x * Fact(x - 1);
    return Fact(n);
}
```

370. What is a using declaration and how does it differ from a using statement?

Intermediate | #using-declaration #disposable | reference

A using declaration, introduced in C# 8, declares a disposable variable whose Dispose is called automatically at the end of the enclosing scope, without an extra indentation block. It is equivalent to a using statement but reads more linearly, especially with several resources. Both are preferred over manual try/finally, and await using does the same for IAsyncDisposable.

```
using var stream = File.OpenRead("data.txt");
// disposed at end of method
```

371. What are static abstract interface members and how do they enable generic math?

Advanced | #static-abstract #generic-math #csharp-11 | reference reference

Introduced in C# 11, interfaces can declare static abstract members that implementing types must provide, enabling polymorphism over static operations. Combined with .NET 7 generic math interfaces such as INumber, this allows generic algorithms that use operators, for example T Add(T a, T b) where T : INumber => a + b. It also underlies features like generic parsing and lets you constrain to interfaces that expose operators and constants.

```
static T Sum<T>(IEnumerable<T> values) where T : INumber<T>
{
    T total = T.Zero;
    foreach (var v in values) total += v;
    return total;
}
```

372. What are ref structs and ref fields?

Advanced | #ref-struct #ref-fields #csharp-11 | reference reference

A ref struct is a stack-only type that cannot be boxed, stored on the heap, or used as a type argument, which lets it hold refs into other memory safely, as Span does. C# 11 added ref fields so a ref struct can contain references to other storage, and the scoped and ref-safety rules ensure those references never outlive their target. These rules are why you cannot use a Span in an async method or a field of an ordinary class. C# 12 also added ref readonly parameters, which pass a reference for efficiency while preventing the callee from modifying the value.

```
ref struct Span2<T>
{
    private ref T _first;
}
```

373. What are inline arrays in C# 12?

Advanced | #inline-arrays #csharp-12 #performance | reference

Inline arrays let a struct declare a fixed-size array of a known length using the System.Runtime.CompilerServices.InlineArray attribute on a single field. The compiler generates an indexer and length, so the array is embedded directly in the struct without a separate heap allocation, which is valuable for high-performance buffer scenarios and interop. You access elements by index just like a normal array.

```
[InlineArray(10)]
public struct Buffer10<T>
{
    private T _element0;
}
var b = new Buffer10<int>();
b[0] = 42;
```

374. What are default interface methods and how do they differ from abstract members?

Advanced | #default-interface-methods #interfaces #csharp-8 | reference

Default interface methods, added in C# 8, let an interface provide a method body that implementing types inherit unless they override it, enabling interface evolution without breaking existing implementers. They are not virtual members on a class hierarchy in the usual sense and are only reachable through the interface, not through a class reference. Use them to add behavior or helper members to a published interface, while remembering they can introduce ambiguity with multiple inheritance of interfaces. On structs, a member marked readonly guarantees it does not mutate the instance, which lets the compiler skip defensive copies when calling it.

```
        public interface ILogger
        {
            void Log(string msg);
            void LogInfo(string msg) => Log($"INFO: {msg}");
        }
```

375. What are interpolated string handlers and how do they improve performance?

Advanced | #interpolated-string-handlers #performance #csharp-10 | reference reference

Interpolated string handlers, added in C# 10, let an interpolated string be processed without first allocating a string when the target type provides a handler, as the compiler rewrites the interpolation into calls the handler can optimize. This enables APIs that build the final text into a destination such as a StringBuilder or a Span, avoiding intermediate allocations. In .NET 6+ string interpolation to a string uses the default handler for efficiency, and custom handlers follow the pattern of AppendLiteral and AppendFormatted methods. For UTF-8 text, C# 11 also added UTF-8 string literals, where a string suffixed with u8 is encoded as a ReadOnlySpan with no allocation.

```
        var sb = new StringBuilder();
        sb.Append($"Value: {42}");
```

Design Patterns

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

376. What does the Single Responsibility Principle (SRP) state?

Beginner | #solid #srp #cohesion | reference

SRP states that a class should have only one reason to change, meaning it should own a single cohesive responsibility rather than several unrelated concerns. A class that both calculates business rules and persists to a database has two reasons to change and should be split. Following SRP makes types smaller, easier to test, and less likely to break when an unrelated requirement changes.

```
// Violates SRP: validation + persistence + email
public class OrderService
{
    public void Place(Order o)
    {
        Validate(o);
        Save(o);
        SendEmail(o);
    }
}
```

377. What is the Open/Closed Principle (OCP)?

Beginner | #solid #ocp #extensibility | reference

OCP says software entities should be open for extension but closed for modification, so new behavior is added by adding code rather than editing and re-testing existing code. Abstractions such as interfaces, abstract base classes, and virtual members are the usual mechanisms. For example, a discount calculator can be extended with a new IDiscount implementation instead of adding another branch to a switch statement.

```
public interface IDiscount
{
    decimal Apply(decimal total);
}

public class VipDiscount : IDiscount
{
    public decimal Apply(decimal total) => total * 0.9m;
}
```

378. What is the Liskov Substitution Principle (LSP)?

Beginner | #solid #lsp #inheritance | reference

LSP requires that objects of a derived type be usable anywhere the base type is expected without breaking the program's correctness. Subtypes must not strengthen preconditions, weaken postconditions, or throw exceptions the base type does not declare. A classic violation is a Square derived from Rectangle, because setting width and height independently cannot hold for a square.

379. What is the Interface Segregation Principle (ISP)?

Beginner | #solid #isp #interfaces | reference

ISP says clients should not be forced to depend on methods they do not use, so large general-purpose interfaces should be split into smaller, role-specific ones. An IWorker interface with Work, Eat, and Sleep forces a robot implementation to throw for Eat, which is a design smell. Small focused interfaces keep implementations honest and make mocking in tests simpler.

```
public interface IWork { void Work(); }
public interface IEat { void Eat(); }

public class Robot : IWork
{
    public void Work() { }
}
```

380. What is the Dependency Inversion Principle (DIP)?

Beginner | #solid #dip #abstractions | reference reference

DIP states that high-level modules should not depend on low-level modules; both should depend on abstractions, and abstractions should not depend on details. Instead of a business service directly creating a database connection, it depends on an interface such as IOrderRepository that the infrastructure layer implements. This inversion enables testing with fakes and swapping implementations without touching business logic.

```
public class OrderService(IOrderRepository repo)
{
    public void Place(Order o) => repo.Add(o);
}
```

381. What is the difference between Factory Method and Abstract Factory?

Beginner | #factory-method #abstract-factory #creational | reference

Factory Method creates a single product through a method that subclasses or callers can override, while Abstract Factory creates families of related products through an interface with multiple creation methods. Abstract Factory is useful when products must be used together, such as a UI toolkit producing matching buttons and checkboxes for Windows or macOS. Both hide concrete types from clients, but Abstract Factory groups several related factories behind one abstraction.

```
public interface IUiFactory
{
    IButton CreateButton();
    ICheckbox CreateCheckbox();
}
```

382. What is the Builder pattern?

Beginner | #builder #creational #fluent-api | reference

Builder separates the construction of a complex object from its representation, letting the same construction process create different representations. It provides a fluent step-by-step API that avoids telescoping constructors with many optional parameters and keeps the built object immutable. In C#, the pattern is often combined with object initializers or a static Create method that returns a fully configured instance.

```
var request = new HttpRequestBuilder()
    .WithUrl("https://example.com")
    .WithMethod(HttpMethod.Get)
    .Build();
```

383. What is the Singleton pattern and why is it often considered an anti-pattern?

Beginner | #singleton #anti-pattern #di | reference

Singleton guarantees a class has exactly one instance and provides a global access point to it, typically via a private constructor and a static Instance property. It is often an anti-pattern because it introduces hidden global state, makes unit testing hard since the instance cannot easily be replaced, and tightly couples callers to a concrete type. In modern .NET the same goal is better achieved by registering a service as a singleton with the built-in DI container and injecting an abstraction.

```
public sealed class Cache
{
    public static Cache Instance { get; } = new();
    private Cache() { }
}
```

384. What is the Prototype pattern and how does C# support it?

Beginner | #prototype #clone #records | reference

Prototype creates new objects by cloning an existing instance rather than calling a constructor, which is useful when construction is expensive or the configuration is complex. C# supports it with the ICloneable interface, though a typed Clone method is preferred, and with record types whose compiler-generated copy constructor makes non-destructive copies simple. A shallow clone copies references while a deep clone duplicates the referenced objects too.

```
public record Person(string Name, List<string> Tags);
var copy = original with { };
var deep = original with { Tags = [.. original.Tags] };
```

385. What is the Adapter pattern?

Beginner | #adapter #structural #integration | reference

Adapter converts the interface of a class into another interface that clients expect, letting incompatible types work together without modifying their source. It is common when integrating a third-party library or legacy component behind your own abstraction. In C#, an adapter is usually a small class that implements your interface and delegates to the incompatible type, which also makes that dependency mockable in tests.

```
public class LegacyLoggerAdapter(ILegacyLog log) : IAppLogger
{
    public void Info(string msg) => log.Write("INFO", msg);
}
```

386. How do you implement a thread-safe Singleton in C#?

Intermediate | #singleton #thread-safety #lazy | reference reference

The cleanest thread-safe approach relies on the runtime's type-initialization guarantees: a static readonly field initialized inline, or a Lazy with LazyThreadSafetyMode.ExecutionAndPublication, gives one instance that is created lazily and safely without explicit locks. A static constructor is also thread-safe because the CLR guarantees it runs once, and double-checked locking with a volatile field works but is easy to get wrong. Note that thread-safe construction does not make the singleton's own members thread-safe, so shared mutable state still needs synchronization.

```
public sealed class Cache
{
    private static readonly Lazy<Cache> _lazy =
        new(() => new Cache(), LazyThreadSafetyMode.ExecutionAndPublication);
    public static Cache Instance => _lazy.Value;
    private Cache() { }
}
```

387. What is the difference between the Decorator and Proxy patterns?

Intermediate | #decorator #proxy #structural | reference reference

Both wrap an object behind the same interface, but Decorator adds responsibilities such as logging, caching, or compression and receives the wrapped object from outside, while Proxy controls access to a subject and typically creates or locates that subject itself. Decorators are composed in a chain and each layer adds behavior, whereas a proxy usually stands in for one real subject to add lazy initialization, security, or remoting. In .NET, Stream wrappers like GZipStream are decorators, while a lazy-loading EF Core navigation property acts as a proxy.

```
public class CachingRepo(IOrderRepository inner) : IOrderRepository
{
    public Order? Get(int id) => _cache.GetOrAdd(id, inner.Get);
}
```

388. What is the Facade pattern?

Intermediate | #facade #structural #simplification | reference

Facade provides a simplified, unified interface to a complex subsystem, hiding many collaborating classes behind a few high-level operations. It does not change existing interfaces for compatibility, unlike Adapter, but instead reduces the number of types a client must interact with. A common .NET example is an application service that orchestrates several repositories and external clients behind one method call.

389. What is the Composite pattern?

Intermediate | #composite #tree #structural | reference

Composite composes objects into tree structures to represent part-whole hierarchies and lets clients treat individual objects and groups uniformly. A node implements the same interface as its leaves and forwards operations to its children, which is ideal for file systems and UI element trees. In .NET, expression trees and the node types in System.Linq.Expressions follow the same recursive structure.

```
public interface INode { int Size(); }
public class Folder(List<INode> children) : INode
{
    public int Size() => children.Sum(c => c.Size());
}
```

390. What is the Bridge pattern?

Intermediate | #bridge #abstraction #composition | reference

Bridge decouples an abstraction from its implementation so both can vary independently, by holding a reference to an implementor interface rather than inheriting concrete behavior. This avoids a combinatorial explosion of subclasses when a hierarchy has two independent dimensions, such as shapes and rendering APIs. It is essentially preferring composition over inheritance for the varying part of a design.

```
public abstract class Shape(IRenderer r)
{
    protected IRenderer Renderer => r;
    public abstract void Draw();
}
```

391. What is the Flyweight pattern?

Intermediate | #flyweight #caching #memory | reference

Flyweight minimizes memory use by sharing as much data as possible between similar objects, splitting state into intrinsic data stored once and extrinsic data passed in by the client. It is valuable when a large number of objects share common immutable data, such as character glyphs in a document or tiles in a game. In .NET, string interning and cached immutable value objects are practical examples of the same idea.

```
private static readonly Dictionary<char, Glyph> Cache = new();
Glyph Get(char c) => Cache.TryGetValue(c, out var g)
    ? g : Cache[c] = new Glyph(c);
```

392. What is the Strategy pattern and how does it relate to delegates?

Intermediate | #strategy #behavioral #delegates | reference

Strategy defines a family of interchangeable algorithms, encapsulates each one behind a common interface, and lets the client select one at runtime. It removes large conditional blocks from the context class and makes each algorithm independently testable. In modern C#, a strategy is often just a delegate such as Func or a keyed DI registration, which avoids defining a type per algorithm.

```
public interface IShipping { decimal Cost(Order o); }

public class Checkout(IShipping shipping)
{
    public decimal Total(Order o) => shipping.Cost(o);
}
```

393. What is the Observer pattern and how is it implemented in .NET?

Intermediate | #observer #events #behavioral | reference reference

Observer defines a one-to-many dependency so that when one object changes state, all its dependents are notified automatically. In .NET this is most commonly implemented with events and delegates, where a publisher exposes an event and subscribers attach handlers, or with IObservable and IObserver for a push-based stream. Care must be taken to unsubscribe handlers to avoid memory leaks, which is a common cause of objects being kept alive.

```
public class Stock
{
    public event EventHandler<decimal>? PriceChanged;
    public void Update(decimal p) => PriceChanged?.Invoke(this, p);
}
```

394. What is the Command pattern?

Intermediate | #command #behavioral #undo | reference

Command encapsulates a request as an object, allowing it to be parameterized, queued, logged, or undone. Each command implements a common interface with an Execute method, and an invoker holds and triggers commands without knowing their details. It underpins undo/redo stacks, task queues, and message-based systems, and in C# a command is frequently a small class implementing an interface such as ICommand in MVVM frameworks.

```
public interface ICommand { void Execute(); }
public class SaveCommand(Document d) : ICommand
{
    public void Execute() => d.Save();
}
```

395. How does the Iterator pattern work in C#?

Intermediate | #iterator #yield #ienumerable | reference reference

Iterator provides a way to access the elements of a collection sequentially without exposing its underlying representation. C# builds the pattern into the language through IEnumerable and IEnumerator, and the yield return statement lets the compiler generate the enumerator state machine for you. Because the compiler transforms an iterator method into a class, iterator methods are lazy and only produce elements as they are requested.

```
public static IEnumerable<int> Evens(int max)
{
    for (int i = 0; i <= max; i += 2)
        yield return i;
}
```

396. How do the State and Template Method patterns differ?

Advanced | #state #template-method #behavioral | reference

State models an object whose behavior changes as its internal state changes, with state objects that typically trigger transitions to other states, whereas Template Method defines the fixed skeleton of an algorithm in a base class and defers individual steps to overridden abstract or virtual members. State is about delegating behavior to a swap-in object at runtime, and Template Method is about reusing an algorithm's structure across subclasses. State is usually implemented with a context holding a current state reference, while Template Method relies on inheritance and a non-virtual public method that calls the protected steps.

```
public interface IState { IState Next(); void Handle(); }

public abstract class Importer
{
    public void Run() { var d = Read(); Store(d); }
    protected abstract string Read();
    protected abstract void Store(string d);
}
```

397. What are the Chain of Responsibility and Mediator patterns?

Advanced | #chain-of-responsibility #mediator #middleware | reference reference

Chain of Responsibility passes a request along a sequence of handlers until one handles it or the chain ends, decoupling the sender from the concrete handler; ASP.NET Core's middleware pipeline is a canonical implementation where each component can handle the request or call the next delegate. Mediator centralizes complex communication between objects through a single hub so that colleagues do not reference each other directly, and libraries such as MediatR apply it to route requests to handlers. Both reduce tight coupling, but Chain of Responsibility is a linear pass-along structure while Mediator is a star topology around one coordinator.

```
app.Use(async (ctx, next) =>
{
    // before
    await next();
    // after
});
```

398. What are the Visitor and Memento patterns?

Advanced | #visitor #memento #behavioral | reference reference

Visitor adds new operations to a type hierarchy without modifying the types, using double dispatch so an accept method on each element calls the matching visit method on the visitor; it is useful when the set of element types is stable but the operations vary. Memento captures an object's internal state so it can be restored later without exposing its internals, which is the basis for undo and rollback. In C#, records and with-expressions make lightweight mementos easy to produce as immutable snapshots.

```
public interface IVisitor { void Visit(Circle c); }
public interface IShape { void Accept(IVisitor v); }
```

399. What are the Repository, Unit of Work, and dependency injection patterns?

Advanced | #repository #unit-of-work #dependency-injection | reference reference

Repository abstracts data access behind a collection-like interface so business code depends on an abstraction instead of an ORM or database, and Unit of Work tracks changes across repositories and commits them in a single transaction, which EF Core's DbContext already implements. Dependency injection is the technique of supplying a class's dependencies from outside rather than having it construct them; constructor injection is the default and is required for mandatory dependencies, property injection suits optional cross-cutting services, and method injection passes a dependency into a single method call. Constructor injection should be preferred because it makes dependencies explicit and leaves the object fully initialized on creation.

```
services.AddScoped<IOrderRepository, OrderRepository>();
services.AddScoped<IUnitOfWork, EfUnitOfWork>();

public class OrderService(IOrderRepository repo, IUnitOfWork uow) { }
```

400. How do patterns relate to modern C#, and why is Service Locator discouraged while Null Object is encouraged?

Advanced | #service-locator #null-object #modern-csharp | reference reference

Many classic patterns are now expressed directly in the language: the built-in DI container replaces hand-written factories and singletons, delegates and Func remove the need for one-type-per-algorithm strategies, and LINQ's expression trees and iterators provide the Visitor and Iterator patterns out of the box. Service Locator, where a class asks a global container for its dependencies at runtime, is discouraged because it hides dependencies from the constructor, complicates testing, and can only be understood by reading the implementation. The Null Object pattern instead provides a do-nothing implementation so callers avoid null checks, and it pairs naturally with DI by registering a NullLogger-style implementation as the default.

```
public sealed class NullLogger : IAppLogger
{
    public void Info(string msg) { }
}
services.AddSingleton<IAppLogger, NullLogger>();
```

Testing & Best Practices

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

401. What is the difference between unit, integration, and end-to-end tests?

Beginner | #test-levels #unit-testing #integration-testing | reference reference

A unit test exercises a single unit of code, usually one class or method, in isolation with its dependencies replaced by test doubles, and it runs fast and deterministically. An integration test verifies that several real components work together, such as your code plus a database or an HTTP endpoint, so it is slower and exercises more of the stack. An end-to-end test drives the whole system through its public surface, for example a browser against a running app, which gives the highest confidence but is the slowest and most brittle. A healthy test suite is dominated by unit tests, supported by fewer integration tests, and a thin layer of end-to-end tests.

402. What is xUnit and how do you write a basic test with Fact and Theory?

Beginner | #xunit #fact #theory | reference

xUnit.net is a free, open-source unit testing framework for .NET that is the default choice in modern templates. A method marked with [Fact] is a single test with no parameters, while [Theory] marks a data-driven test whose parameters come from attributes such as [InlineData]. The test runner discovers these attributes and executes each theory once per data row, reporting them as separate test cases.

```
public class CalculatorTests
{
    [Fact]
    public void Add_ReturnsSum() =>
        Assert.Equal(4, Calculator.Add(2, 2));

    [Theory]
    [InlineData(1, 2, 3)]
    [InlineData(-1, 1, 0)]
    public void Add_IsCorrect(int a, int b, int expected) =>
        Assert.Equal(expected, Calculator.Add(a, b));
}
```

403. What is the Arrange-Act-Assert pattern?

Beginner | #arrange-act-assert #test-structure | reference

Arrange-Act-Assert is a convention for structuring a test into three clearly separated phases: arrange the objects and inputs, act by invoking the behavior under test, and assert that the outcome matches expectations. It makes tests readable and keeps the intent obvious, and a well-structured test has a single Act with a focused set of assertions. Splitting the phases with blank lines or comments is a common style that reviewers and tools rely on.

```
var account = new Account(100);      // Arrange
account.Withdraw(40);                 // Act
Assert.Equal(60, account.Balance);    // Assert
```

404. What are test doubles, and what is the difference between mocks, stubs, fakes, and spies?

Beginner | #test-doubles #mocks #fakes | reference

A test double is any object that stands in for a real dependency during a test. A stub returns canned answers, a fake has a working but simplified implementation such as an in-memory repository, a spy records how it was called so the test can inspect it, and a mock is pre-programmed with expectations about which calls must occur. The practical distinction is whether the double mainly provides values, provides behavior, or verifies interactions.

405. What assertion methods does the built-in Assert class provide?

Beginner | #assertions #xunit | reference

The Assert class offers methods such as Equal, NotEqual, True, False, Null, NotNull, Same, Contains, and InRange for verifying outcomes, each with overloads for expected messages. Modern frameworks add Throws and ThrowsAsync to assert that a delegate throws a specific exception. Using the most specific assertion available produces clearer failure messages than a generic Assert.True comparison.

```
        Assert.Equal(3, list.Count);
        Assert.NotNull(result);
        Assert.Contains("key", dictionary.Keys);
```

406. How do xUnit, NUnit, and MSTest differ?

Beginner | #xunit #nunit #mstest | reference reference

xUnit.net uses constructor-based setup and IDisposable for teardown instead of attributes, and creates a new instance of the test class for every test, which enforces isolation by default. NUnit is attribute-heavy with [SetUp], [TearDown], and rich parameterization, and MSTest is the Microsoft-native framework with [TestClass] and [TestMethod] that integrates tightly with Visual Studio and the Microsoft.Testing.Platform. All three run on .NET and are supported by dotnet test, so the choice is mostly about team preference, ecosystem, and available tooling.

407. What is test isolation and why does it matter?

Beginner | #test-isolation #parallelism | reference

Test isolation means each test runs independently and its result does not depend on other tests or on the order in which they execute. Tests that share mutable static state, files, or a database can pass alone but fail together, which makes failures hard to reproduce. xUnit helps by instantiating the test class per test and by running test collections in parallel, so shared resources must be made thread-safe or confined to fixtures.

408. What makes a test deterministic, and how do you avoid flaky tests?

Beginner | #determinism #flaky-tests | reference reference

A deterministic test always produces the same result given the same code, with no dependence on the current time, random values, machine culture, network, or execution order. To avoid flakiness, inject abstractions for clocks and randomness such as TimeProvider or an IClock interface, avoid real network calls, seed any random generators, and never rely on Thread.Sleep for synchronization. Retrying a flaky test hides the underlying nondeterminism, so the fix should remove the source of the instability rather than mask it.

```
public class Service(TimeProvider clock)
{
    public bool IsOpen => clock.GetUtcNow().Hour is >= 9 and < 17;
}
```

409. What is TDD and how does it relate to BDD?

Beginner | #tdd #bdd #red-green-refactor | reference

Test-driven development is a cycle of writing a failing test first (red), making it pass with the simplest code (green), and then refactoring while the tests stay green. It drives design toward testable, loosely coupled code and provides a regression suite as a byproduct. Behavior-driven development extends the idea by describing behavior in business-readable language such as Given/When/Then, which tools like SpecFlow or Reqnroll map to executable tests.

410. What is code coverage and how should it be used?

Beginner | #code-coverage #coverlet | reference

Code coverage measures how much of your code is exercised by the tests, commonly reported as line and branch coverage. On .NET you can collect it with dotnet test --collect:"Code Coverage" or with Coverlet, and the output can be consumed by tools such as ReportGenerator. Coverage is a useful signal for finding untested code, but a high percentage does not prove correctness because a test can execute a line without asserting anything meaningful.

```
dotnet test --collect:"XPlat Code Coverage"
```

411. What are IClassFixture and ICollectionFixture in xUnit?

Intermediate | #xunit-fixtures #shared-context | reference

IClassFixture shares a single instance of a fixture across all tests in one test class, which is ideal for expensive setup such as starting an in-memory host once per class. ICollectionFixture shares a fixture across every test class in a collection, so several classes can reuse one resource such as a database container. Both are implemented by having the test class or collection definition request the fixture through the constructor or a CollectionDefinition attribute.

```
[CollectionDefinition("db")]
public class DbCollection : ICollectionFixture<DbFixture> { }

[Collection("db")]
public class RepoTests(DbFixture fx) { }
```

412. How do you write parameterized tests beyond InlineData?

Intermediate | #parameterized-tests #memberdata #theorydata | reference

InlineData only supports compile-time constants, so xUnit provides MemberData to source arguments from a static property or method and ClassData to source them from a type implementing IEnumerable. This lets you build cases from computed values, share a data set across tests, or generate many combinations. Each generated case runs and reports independently, and grouping with TheoryData gives compile-time type checking of the data.

```
public static TheoryData<int, int> Cases => new()
{
    { 1, 2 }, { 3, 4 }
};

[Theory]
[MemberData(nameof(Cases))]
public void Test(int a, int b) => Assert.True(a < b);
```

413. How do you test asynchronous code correctly?

Intermediate | #async-testing #await | reference

Test methods should be declared async Task rather than async void so the runner can await them and observe exceptions. Return the task instead of using .Result or .Wait(), which can deadlock or wrap exceptions in AggregateException, and await the assertion or use the framework's async assertion helpers. For timeouts, use a CancellationToken with a bounded delay so a hung test fails fast rather than blocking the run.

```
[Fact]
public async Task Fetch_ReturnsData()
{
    var result = await client.GetAsync("/data");
    result.EnsureSuccessStatusCode();
    Assert.Equal(HttpStatusCode.OK, result.StatusCode);
}
```

414. How do you test that code throws an exception?

Intermediate | #exceptions #assert-throws | reference

xUnit provides Assert.Throws and its async counterpart Assert.ThrowsAsync, which take a delegate and return the caught exception so you can assert on its message or properties. Asserting the exact exception type is important because Assert.Throws also fails if a different or derived type is thrown, unlike a looser check. Avoid try/catch with a manual Assert.Fail, since it is verbose and easy to write incorrectly.

```
var ex = Assert.Throws<ArgumentNullException>(() =>
    service.Get(null!));
Assert.Equal("id", ex.ParamName);
```

415. How do Moq and NSubstitute differ when creating mocks?

Intermediate | #moq #nsubstitute #mocking | reference

Moq builds a mock from an interface or virtual members and configures it with a fluent Setup and Returns API, then verifies calls with Verify, which reads as an explicit mock-first style. NSubstitute aims for a more natural syntax where you call methods on a substitute to configure them, for example repo.Get(1).Returns(item), and check interactions with Received(). Both generate dynamic proxies and support argument matchers, callbacks, and strict behavior. The choice is largely stylistic; Moq requires the mocked members to be virtual on classes, and neither can mock non-virtual, static, or sealed members without an alternative such as a hand-written fake.

```
var repo = Substitute.For<IOrderRepository>();
repo.Get(1).Returns(new Order());
var svc = new OrderService(repo);
svc.Place(new Order());
repo.Received().Add(Arg.Any<Order>());
```

416. When is mocking considered a code smell?

Intermediate | #mocking #test-smells #design | reference

Heavy mocking is a smell when a test must set up many unrelated expectations just to reach the behavior under test, which usually signals that the class has too many responsibilities or depends on concrete types instead of abstractions. Verifying every interaction also couples tests to implementation details, so refactoring breaks tests even though behavior is unchanged. Prefer testing observable outcomes, use fakes or in-memory implementations for collaborators, and reserve strict interaction verification for cases where the interaction itself is the contract.

417. What is FluentAssertions and why use it?

Intermediate | #fluentassertions #assertions | reference

FluentAssertions is a third-party library that replaces Assert calls with a readable fluent syntax such as result.Should().Be(3) and collection.Should().ContainSingle(). It produces detailed failure messages that show the actual and expected values and, for collections and objects, the specific difference. Its extensibility allows custom assertions for domain types, which keeps tests expressive, though some teams avoid the extra dependency in favor of built-in asserts.

```
result.Should().Be(3);
orders.Should().HaveCount(2)
    .And.OnlyContain(o => o.Total > 0);
```

418. What is a test data builder and why is it useful?

Intermediate | #test-data-builders #maintainability | reference

A test data builder is a helper that constructs valid domain objects with sensible defaults and lets a test override only the properties relevant to that test. It removes the noise of repeating large object initializers and makes the intent of each test obvious, because you can see exactly which field matters. Builders reduce the cost of adding a new required property, since only the default changes and existing tests keep compiling.

```
var order = new OrderBuilder()
    .WithTotal(100m)
    .WithStatus(OrderStatus.Paid)
    .Build();
```

419. How do you write integration tests for an ASP.NET Core app with WebApplicationFactory?

Intermediate | #integration-testing #webapplicationfactory #aspnetcore | reference reference

WebApplicationFactory boots your application in memory using the real hosting pipeline and exposes an HttpClient that sends requests to it without a network listener. You typically subclass it to override the host configuration, such as replacing a database registration with a test double or in-memory provider, and then assert on real HTTP responses. This approach exercises routing, model binding, filters, and middleware, giving much higher confidence than unit tests while staying fast and self-contained.

```
public class ApiTests(WebApplicationFactory<Program> factory)
    : IClassFixture<WebApplicationFactory<Program>>
{
    [Fact]
    public async Task Get_ReturnsOk()
    {
        var client = factory.CreateClient();
        var res = await client.GetAsync("/health");
        res.EnsureSuccessStatusCode();
    }
}
```

420. How do you run tests in CI, and how can you make that efficient?

Intermediate | #ci #dotnet-test | reference reference

In continuous integration you run dotnet test as part of the build pipeline, publish the results in TRX or JUnit format for the CI dashboard, and collect coverage for trend analysis. You can speed up large suites with dotnet test --filter to run a subset, or by splitting projects and running them in parallel across agents. Failing the pipeline on any failed test, and treating coverage and test duration as tracked metrics, keeps the suite trustworthy over time.

```
dotnet test --logger "trx;LogFileName=results.trx" \
  --collect:"XPlat Code Coverage"
```

421. What are Testcontainers and when should you use them?

Advanced | #testcontainers #integration-testing #docker | reference

Testcontainers is a library that starts real dependencies such as SQL Server, PostgreSQL, or Redis in disposable Docker containers for the duration of a test or fixture. It lets integration tests run against the actual database engine rather than an in-memory substitute, which catches behavior that providers like the EF Core InMemory provider do not reproduce, such as constraints and SQL translation. Containers add startup cost and require Docker on the build agent, so they are best shared through a collection fixture and used for a focused set of integration tests.

```
        await new MsSqlContainer("mcr.microsoft.com/mssql/server:2022-latest")
            .StartAsync();
```

422. What is mutation testing and what does it add over code coverage?

Advanced | #mutation-testing #stryker #test-quality | reference

Mutation testing tools such as Stryker.NET introduce small deliberate faults, called mutants, into the source and then run the test suite to see whether the tests fail; a surviving mutant means the tests executed that code without verifying its behavior. This directly measures the quality of assertions rather than just which lines were reached, exposing tests that cover code but assert nothing. It is more expensive to run than normal tests, so it is often applied to critical projects or run on a schedule rather than on every commit.

423. How do you write performance tests with BenchmarkDotNet?

Advanced | #benchmarkdotnet #performance | reference

BenchmarkDotNet is the standard .NET library for microbenchmarks; you mark methods with [Benchmark] in a class and run it, and the tool handles warmup, iteration counts, and statistical analysis so the numbers are meaningful. It reports metrics such as mean, median, and allocations, and supports comparing variants with [BenchmarkBaseline] or by parameterizing with [Params]. Benchmarks must always be run in Release configuration outside a debugger, because JIT optimizations and the debugger change results dramatically. For application-level load testing, tools such as k6 or NBomber are more appropriate than microbenchmarks.

```
        [MemoryDiagnoser]
        public class Parsing
        {
            [Benchmark]
            public int Parse() => int.Parse("12345");
        }
```

424. How do code analyzers and coding standards improve code quality?

Advanced | #analyzers #code-style #editorconfig | reference reference

The .NET SDK ships Roslyn analyzers and code style rules that flag potential bugs, API misuse, and style deviations at build time, and their severity can be configured through .editorconfig or the AnalysisMode and EnforceCodeStyleInBuild properties. Enforcing a consistent standard, often captured in an .editorconfig shared across the repo, reduces review noise and lets the compiler catch issues that would otherwise be caught by humans. Treating warnings as errors with TreatWarningsAsErrors in CI ensures the rules are actually honored.

```
<PropertyGroup>
  <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
  <AnalysisMode>Recommended</AnalysisMode>
</PropertyGroup>
```

425. How should you document and version a public API, and what breaks compatibility?

Advanced | #api-versioning #documentation #semver | reference reference

Public APIs should be documented with XML documentation comments so tooling can generate IntelliSense and reference docs, and changes should follow semantic versioning where a major bump signals a breaking change. Binary compatibility means existing compiled callers keep working, which is broken by removing or renaming public members, changing signatures, or making a previously optional member required, while source compatibility can break even when binary compatibility holds. Tools such as the API compatibility analyzer and package validation detect these breaks during a build so they can be caught before release.

```
/// <summary>Adds two integers.</summary>
/// <param name="a">First addend.</param>
public static int Add(int a, int b) => a + b;
```

.NET Platform & Tooling

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

426. What is the Common Language Runtime (CLR)?

Beginner | #clr #runtime | reference reference

The CLR is the virtual machine that executes managed .NET code, providing just-in-time compilation of IL to native code, automatic garbage collection, type safety, exception handling, and threading. It also enforces the security and memory-safety rules that make managed code safer than raw native code. On modern .NET the implementation is CoreCLR, which runs on Windows, Linux, and macOS, whereas the older .NET Framework used a Windows-only CLR.

427. What are the Common Type System (CTS) and the Common Language Specification (CLS)?

Beginner | #cts #cls #interop | reference reference

The CTS defines how types are declared, used, and managed in the runtime, specifying that all types derive from System.Object and distinguishing value types from reference types, along with rules for visibility, inheritance, and interfaces. The CLS is a subset of the CTS that a language must support to interoperate with other .NET languages, so a library marked [assembly: CLSCompliant(true)] avoids types and members that only some languages understand. Following the CLS is important when your library will be consumed from languages other than C#.

```
        [assembly: CLSCompliant(true)]
```

428. What is an assembly, and what is metadata?

Beginner | #assemblies #metadata | reference reference

An assembly is the fundamental unit of deployment, versioning, and reuse in .NET; it is typically a .dll or .exe that contains compiled IL, a manifest describing its identity and dependencies, and metadata. Metadata is the self-describing information about the types and members an assembly defines and references, which the runtime uses for loading, reflection, and cross-assembly calls. Because metadata is embedded, .NET components do not rely on separate type libraries or header files the way classic COM and C++ did.

429. What is strong naming an assembly?

Beginner | #strong-naming #assembly-identity | reference

Strong naming signs an assembly with an RSA key pair, producing a strong name that includes the public key and a public key token, which gives the assembly a globally unique identity and prevents name collisions. It also allows the assembly to be placed in the global assembly cache on .NET Framework and was historically used to detect tampering. Strong names are not a security boundary: on .NET Framework the signature is verified, but .NET Core and .NET 5+ do not validate strong-name signatures at load time, so they do not guarantee trust.

```
        <SignAssembly>true</SignAssembly>
        <AssemblyOriginatorKeyFile>key.snk</AssemblyOriginatorKeyFile>
```

430. What are NuGet packages?

Beginner | #nuget #packages | reference reference

NuGet is the package manager for .NET, and a package is a .nupkg archive containing assemblies, content, and metadata such as dependencies and a version. Modern projects reference packages with a PackageReference item in the project file, and dotnet restore resolves and downloads the transitive dependency graph into a local cache. Publishing your own package is done with dotnet pack, optionally pushing to nuget.org or a private feed.

```
<ItemGroup>
  <PackageReference Include="Serilog" Version="4.0.0" />
</ItemGroup>
```

431. What is an SDK-style project file?

Beginner | #csproj #sdk-style #msbuild | reference

An SDK-style project uses a minimal .csproj that imports a project SDK, typically Microsoft.NET.Sdk, and relies on sensible defaults instead of listing every file. Source files are included implicitly by glob, common properties and targets are provided by the SDK, and dependencies use PackageReference. This format is used by .NET Core and .NET 5+ and can also target .NET Framework, and it replaces the verbose legacy project format.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

432. What are target frameworks and TFMs?

Beginner | #target-framework #tfm | reference

A target framework is the set of APIs a project is compiled against, identified by a Target Framework Moniker such as net8.0 for .NET 8, net9.0 for .NET 9, net48 for .NET Framework 4.8, or netstandard2.0 for the .NET Standard contract. The TargetFramework property in the project file selects it, and the compiler restricts you to APIs that exist in that framework. Choosing a target framework determines which platforms the app or library can run on.

```
<TargetFramework>net8.0</TargetFramework>
```

433. How do .NET Framework, .NET Core, and .NET 5+ relate?

Beginner | #net-framework #net-core #versions | reference reference

.NET Framework is the original Windows-only implementation, with 4.8 as its final major version, and it remains supported for existing Windows applications. .NET Core was a cross-platform, open-source rewrite released from 1.0 to 3.1 that dropped the Framework name in version 5, and .NET 5 and later are simply called .NET and continue the Core lineage with even-numbered releases as LTS. New development should target modern .NET, such as .NET 8 LTS or .NET 9, because it offers better performance, containers, and cross-platform support.

434. What are the most common dotnet CLI commands?

Beginner | #dotnet-cli #tooling | reference

dotnet new scaffolds a project or file from a template, dotnet build compiles it, dotnet run builds and executes it, and dotnet test discovers and runs tests. dotnet publish produces deployment output, dotnet pack creates a NuGet package, and dotnet add package adds a dependency. These commands work identically across platforms and drive the same MSBuild engine the IDEs use.

```
dotnet new console -o MyApp
dotnet build
dotnet test
dotnet publish -c Release -r linux-x64
```

435. What is a solution file and how do you work with it from the CLI?

Beginner | #solution #sln | reference

A solution file groups multiple projects so they can be opened and built together, and the traditional format uses a .sln extension with its own text syntax. You manage it with dotnet sln add, dotnet sln remove, and dotnet sln list, and building the solution builds all contained projects. .NET 9 introduced the simpler XML-based .slnx format, which is more readable and merge-friendly.

```
dotnet new sln -n MyApp
dotnet sln add src/MyApp/MyApp.csproj
```

436. What are IL, JIT, ReadyToRun, and NativeAOT?

Intermediate | #il #jit #native-aot | reference reference

C# compiles to Intermediate Language (IL, also called CIL) stored in the assembly, and the just-in-time compiler translates that IL to native machine code at runtime, method by method, using tiered compilation to optimize hot paths. ReadyToRun precompiles IL to native code ahead of time for faster startup while keeping IL for fallback and tiering, and NativeAOT compiles the entire app to a standalone native executable with no JIT. NativeAOT gives the fastest startup and lowest memory but does not support runtime code generation or unrestricted reflection, so libraries relying on those features are incompatible.

```
dotnet publish -c Release -r linux-x64 \
  -p:PublishAot=true
```

437. What is multi-targeting and when do you use it?

Intermediate | #multi-targeting #conditional-compilation | reference

Multi-targeting compiles a single project for several target frameworks by listing them in the plural TargetFrameworks property, producing an output folder per framework. You then use conditional compilation symbols such as NET8_0_OR_GREATER or MSBuild conditions to include code or package references only where they are supported. It is common for libraries that must support both modern .NET and .NET Standard or .NET Framework consumers.

```
<TargetFrameworks>net8.0;netstandard2.0</TargetFrameworks>

#if NET8_0_OR_GREATER
    // modern API
#endif
```

438. What is .NET Standard and is it still recommended?

Intermediate | #net-standard #library-targeting | reference

.NET Standard is a formal specification of the APIs that every .NET implementation must provide, letting one library run on .NET Framework, .NET Core, and other implementations; netstandard2.0 has the broadest reach and netstandard2.1 adds newer APIs but is not supported by .NET Framework. Since .NET 5 unified the platform, Microsoft recommends new libraries target a concrete framework such as net8.0 for access to the full API surface and performance improvements. .NET Standard remains useful only when a library must also support .NET Framework or older runtimes.

439. What is MSBuild and how does it relate to the dotnet CLI?

Intermediate | #msbuild #build | reference

MSBuild is the XML-based build engine for .NET, and every SDK-style project file is an MSBuild file made up of properties, items, and targets. The dotnet CLI commands such as build and publish are thin wrappers that invoke MSBuild, which is why they honor the same project settings. You can customize builds by adding targets and properties in the project file or in imported files like Directory.Build.props, and run MSBuild directly with dotnet msbuild when needed.

```
dotnet msbuild -t:Build -p:Configuration=Release
```

440. What is global.json and why use it?

Intermediate | #global-json #sdk-versioning | reference

global.json is an optional file placed in a project directory or higher that pins which .NET SDK version the CLI should use, ensuring a consistent toolchain across a team and in CI. It can specify an exact version and a rollForward policy that controls whether a newer patch, feature band, or major version may satisfy the request. Without it, dotnet commands use the highest installed SDK, which can cause surprising build differences between machines.

```
{
  "sdk": {
    "version": "8.0.400",
    "rollForward": "latestFeature"
  }
}
```

441. What is Directory.Build.props?

Intermediate | #directory-build-props #msbuild | reference

Directory.Build.props is an MSBuild file that the build engine automatically imports for every project in its directory and all subdirectories, letting you define shared properties such as LangVersion, Nullable, TargetFramework, or common package references in one place. This avoids duplicating the same settings across many project files and keeps them consistent. A companion file, Directory.Build.targets, is imported later and is used for shared targets and items that need to run after the project is evaluated.

```
<Project>
  <PropertyGroup>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

442. What are runtime identifiers (RIDs)?

Intermediate | #rid #publishing | reference

A runtime identifier names a specific target platform, such as win-x64, linux-arm64, or osx-arm64, and is defined by the RuntimeIdentifier property or the -r command-line option. RIDs are required for self-contained publishing, NativeAOT, and any scenario where RID-specific native assets must be selected. The .NET RID catalog lists the supported values, and portable RIDs are being replaced by the graph-based RID system to simplify cross-distribution targeting.

```
dotnet publish -c Release -r win-x64 --self-contained
```

443. What is single-file publishing?

Intermediate | #single-file #publishing | reference

Single-file publishing bundles the application and its dependencies into one executable by setting PublishSingleFile, so end users see a single file to deploy. Managed assemblies are loaded directly from the bundle, and native libraries can be embedded with IncludeNativeLibrariesForSelfExtract or left beside the executable. It is often combined with self-contained publishing and trimming to reduce the size, and on .NET 5 and later it works together with ReadyToRun and NativeAOT options.

```
dotnet publish -c Release -r linux-x64 --self-contained \
  -p:PublishSingleFile=true
```

444. What is trimming and what are its risks?

Intermediate | #trimming #reflection #publishing | reference

Trimming analyzes the application's static references and removes unused code from the app and its dependencies, reducing the published size, and it is enabled with PublishTrimmed. The main risk is that code reached only through reflection or late binding can be removed or broken, so the SDK emits trim warnings and you can preserve members with attributes such as DynamicallyAccessedMembers or with a trimmer root descriptor. TrimMode controls how aggressive trimming is, and trimming works with self-contained and NativeAOT publishes.

```
dotnet publish -c Release -r linux-x64 --self-contained \
  -p:PublishTrimmed=true
```

445. What is the difference between self-contained and framework-dependent deployment?

Intermediate | #deployment #self-contained #framework-dependent | reference

A framework-dependent deployment contains only your application and requires the matching .NET runtime to be installed on the target machine, producing a small output. A self-contained deployment includes the runtime and libraries with the app, so it runs without a separate runtime install but is much larger and requires a specific runtime identifier. Framework-dependent is the default and usually preferred for servers and containers where the runtime is provided, while self-contained suits scenarios where you cannot control the host environment.

```
dotnet publish -c Release -r linux-x64 --self-contained false
# vs
# --self-contained true
```

446. How do you containerize a .NET application with Docker?

Advanced | #docker #containers #publishing | reference reference

The standard approach is a multi-stage Dockerfile: build and publish with the larger SDK image, then copy only the published output into a smaller runtime image such as mcr.microsoft.com/dotnet/aspnet for web apps. Since .NET 8 you can skip the Dockerfile entirely with dotnet publish /t:PublishContainer, which builds an image from project properties and supports chiseled or distroless base images and a non-root user. For NativeAOT or self-contained apps, a runtime-deps image is enough because the runtime is already inside the output.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

447. How does the built-in dependency injection container work?

Advanced | #dependency-injection #service-lifetimes | reference reference

Microsoft.Extensions.DependencyInjection provides an IServiceCollection where services are registered with a lifetime: AddTransient creates a new instance per resolution, AddScoped creates one per scope, and AddSingleton creates one for the application's lifetime. You resolve services through an IServiceProvider, and ASP.NET Core creates a scope per HTTP request so scoped services are shared within that request. Constructor injection is the preferred pattern, and you must avoid captive dependencies, where a longer-lived service holds a shorter-lived one, because that keeps the shorter-lived instance alive and can break scoped semantics.

```
var services = new ServiceCollection();
services.AddSingleton<IClock, SystemClock>();
services.AddScoped<IOrderRepository, OrderRepository>();
services.AddTransient<OrderService>();

using var provider = services.BuildServiceProvider();
using var scope = provider.CreateScope();
var svc = scope.ServiceProvider.GetRequiredService<OrderService>();
```

448. How do configuration and the options pattern work in .NET?

Advanced | #configuration #options-pattern #appsettings | reference reference

IConfiguration is built from one or more providers, such as appsettings.json, appsettings.{Environment}.json, environment variables, user secrets in development, and command-line arguments, with later providers overriding earlier ones. Configuration keys form a hierarchy, and environment variables use a double underscore such as ConnectionStrings__Default to represent nesting. The options pattern binds a configuration section to a strongly typed class with services.Configure, which you then consume as IOptions for constant values, IOptionsSnapshot for per-request reloads, or IOptionsMonitor for change notifications, and you can validate it with data annotations or ValidateOnStart.

```
builder.Services.Configure<SmtpOptions>(
    builder.Configuration.GetSection("Smtp"));

public class Mailer(IOptions<SmtpOptions> options)
{
    private readonly SmtpOptions _o = options.Value;
}
```

449. What are the host builder, the minimal hosting model, and background services?

Advanced | #generic-host #minimal-hosting #background-service | reference reference

The generic host, built with Host.CreateDefaultBuilder, wires up configuration, dependency injection, logging, and application lifetime, and ASP.NET Core 6 and later use the minimal hosting model where WebApplication.CreateBuilder combines host and web setup in a few statements instead of a separate Startup class. IHostedService defines code that runs when the host starts and stops, and BackgroundService is a convenient base class for long-running work that loops until cancellation is requested. Background services are registered with AddHostedService and should honor the provided CancellationToken so they shut down cleanly.

```
public class Worker(ILogger<Worker> log) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            log.LogInformation("tick");
            await Task.Delay(1000, ct);
        }
    }
}

builder.Services.AddHostedService<Worker>();
```

450. How do logging abstractions, GC configuration, globalization mode, and cross-platform concerns affect a .NET app?

Advanced | #logging #garbage-collection #globalization | reference reference

Microsoft.Extensions.Logging provides ILogger and ILoggerFactory so code depends on the abstraction rather than a specific library, with pluggable providers such as console, debug, and OpenTelemetry and filtering by log level. The garbage collector can run in workstation mode, tuned for low latency on client machines, or server mode, which uses a heap per core for throughput and is the default for ASP.NET Core; you configure it with the ServerGarbageCollection and ConcurrentGarbageCollection properties or runtime settings. Setting InvariantGlobalization to true removes the ICU dependency and shrinks containers, but all culture-aware operations behave as invariant, which is unsuitable when you need real localization. For cross-platform code, avoid Windows-only APIs, use Path.Combine and Environment.NewLine instead of hard-coded separators, and account for case-sensitive file systems and differing line endings.

```
        <PropertyGroup>
          <ServerGarbageCollection>true</ServerGarbageCollection>
          <InvariantGlobalization>true</InvariantGlobalization>
        </PropertyGroup>
```

ASP.NET Core & Web

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

451. What is the difference between ASP.NET Core and ASP.NET Framework?

Beginner | #aspnet-core #aspnet-framework #cross-platform | reference reference

ASP.NET Core is a cross-platform, open-source framework that runs on modern .NET on Windows, Linux, and macOS, whereas ASP.NET Framework only runs on Windows on the full .NET Framework. ASP.NET Core unifies MVC and Web API into one programming model, has built-in dependency injection, is side-by-side versionable, and is far faster and more modular. ASP.NET Framework is in maintenance mode and receives only security and reliability fixes, so new projects should target ASP.NET Core.

452. What is the minimal hosting model and what does WebApplication.CreateBuilder do?

Beginner | #minimal-hosting #webapplication #program-cs | reference reference

The minimal hosting model, introduced in .NET 6, replaces Startup.cs with a single top-level Program.cs that configures services and the request pipeline in one place. WebApplication.CreateBuilder(args) creates a builder preconfigured with sensible defaults such as Kestrel, configuration providers, logging, and the DI container. You register services on builder.Services, call builder.Build() to obtain a WebApplication, add middleware, map endpoints, and call app.Run(). The older Startup class with ConfigureServices and Configure is still supported for backwards compatibility.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
var app = builder.Build();
app.MapControllers();
app.Run();
```

453. What are the three service lifetimes in the ASP.NET Core dependency injection container?

Beginner | #dependency-injection #lifetimes #services | reference

Transient services are created every time they are requested, scoped services are created once per client request (per scope), and singleton services are created once for the lifetime of the application. They are registered with AddTransient, AddScoped, and AddSingleton respectively. Choosing the wrong lifetime causes bugs such as a singleton capturing a scoped service, which effectively promotes that scoped instance to a singleton, so always match the lifetime to the state the service holds.

```
builder.Services.AddSingleton<IClock, SystemClock>();
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddTransient<IEmailSender, SmtpEmailSender>();
```

454. When would you choose controllers over minimal APIs?

Beginner | #minimal-apis #controllers #web-api | reference reference

Minimal APIs define endpoints as lambda expressions mapped directly on the WebApplication and are ideal for small services, microservices, and lightweight JSON APIs with little ceremony. Controllers are classes that group related actions and support features such as filters, conventions, content negotiation, and rich model binding, which suits large applications with many endpoints and cross-cutting concerns. Both build on the same routing system and can be mixed in a single application.

455. What is the difference between attribute routing and conventional routing?

Beginner | #routing #attribute-routing #conventional-routing | reference reference

Conventional routing defines route templates centrally, typically with app.MapControllerRoute, and matches incoming URLs to controller actions by pattern. Attribute routing places templates directly on actions with attributes such as [Route], [HttpGet], and [HttpPost], giving precise per-endpoint control and HTTP verb mapping. Attribute routing is the default and recommended approach for Web APIs, while conventional routing is common for MVC applications with many similar controller/action pairs.

456. How does model binding and DataAnnotations validation work in ASP.NET Core?

Beginner | #model-binding #validation #dataannotations | reference reference

Model binding maps request data from route values, query string, form fields, and JSON bodies to action parameters or complex types, using attributes such as [FromRoute], [FromQuery], [FromBody], and [FromForm] to specify sources. You annotate model properties with DataAnnotations attributes like [Required], [StringLength], [Range], and [RegularExpression], and ASP.NET Core validates them during binding. With the [ApiController] attribute, an invalid model automatically produces a 400 response with a ProblemDetails body containing the errors exposed through ModelState.

```
public class CreateUser
{
    [Required, StringLength(50)]
    public string Name { get; set; } = "";
    [Range(18, 120)]
    public int Age { get; set; }
}
```

457. What is the difference between IActionResult and ActionResult?

Beginner | #action-results #web-api #swagger | reference

IActionResult is an interface implemented by every action result such as OkObjectResult, NotFoundResult, and FileResult, and it is used when an action can produce several different result types. ActionResult is a generic type that lets an action return either a value of type T or an IActionResult, giving OpenAPI/Swagger a concrete success response type while still allowing status codes. Returning ActionResult therefore improves API metadata and keeps action code concise.

```
public ActionResult<Product> Get(int id)
{
    var product = _repo.Find(id);
    if (product is null)
        return NotFound();
    return product;
}
```

458. How does configuration and environment handling work in ASP.NET Core?

Beginner | #configuration #environments #logging | reference reference

Configuration is composed from ordered providers including appsettings.json, appsettings.{Environment}.json, user secrets, environment variables, and command-line arguments, where later providers override earlier ones. The environment name comes from ASPNETCORE_ENVIRONMENT and defaults to Production, and Development enables the developer exception page and more verbose logging. You can branch on app.Environment.IsDevelopment() or inject IWebHostEnvironment to change behavior per environment, and logging is configured through the same builder.

459. What is the difference between authentication and authorization?

Beginner | #authentication #authorization #security | reference reference

Authentication establishes who the user is by validating credentials such as a cookie or a JWT and producing a ClaimsPrincipal on HttpContext.User. Authorization then decides whether that authenticated identity is permitted to perform an action, enforced with [Authorize], roles, or policies. Authentication must run before authorization in the pipeline via UseAuthentication and UseAuthorization, and an unauthenticated request results in a 401 while an unauthorized one results in a 403.

460. How do you serve static files in ASP.NET Core?

Beginner | #static-files #wwwroot #middleware | reference

Add app.UseStaticFiles() to the pipeline to serve files from the wwwroot folder, placing it early so files are handled before the routing middleware. Static files are served without authorization by default, so files that must be protected should not live in wwwroot. UseStaticFiles can take a StaticFileOptions instance to configure content types, caching headers, or a custom file provider, and UseDefaultFiles plus UseDirectoryBrowser add default-document and directory listing support.

461. How does the middleware pipeline work and why does ordering matter?

Intermediate | #middleware #pipeline #ordering | reference reference

Middleware components are assembled into a pipeline in the order they are added in Program.cs, and each component can run logic before and after calling next to pass control onward. Ordering matters because components such as UseExceptionHandler, UseHttpsRedirection, UseCors, UseAuthentication, and UseAuthorization must appear before the endpoints they protect. A component that does not call next short-circuits the pipeline, which is how terminal middleware such as UseStaticFiles responds without invoking later components.

```
        app.UseExceptionHandler();
        app.UseHttpsRedirection();
        app.UseCors();
        app.UseAuthentication();
        app.UseAuthorization();
        app.MapControllers();
```

462. What types of filters exist in ASP.NET Core MVC and what is each used for?

Intermediate | #filters #mvc #cross-cutting | reference

MVC filters run at different stages of request processing: authorization filters run first, resource filters run around model binding, action filters run before and after the action, exception filters handle unhandled exceptions, and result filters run around result execution. They can be registered globally, per controller, or per action, and they support dependency injection through the container. Filters are useful for cross-cutting concerns such as logging, validation, and caching that need MVC-specific context.

463. What is the difference between an action filter and middleware?

Intermediate | #action-filters #middleware #mvc | reference reference

Middleware is part of the low-level request pipeline, runs for every request, and has no knowledge of MVC concepts such as controllers, actions, or model state. Action filters run only around MVC and Razor Pages action execution and have access to ActionExecutingContext and ActionExecutedContext, making them ideal for concerns that need action metadata. Use middleware for transport-level concerns like exception handling and CORS, and use filters for MVC-specific cross-cutting logic.

464. What is the options pattern and how do IOptions, IOptionsSnapshot, and IOptionsMonitor differ?

Intermediate | #options-pattern #configuration #di | reference

The options pattern binds a configuration section to a strongly typed class with services.Configure(config.GetSection(...)), avoiding scattered IConfiguration usage. IOptions is a singleton that never changes after startup, IOptionsSnapshot is scoped and recomputed per request which suits reloadable configuration, and IOptionsMonitor is a singleton that exposes CurrentValue and raises change notifications. Choose the interface based on whether the values must react to configuration reloads at runtime.

```
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));

public class TokenService(IOptionsSnapshot<JwtOptions> options)
{
    private readonly JwtOptions _opts = options.Value;
}
```

465. How do you configure JWT bearer authentication in ASP.NET Core?

Intermediate | #jwt #bearer #authentication | reference

You call AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...) and set TokenValidationParameters for the issuer, audience, signing key, and lifetime validation, then add app.UseAuthentication() and app.UseAuthorization(). The handler validates the token from the Authorization: Bearer header and populates HttpContext.User with its claims. In production, prefer validating signatures against the identity provider's published metadata and keys rather than a hardcoded symmetric secret.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidIssuer = config["Jwt:Issuer"],
            IssuerSigningKey = new SymmetricSecurityKey(key)
        };
    });
```

466. What is policy-based authorization and how do you create a custom policy?

Intermediate | #authorization #policies #claims | reference

Policy-based authorization decouples authorization rules from roles by registering named policies that are built from requirements and evaluated by handlers. You register a policy with services.AddAuthorization(options => options.AddPolicy("AtLeast21", p => p.Requirements.Add(new MinimumAgeRequirement(21)))), implement an AuthorizationHandler for the requirement, and apply the policy with [Authorize(Policy = "AtLeast21")]. Policies can combine multiple requirements and use claims or custom logic, making them more flexible than legacy role checks.

```
builder.Services.AddAuthorization(o =>
    o.AddPolicy("AtLeast21", p => p.Requirements.Add(new MinimumAgeRequirement(21))));

[Authorize(Policy = "AtLeast21")]
public IActionResult Buy() => Ok();
```

467. How does CORS work in ASP.NET Core and how is it configured?

Intermediate | #cors #security #middleware | reference

Browsers enforce the same-origin policy, and CORS is a server-side opt-in that tells browsers which cross-origin requests are allowed through response headers. You register policies with AddCors and enable them with app.UseCors, and the middleware must run after UseRouting and before UseAuthorization. Configure origins, methods, and headers explicitly, and remember that AllowCredentials cannot be combined with AllowAnyOrigin for security reasons.

```
builder.Services.AddCors(o => o.AddPolicy("web", p => p
    .WithOrigins("https://app.example.com")
    .AllowAnyHeader()
    .AllowAnyMethod()));

app.UseCors("web");
```

468. How do you handle exceptions globally in ASP.NET Core?

Intermediate | #exception-handling #problem-details #error-handling | reference reference

Use app.UseExceptionHandler to catch unhandled exceptions and produce a consistent response, and app.UseDeveloperExceptionPage only in development. In .NET 8 the recommended approach is AddProblemDetails combined with UseExceptionHandler, which writes an RFC 7807 ProblemDetails response and can include a trace id for correlation. For custom logic you can implement IExceptionHandler and register it, but you should never leak stack traces or internal details in production responses.

```
builder.Services.AddProblemDetails();
app.UseExceptionHandler();

// In development only
if (app.Environment.IsDevelopment())
    app.UseDeveloperExceptionPage();
```

469. How does FluentValidation integrate with ASP.NET Core?

Intermediate | #fluentvalidation #validation #web-api | reference

FluentValidation defines validation rules in separate validator classes deriving from AbstractValidator, keeping validation logic out of the model and supporting complex, conditional, and reusable rules. You register validators with AddValidatorsFromAssembly and enable automatic integration with AddFluentValidationAutoValidation, which runs them as part of the MVC model validation pipeline. Failed rules populate ModelState, so [ApiController] endpoints still return the standard 400 ProblemDetails response.

```
public class CreateUserValidator : AbstractValidator<CreateUser>
{
    public CreateUserValidator()
    {
        RuleFor(x => x.Name).NotEmpty().MaximumLength(50);
        RuleFor(x => x.Age).InclusiveBetween(18, 120);
    }
}
```

470. What are best practices for designing and versioning REST APIs, and how do you document them?

Intermediate | #rest #versioning #openapi | reference reference

Design around resources and HTTP semantics: use nouns in URIs, the correct verbs and status codes, idempotent PUT/DELETE, and consistent error payloads such as ProblemDetails. Version APIs explicitly (for example via URL segment, header, or query string) using the Microsoft.AspNetCore.Mvc.Versioning package, and avoid breaking changes within a version. Generate OpenAPI documents with Microsoft.AspNetCore.OpenApi or Swashbuckle and expose interactive docs such as Swagger UI in non-production environments.

471. How does the built-in rate limiting middleware work in ASP.NET Core?

Advanced | #rate-limiting #performance #middleware | reference

AddRateLimiter registers named policies using a fixed window, sliding window, token bucket, or concurrency limiter, and app.UseRateLimiter enables them in the pipeline. You apply a policy globally, per controller with [EnableRateLimiting], or per endpoint with RequireRateLimiting, and you can partition limits by API key, user, or IP address using RateLimitPartition. Rejected requests return 503 by default but the rejection response can be customized, commonly to 429 with a Retry-After header.

```
builder.Services.AddRateLimiter(o => o.AddFixedWindowLimiter("fixed", opt =>
{
    opt.PermitLimit = 100;
    opt.Window = TimeSpan.FromMinutes(1);
}));

app.UseRateLimiter();
app.MapControllers().RequireRateLimiting("fixed");
```

472. How does output caching differ from response caching, and where does response compression fit?

Advanced | #output-caching #response-compression #performance | reference reference

Output caching, added in .NET 7, stores entire responses server-side and is configured with AddOutputCache and app.UseOutputCache, then applied with .CacheOutput or a named policy on endpoints. Unlike response caching it does not depend on client or proxy cache headers and lets you invalidate entries by tag with IOutputCacheStore.EvictByTagAsync. Response compression is complementary: AddResponseCompression plus UseResponseCompression compresses payloads such as gzip or Brotli to reduce bandwidth regardless of caching.

```
builder.Services.AddOutputCache();
builder.Services.AddResponseCompression(o => o.EnableForHttps = true);
app.UseResponseCompression();
app.UseOutputCache();
app.MapGet("/products", GetProducts).CacheOutput(p => p.Expire(TimeSpan.FromMinutes(1)));
```

473. How do you implement health checks in ASP.NET Core?

Advanced | #health-checks #monitoring #hosting | reference

AddHealthChecks registers health checks and MapHealthChecks exposes an endpoint, commonly /health, that reports Healthy, Degraded, or Unhealthy. You add checks for databases, brokers, and external services with packages such as AspNetCore.HealthChecks.SqlServer, and you can tag and filter checks so Kubernetes liveness probes differ from readiness probes. Health endpoints should be restricted or placed on an internal port so they do not leak sensitive dependency details.

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>("db", tags: new[] { "ready" });

app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = r => r.Tags.Contains("ready")
});
```

474. How do you run background work and manage outbound HTTP calls in ASP.NET Core?

Advanced | #background-services #httpclientfactory #hosted-services | reference reference

Implement IHostedService or derive from BackgroundService and register it with AddHostedService; the generic host starts it on startup and signals cancellation on shutdown. In long-running loops you should honor the injected CancellationToken, and you must never resolve scoped services directly from a singleton hosted service. Use IServiceScopeFactory to create a scope per unit of work, and use IHttpClientFactory (AddHttpClient) to obtain HttpClient instances so sockets are pooled and DNS changes are respected instead of newing up clients.

```
public class Worker(IServiceScopeFactory scopeFactory) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        using var scope = scopeFactory.CreateScope();
        var repo = scope.ServiceProvider.GetRequiredService<IJobRepository>();
        // process work, honoring ct
    }
}
```

475. When would you use SignalR versus gRPC for real-time or service-to-service communication?

Advanced | #signalr #grpc #real-time | reference reference

SignalR provides an abstraction over WebSockets with automatic fallback to server-sent events or long polling and supports server-to-client push, groups, and reconnection, making it well suited to browser clients and real-time UI scenarios. gRPC is a contract-first, HTTP/2-based RPC framework using Protocol Buffers that offers high performance, streaming, and strong typing, ideal for service-to-service calls and polyglot microservices. gRPC-Web is needed for browser clients, whereas SignalR already works natively in browsers, so choose based on client type, latency, and whether you need request/response RPC or push messaging.

Entity Framework & Data Access

25 questions · 10 Beginner · 10 Intermediate · 5 Advanced

476. How does EF Core compare with ADO.NET and Dapper?

Beginner | #ef-core #dapper #ado-net | reference reference

ADO.NET is the low-level data access API where you write SQL and manage connections, commands, and readers manually, giving maximum control and minimum overhead. Dapper is a micro-ORM that maps query results to objects while still requiring you to write SQL, offering a good balance of performance and convenience. EF Core is a full ORM with change tracking, LINQ translation, migrations, and relationship management, trading some raw performance for productivity, so it suits complex domain models while Dapper or ADO.NET fit high-throughput or hand-tuned SQL scenarios.

477. What is the correct lifetime and scoping for a DbContext?

Beginner | #dbcontext #lifetime #di | reference

A DbContext is not thread-safe and is designed to be short-lived, typically one instance per request, which is why it is registered with AddDbContext using the scoped lifetime. Sharing a context across concurrent threads or holding one for the whole application lifetime leads to change-tracking memory growth and concurrency bugs. In ASP.NET Core the scoped registration matches the request scope, and in background work you create a scope per unit of work with IServiceScopeFactory.

```
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(connectionString));
```

478. What is a DbSet and how does it relate to the model?

Beginner | #dbset #dbcontext #entities | reference

A DbSet represents a queryable collection of entities of type T and acts as the entry point for querying and saving those entities. It is exposed as a property on your DbContext, and LINQ queries against it are translated into SQL. Each entity in a DbSet has an associated entity state tracked by the change tracker, and DbSet also exposes methods like Add, Find, and Remove that operate on the set.

479. How does LINQ to Entities translation work and what can prevent it?

Beginner | #linq #translation #querying | reference

EF Core translates LINQ query expressions into provider-specific SQL by walking the expression tree, so only operations the provider understands can be executed in the database. If a query uses a method or construct that cannot be translated, EF Core throws an InvalidOperationException at translation time rather than silently fetching all rows, except for the final projection which can be evaluated client-side. You keep queries translatable by using supported methods, avoiding custom delegates in Where clauses, and projecting to DTOs with Select.

```
var names = await db.Products
    .Where(p => p.Price > 10)
    .Select(p => p.Name)
    .ToListAsync();
```

480. What is the difference between tracking and no-tracking queries, and what does AsNoTracking do?

Beginner | #asnotracking #change-tracking #performance | reference

By default a query returns tracked entities, meaning the change tracker keeps a reference and snapshots so it can detect modifications and generate updates on SaveChanges. AsNoTracking tells EF Core to skip this bookkeeping, which reduces memory use and speeds up read-only queries, especially for large result sets. No-tracking entities are not updated by SaveChanges, and you can make a context default to no-tracking with QueryTrackingBehavior.NoTracking and opt back in with AsTracking.

```
var products = await db.Products
    .AsNoTracking()
    .Where(p => p.Active)
    .ToListAsync();
```

481. How do you load related data with eager, explicit, and lazy loading?

Beginner | #include #eager-loading #lazy-loading | reference

Eager loading uses Include and ThenInclude in the query to fetch related entities in the same round trip and is the most predictable option. Explicit loading keeps the entity untracked-relationship-free and loads navigation properties on demand with Entry(...).Collection(...).LoadAsync() or .Reference(...).LoadAsync(). Lazy loading requires the Microsoft.EntityFrameworkCore.Proxies package and virtual navigation properties, loading relationships transparently on first access but risking the N+1 problem.

```
var order = await db.Orders
    .Include(o => o.Lines)
        .ThenInclude(l => l.Product)
    .FirstAsync(o => o.Id == id);
```

482. How do migrations work and what are the common commands?

Beginner | #migrations #tooling #schema | reference

Migrations are a code-based record of how the database schema evolves with your model, generated by the dotnet-ef tooling and stored as C# classes with Up and Down methods. You create one with dotnet ef migrations add , apply it with dotnet ef database update, and revert to a previous migration by passing its name to database update or removing the last unapplied one with migrations remove. You can generate a SQL script with dotnet ef migrations script for controlled deployment.

```
dotnet ef migrations add AddOrdersTable
dotnet ef database update
dotnet ef migrations remove
```

483. What is the difference between Fluent API and DataAnnotations for model configuration?

Beginner | #fluent-api #dataannotations #model-configuration | reference

DataAnnotations configure the model with attributes on entity properties, which is concise but limited and mixes persistence concerns into the domain classes. The Fluent API configures the model imperatively in OnModelCreating, supports everything including relationships, keys, indexes, and shadow properties, and keeps entities clean. Most teams use Fluent API for relationships and constraints and DataAnnotations for simple validation, with Fluent API taking precedence when both are present.

```
protected override void OnModelCreating(ModelBuilder b)
{
    b.Entity<Order>()
        .HasKey(o => o.Id);
    b.Entity<Order>()
        .Property(o => o.Total)
        .HasPrecision(18, 2);
}
```

484. How does SaveChanges and change tracking work?

Beginner | #savechanges #change-tracking #persistence | reference

When you query tracked entities, the change tracker stores a snapshot, and on SaveChanges it compares current values to detect modifications and marks each entity as Added, Modified, Deleted, or Unchanged. SaveChanges generates the corresponding INSERT, UPDATE, and DELETE statements and wraps them in a transaction by default so the changes commit atomically. Detached or no-tracking entities are ignored, so you must attach them and set their state if you want them persisted.

```
var p = await db.Products.FindAsync(id);
p!.Price = 9.99m;          // Modified
await db.SaveChangesAsync(); // generates UPDATE
```

485. What is the difference between client and server evaluation in EF Core queries?

Beginner | #client-evaluation #translation #querying | reference

Server evaluation means the part of the query the provider understands is translated to SQL and executed in the database, which is efficient and reduces data transfer. Client evaluation happens when an expression cannot be translated; in EF Core 3.0 and later the middle of a query throws instead of silently evaluating on the client, and only the final Select projection may run in memory. To use untranslatable logic you should call AsEnumerable or AsAsyncEnumerable after a translatable portion to explicitly switch to client evaluation.

```
var recent = await db.Orders
    .Where(o => o.CreatedAt > cutoff)   // server
    .AsEnumerable()                     // switch to client
    .Select(o => MyLocalFormat(o));
```

486. What is the N+1 query problem and how do you avoid it in EF Core?

Intermediate | #n+1 #performance #include | reference

The N+1 problem occurs when code loads a list of N entities and then triggers one additional query per entity to fetch a related collection, producing N+1 round trips. It is most common with lazy loading or when a navigation property is accessed in a loop. Avoid it with eager loading via Include, projecting the needed data with Select, or using split or batched queries, and verify by logging SQL or inspecting the query count.

```
// Bad: lazy loading triggers a query per customer
foreach (var c in customers)
    Console.WriteLine(c.Orders.Count);

// Good: single query
var list = await db.Customers.Include(c => c.Orders).ToListAsync();
```

487. What are split queries and when should you use AsSplitQuery?

Intermediate | #split-queries #performance #include | reference

By default EF Core translates Include chains into a single SQL query with JOINs, which can produce a cartesian explosion when several collections are included, inflating the result set. AsSplitQuery emits one query per included collection, avoiding duplication at the cost of multiple round trips and a potential inconsistency if data changes between queries. Use split queries when you include multiple collections or load large related sets, and keep single queries when the result is small or consistency matters most.

```
var blogs = await db.Blogs
    .Include(b => b.Posts)
    .Include(b => b.Contributors)
    .AsSplitQuery()
    .ToListAsync();
```

488. How do you execute raw SQL safely and avoid SQL injection?

Intermediate | #raw-sql #sql-injection #security | reference

FromSqlRaw and ExecuteSqlRaw interpolate parameters you supply, and you must pass user input as parameters rather than concatenating it into the SQL string. FromSqlInterpolated and ExecuteSqlInterpolated take interpolated strings and turn the holes into parameters automatically, making them the safer choice, and ExecuteUpdate/ExecuteDelete cover most bulk scenarios without raw SQL. Always parameterize values and never build SQL by string concatenation from untrusted input.

```
// Safe: parameterized
var name = userInput;
var users = await db.Users
    .FromSqlInterpolated($"SELECT * FROM Users WHERE Name = {name}")
    .ToListAsync();
```

489. How do you configure relationships in EF Core?

Intermediate | #relationships #fluent-api #foreign-key | reference

EF Core infers relationships from navigation properties and foreign keys by convention, and you refine them with the Fluent API. HasOne/WithMany configures one-to-many, HasOne/WithOne configures one-to-one, and many-to-many is configured with HasMany/WithMany using an implicit or explicit join entity. You can specify the foreign key with HasForeignKey and configure delete behavior with OnDelete.

```
        b.Entity<Order>()
            .HasOne(o => o.Customer)
            .WithMany(c => c.Orders)
            .HasForeignKey(o => o.CustomerId)
            .OnDelete(DeleteBehavior.Restrict);
```

490. How does cascade delete behave in EF Core and how do you control it?

Intermediate | #cascade-delete #relationships #delete-behavior | reference

Cascade delete removes dependent rows automatically when a principal is deleted, and EF Core configures it by convention for required relationships. You control it per relationship with OnDelete(DeleteBehavior.Cascade | Restrict | SetNull | NoAction | ClientSetNull). Restrict prevents deletion of a principal that still has dependents, and SetNull nulls the foreign key, which requires the key to be nullable. Note that database-level cascades and EF's in-memory cascade can differ, so the behavior should be configured deliberately.

```
        b.Entity<OrderLine>()
            .HasOne(l => l.Order)
            .WithMany(o => o.Lines)
            .OnDelete(DeleteBehavior.Cascade);
```

491. How do you handle optimistic concurrency with a RowVersion token?

Intermediate | #concurrency #rowversion #optimistic-locking | reference

Optimistic concurrency assumes conflicts are rare and detects them at save time by comparing a concurrency token. You add a byte[] property marked [Timestamp] (or configured with IsRowVersion), and EF Core includes it in the WHERE clause of UPDATE and DELETE statements. If no row matches, EF Core throws DbUpdateConcurrencyException, which you catch to reload the current values and resolve the conflict, typically by retrying or informing the user.

```
        public class Product
        {
            public int Id { get; set; }
            [Timestamp] public byte[] RowVersion { get; set; } = default!;
        }

        try { await db.SaveChangesAsync(); }
        catch (DbUpdateConcurrencyException ex) { /* resolve */ }
```

492. How do transactions work in EF Core?

Intermediate | #transactions #atomicity #savechanges | reference

SaveChanges wraps all pending changes in a single transaction by default, so either every statement commits or none does. To span multiple SaveChanges calls or raw SQL commands you start an explicit transaction with Database.BeginTransactionAsync and commit or roll back it yourself. For distributed scenarios you can enlist in an ambient transaction, and you can also configure execution strategies so retries interact correctly with transactions.

```
await using var tx = await db.Database.BeginTransactionAsync();
try
{
    db.Orders.Add(order);
    await db.SaveChangesAsync();
    await tx.CommitAsync();
}
catch
{
    await tx.RollbackAsync();
    throw;
}
```

493. How do global query filters work and when are they useful?

Intermediate | #query-filters #soft-delete #multi-tenancy | reference

Global query filters apply a predicate to every query for an entity type, defined with HasQueryFilter in OnModelCreating. They are commonly used for soft deletes, multi-tenancy, and excluding inactive rows, and they are automatically applied to LINQ queries including those reached through Include. You can bypass a filter for a specific query with IgnoreQueryFilters, but note that filters are not applied to required navigation properties loaded implicitly or to raw SQL.

```
b.Entity<Post>().HasQueryFilter(p => !p.IsDeleted);

// bypass when needed
var all = await db.Posts.IgnoreQueryFilters().ToListAsync();
```

494. What are compiled queries and when do they help?

Intermediate | #compiled-queries #performance #querying | reference

EF Core compiles the LINQ expression tree into SQL once and caches it per query shape, so repeated executions of the same shape reuse that work. EFQueryCompiledQueryCache reduces the overhead of the first translation, and EF.CompileAsyncQuery lets you precompile a query delegate that you invoke with the context and parameters. Compiled queries mainly help hot paths executed many times with different parameters, since the gain is small compared to actual database round-trip cost.

```
private static readonly Func<AppDbContext, int, Task<Product?>> _byId =
    EF.CompileAsyncQuery((AppDbContext db, int id) =>
        db.Products.FirstOrDefault(p => p.Id == id));

var product = await _byId(db, 42);
```

495. How do ExecuteUpdate and ExecuteDelete differ from the traditional approach?

Intermediate | #bulk-operations #executeupdate #performance | reference

ExecuteUpdate and ExecuteDelete, introduced in EF Core 7, translate directly to UPDATE and DELETE statements and run immediately without loading entities into memory. This is far more efficient than fetching entities, modifying them, and calling SaveChanges, which requires tracking and multiple round trips. They bypass the change tracker, so tracked entities in memory become stale, and they cannot be combined with the normal SaveChanges batching or entity events.

```
await db.Products
    .Where(p => p.Discontinued)
    .ExecuteDeleteAsync();

await db.Products
    .Where(p => p.CategoryId == 3)
    .ExecuteUpdateAsync(s => s.SetProperty(p => p.Price, p => p.Price * 1.1m));
```

496. What are EF Core interceptors and what can they do?

Advanced | #interceptors #savechanges #extensibility | reference

Interceptors let you hook into EF Core operations such as SaveChanges, query execution, connection open/close, and transaction events by implementing interfaces like ISaveChangesInterceptor or IDbCommandInterceptor. They are registered with AddInterceptors on the context options and can inspect or modify commands, add audit fields, or log timing. Interceptors are more powerful and lower-level than change-tracking events because they can rewrite SQL and control execution, and multiple interceptors can be composed in order.

```
public class AuditInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData e, InterceptionResult<int> r, CancellationToken ct)
    {
        // stamp CreatedAt/UpdatedAt
        return base.SavingChangesAsync(e, r, ct);
    }
}
```

497. What are owned entities and value converters, and how do they differ?

Advanced | #owned-entities #value-converters #modeling | reference reference

Owned entity types are configured with OwnsOne or OwnsMany and map a related CLR type as part of the owner, either in the same table (table splitting) or a separate table, giving them an identity tied to the owner. Value converters map a property's CLR type to a different provider type, for example storing an enum as a string or a strongly typed id as a GUID, using HasConversion. Owned types model structure and relationships, while value converters just change how a single column is persisted.

```
b.Entity<Customer>().OwnsOne(c => c.Address);

b.Entity<User>()
    .Property(u => u.Status)
    .HasConversion<string>();
```

498. How does connection resiliency work with EnableRetryOnFailure?

Advanced | #resiliency #retry #execution-strategy | reference

EnableRetryOnFailure configures an execution strategy that automatically retries operations that fail with transient database errors such as temporary network drops or deadlocks, using a configurable retry count and delay. Because retries can re-execute a unit of work, you should wrap user-initiated transactions in strategy.ExecuteAsync so the whole transaction is retried safely. Retrying is not appropriate for non-idempotent operations without care, and you should combine it with logging to detect persistent failures.

```
        builder.Services.AddDbContext<AppDbContext>(o =>
            o.UseSqlServer(conn,
                sql => sql.EnableRetryOnFailure(
                    maxRetryCount: 5,
                    maxRetryDelay: TimeSpan.FromSeconds(10),
                    errorNumbersToAdd: null)));
```

499. What performance best practices should you apply with EF Core?

Advanced | #performance #best-practices #optimization | reference

Read-only queries should use AsNoTracking, and you should project only the columns and rows you need with Select rather than loading full entities. Avoid the N+1 problem with Include or projections, use AsSplitQuery when including multiple collections, and prefer ExecuteUpdate/ExecuteDelete for bulk changes. Keep DbContext instances short-lived, add indexes for frequently filtered and joined columns, enable compiled queries on hot paths, and measure with logging or a profiler instead of guessing.

500. Should you use the repository and unit-of-work patterns with EF Core?

Advanced | #repository #unit-of-work #architecture | reference reference

DbContext already implements both the unit-of-work pattern through its change tracker and SaveChanges and a repository-like abstraction through DbSet, so wrapping it in an extra generic repository often adds indirection without value. A custom repository can still be worthwhile to centralize complex queries, enforce domain invariants, or abstract persistence for testing and multiple data sources. If you use one, keep it thin, expose IQueryable sparingly to avoid leaking EF-specific behavior, and avoid sharing a DbContext across threads.